

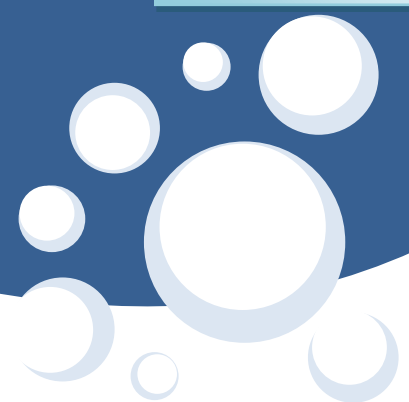
Autor: Rafael Alberto Moreno Parra



# Iniciando con **JAVASCRIPT**



**2026**



Contenido

Contenido ..... 2

Tabla de Ilustraciones..... 7

Otros libros del autor ..... 9

Página web del autor y canal en Youtube ..... 9

Sitio en GitHub ..... 9

Licencia de este libro ..... 9

Licencia del software ..... 9

Introducción ..... 10

Iniciando ..... 11

    Navegadores ..... 12

    Editores ..... 12

    Código fuente ..... 12

    Preparando el entorno para trabajar con JavaScript..... 12

    Probando que el servidor de aplicaciones funciona bien..... 16

Extensiones útiles en Visual Studio Code ..... 17

    ESLint..... 18

    Prettier ..... 18

    JavaScript (ES6) code snippets ..... 18

    GitLens ..... 18

    Live Server..... 19

    GitHub Copilot ..... 19

Capítulo 1: Fundamentos ..... 21

    Iniciando..... 22

    Comentarios en el código ..... 23

    ¿Dónde guardar los archivos HTML? ..... 24

    El tradicional “Hola Mundo” ..... 25

    Uso de variables ..... 26

    Operaciones matemáticas ..... 27

    Asignación ..... 28

    Operaciones con cadenas ..... 29

    Impresión de caracteres propios del idioma español ..... 31

    Constantes matemáticas ..... 32

    Funciones trigonométricas..... 33

    Funciones Matemáticas ..... 34

    Captura de datos tipo texto por pantalla ..... 35

    Captura de datos tipo numérico por pantalla..... 36

    Ejemplos de cálculos ..... 37

    Manejo de errores con números ..... 38

        isNaN..... 38

        isFinite..... 39

        Try...catch ..... 40

    Uso de typeof para detectar el tipo de dato de una variable..... 41

    Operaciones de bit ..... 42

        Convertir un entero a su representación binaria ..... 42

        Operaciones OR, AND y XOR directas..... 43

        Multiplicar usando operaciones de bit..... 44

        Dividir usando operaciones de bit ..... 45

        Intercambiar valores usando operaciones de bit..... 46

    Corrigiendo errores en JavaScript..... 47

Capítulo 2.    Si Condicional ..... 48

|                                                                            |     |
|----------------------------------------------------------------------------|-----|
| Si Condicional .....                                                       | 49  |
| Operadores lógicos: Conjunción (Y, &&) y Disyunción (O,   ) .....          | 51  |
| Uso del switch .....                                                       | 52  |
| Capítulo 3. Ciclos .....                                                   | 53  |
| Ciclo for .....                                                            | 54  |
| Ciclo while.....                                                           | 56  |
| Ciclo do - while .....                                                     | 59  |
| Uso del break .....                                                        | 62  |
| Uso del continue .....                                                     | 63  |
| Ciclos anidados y salir de estos .....                                     | 64  |
| Números aleatorios .....                                                   | 65  |
| Distribución Normal.....                                                   | 66  |
| Distribución Triangular.....                                               | 67  |
| Distribución Uniforme .....                                                | 68  |
| Capítulo 4. Funciones o subrutinas .....                                   | 69  |
| Ejemplos de uso de la librería .....                                       | 75  |
| Método de Bisección.....                                                   | 79  |
| Ámbito (scope) de las variables y funciones .....                          | 80  |
| Funciones recursivas .....                                                 | 82  |
| Uso de eval como evaluador de expresiones algebraicas .....                | 84  |
| Funciones que reciben distinto número de parámetros.....                   | 85  |
| Instrucciones que ejecutan controlando el tiempo .....                     | 86  |
| Capítulo 5. Arreglos unidimensionales o vectores .....                     | 88  |
| Borrar elementos con splice( ) .....                                       | 91  |
| Operaciones con arreglos .....                                             | 92  |
| Ordenación de arreglo unidimensional .....                                 | 93  |
| Funciones genéricas para arreglos.....                                     | 94  |
| Retornar arreglos desde funciones.....                                     | 95  |
| Implementación de algoritmos de ordenación .....                           | 95  |
| Algoritmo de burbuja.....                                                  | 95  |
| Algoritmo de ordenación por selección.....                                 | 97  |
| Algoritmo de ordenación por inserción.....                                 | 98  |
| Algoritmo de ordenación QuickSort.....                                     | 99  |
| Algoritmo de ordenación MergeSort .....                                    | 100 |
| Manejo de Fechas .....                                                     | 101 |
| Capítulo 6. Manejo de cadenas o strings .....                              | 102 |
| Ejemplos de programas usando cadenas .....                                 | 105 |
| Invertir Cadena .....                                                      | 105 |
| Quitar los espacios de una cadena .....                                    | 106 |
| Quitar las vocales de una cadena .....                                     | 107 |
| Quitar caracteres no permitidos .....                                      | 108 |
| Un sencillo cifrado / descifrado.....                                      | 109 |
| Un sencillo cifrado / descifrado con clave de cifrado.....                 | 110 |
| Sumar largos números enteros almacenados en cadenas .....                  | 111 |
| Capítulo 7. Arreglos bidimensionales .....                                 | 112 |
| Con números.....                                                           | 113 |
| Con caracteres.....                                                        | 114 |
| Función genérica .....                                                     | 115 |
| Funciones genéricas .....                                                  | 116 |
| Ejemplos de programas usando arreglos bidimensionales.....                 | 117 |
| Poner una torre al azar en un tablero de ajedrez y mostrar su ataque ..... | 117 |
| Poner un Rey al azar en un tablero de ajedrez y mostrar su ataque .....    | 118 |

|                                                                                   |     |
|-----------------------------------------------------------------------------------|-----|
| Poner un alfil al azar en un tablero de ajedrez y mostrar su ataque.....          | 120 |
| Poner una Reina al azar en un tablero de ajedrez y mostrar su ataque .....        | 122 |
| Ruta del menor costo .....                                                        | 123 |
| Resolver Sudokus .....                                                            | 125 |
| Poner los barcos en el juego batalla naval.....                                   | 127 |
| Capítulo 8. Programación orientada a objetos .....                                | 129 |
| Definiendo clases, constructores y atributos .....                                | 130 |
| Definiendo métodos.....                                                           | 131 |
| Métodos y atributos.....                                                          | 132 |
| Atributos y métodos privados .....                                                | 133 |
| Getters y Setters .....                                                           | 134 |
| Herencia .....                                                                    | 135 |
| Herencia y constructor.....                                                       | 136 |
| Sobrecarga de métodos .....                                                       | 137 |
| Capítulo 9. Estructuras de Datos.....                                             | 138 |
| Árbol binario.....                                                                | 139 |
| Recorrido en pre-orden.....                                                       | 140 |
| Recorrido en in-orden.....                                                        | 141 |
| Recorrido en post-orden .....                                                     | 142 |
| Ordenación usando un árbol binario .....                                          | 143 |
| Uso de JSON .....                                                                 | 144 |
| Capítulo 10. Control sobre la página HTML .....                                   | 148 |
| Captura el evento de dar clic en un <div> .....                                   | 151 |
| Captura el evento de dar doble clic en un <div> .....                             | 153 |
| Captura el evento de pasar el cursor del ratón por encima de un <div> .....       | 153 |
| Captura el evento de mover el cursor del ratón fuera del <div> .....              | 154 |
| Captura el evento apenas cargue la página .....                                   | 155 |
| Captura el evento cuando de clic al interior del <div> .....                      | 156 |
| Captura el evento al soltar el botón del ratón dentro del <div> .....             | 157 |
| Captura el evento de mover el cursor del ratón.....                               | 158 |
| Captura el evento cuando se cambia el tamaño de la ventana.....                   | 159 |
| Captura el evento de dar "submit" en un formulario .....                          | 160 |
| Captura el evento de entrar a la caja de texto .....                              | 161 |
| Captura el evento al salir de una caja de texto.....                              | 162 |
| Captura el evento al presionar una tecla dentro de una caja de texto .....        | 163 |
| Captura el evento cuando suelta la tecla .....                                    | 164 |
| Captura el evento de cambio del texto en una caja de texto .....                  | 165 |
| Mostrar el código de la tecla presionada.....                                     | 166 |
| Pregunta antes de ir a la página enlazada .....                                   | 167 |
| Capítulo 11. DOM (Document Object Model).....                                     | 168 |
| Cambiar atributos de un objeto.....                                               | 169 |
| Cambiar el color de fondo de un <div> .....                                       | 170 |
| Hacer visible o no un <div> .....                                                 | 171 |
| Poner o quitar texto de un <div> .....                                            | 172 |
| Cambiar el aspecto del objeto cuando está seleccionado .....                      | 173 |
| Validar valores al saltar a otro objeto .....                                     | 174 |
| Contar objetos.....                                                               | 175 |
| Obtener lista de objetos.....                                                     | 176 |
| Obtener determinado objeto de una lista.....                                      | 177 |
| Obtener lista de determinado tipo de objetos y ver el valor de sus atributos..... | 178 |
| Obtener la lista de determinado tipo de objetos y cambiar sus atributos .....     | 179 |

|                                                                             |     |
|-----------------------------------------------------------------------------|-----|
| Mostrar la ubicación del documento .....                                    | 180 |
| Cambiar el estilo CSS de un determinado objeto .....                        | 181 |
| Mostrar la codificación del documento.....                                  | 182 |
| Mostrar el título del documento.....                                        | 183 |
| Salto de línea sin usar <br> .....                                          | 184 |
| Mostrar la fecha de modificación del documento.....                         | 185 |
| Mostrar el tipo de documento .....                                          | 186 |
| Mostrar el dominio del documento.....                                       | 187 |
| Mostrar el estado del documento.....                                        | 188 |
| Mostrar las dimensiones del documento .....                                 | 189 |
| Mostrar el número de enlaces .....                                          | 190 |
| Mostrar hacia donde apuntan los enlaces .....                               | 191 |
| Crear botones en tiempo de ejecución .....                                  | 192 |
| Crear <p> en tiempo de ejecución .....                                      | 193 |
| Crear nuevos <p> y contarlos en tiempo de ejecución.....                    | 194 |
| Contar el número de objetos en el documento .....                           | 195 |
| Contar el número de formularios en el documento .....                       | 196 |
| Contar el número de imágenes del documento.....                             | 197 |
| Describir los objetos de la página .....                                    | 198 |
| Abrir una ventana .....                                                     | 199 |
| Abrir un conjunto de ventanas .....                                         | 200 |
| Un formulario que ejecuta un algoritmo local al enviar la información ..... | 201 |
| Valida un correo con una expresión regular .....                            | 202 |
| Valida una URL con expresiones regulares.....                               | 203 |
| Capítulo 12. Gráficos .....                                                 | 204 |
| Dibujar líneas .....                                                        | 206 |
| Cambiar el grosor de una línea.....                                         | 207 |
| Dar un color determinado a una línea .....                                  | 208 |
| Color RGB .....                                                             | 209 |
| Estilos para finalizar las líneas .....                                     | 210 |
| Dibujar varias líneas.....                                                  | 211 |
| Juntar líneas.....                                                          | 212 |
| Probando los otros estilos de juntar líneas.....                            | 213 |
| Varias líneas.....                                                          | 214 |
| Genera una figura cerrada y la rellena .....                                | 215 |
| Dibujar varias líneas horizontales con un ciclo .....                       | 216 |
| Dibujar varias líneas verticales con un ciclo .....                         | 217 |
| Ilusión de curva con varias líneas.....                                     | 218 |
| Dibujar círculos.....                                                       | 220 |
| Dibujar curvas .....                                                        | 221 |
| Curva de Bézier .....                                                       | 222 |
| Combinando curvas.....                                                      | 223 |
| Rectángulo y gradiente de color .....                                       | 224 |
| Variando en forma oblicua con varios colores.....                           | 225 |
| Variando en forma vertical .....                                            | 226 |
| Variando en forma horizontal.....                                           | 227 |
| Variando en círculos .....                                                  | 228 |
| Trabajando con imágenes .....                                               | 229 |
| Variar posición y tamaño de la imagen.....                                  | 230 |
| Convertir imagen a escala de grises .....                                   | 231 |

Dibujar letras .....232

Sombras .....235

Semitransparente .....236

Capítulo 13. Librería Gráfica .....237

Referencias .....258



## Tabla de Ilustraciones

|                                                                                                       |     |
|-------------------------------------------------------------------------------------------------------|-----|
| Ilustración 1: Lenguajes de programación para aplicaciones Web en el lado del cliente .....           | 10  |
| Ilustración 2: Sitio web para la descarga de XAMPP.....                                               | 12  |
| Ilustración 3: Pantalla de advertencia sobre el User Account Control .....                            | 13  |
| Ilustración 4: Solo Apache, MySQL, PHP y phpMyAdmin.....                                              | 13  |
| Ilustración 5: El directorio donde se instalará XAMPP.....                                            | 13  |
| Ilustración 6: Apache HTTP Server requiere permisos del Firewall, se presiona "Permitir acceso" ..... | 14  |
| Ilustración 7: Finaliza la instalación .....                                                          | 14  |
| Ilustración 8: Panel de control de XAMPP, los servicios están inactivos. ....                         | 14  |
| Ilustración 9: Iniciando el servidor web Apache. ....                                                 | 15  |
| Ilustración 10: MySQL requiere permisos en el firewall, clic en "Permitir acceso". ....               | 15  |
| Ilustración 11: Los dos servicios están activos.....                                                  | 15  |
| Ilustración 12: ¿Apache funciona? abre un navegador y se escribe http://localhost. ....               | 16  |
| Ilustración 13: ESLint.....                                                                           | 18  |
| Ilustración 14: Prettier .....                                                                        | 18  |
| Ilustración 15: JavaScript (ES6) code snippets .....                                                  | 18  |
| Ilustración 16: GitLens .....                                                                         | 19  |
| Ilustración 17: Live Server.....                                                                      | 19  |
| Ilustración 18: GitHub Copilot .....                                                                  | 19  |
| Ilustración 22: Imprimir caracteres propios del idioma español.....                                   | 31  |
| Ilustración 23: En Chrome puede pasar que no se impriman .....                                        | 31  |
| Ilustración 24: Funciona en Mozilla Firefox .....                                                     | 31  |
| Ilustración 28: Ante errores en JavaScript, el navegador queda en blanco.....                         | 47  |
| Ilustración 29: Presionando F12 en el navegador Edge y dando clic en Consola se muestra el error..... | 47  |
| Ilustración 30: Sudoku resuelto .....                                                                 | 126 |
| Ilustración 31: Árbol binario .....                                                                   | 140 |
| Ilustración 32: Lienzo ("canvas") visible usando un borde .....                                       | 205 |
| Ilustración 33: Línea dibujada .....                                                                  | 206 |
| Ilustración 34: Línea gruesa .....                                                                    | 207 |
| Ilustración 35: Color a la línea .....                                                                | 208 |
| Ilustración 36: Línea con color modificando los parámetros RGB .....                                  | 209 |
| Ilustración 37: Las líneas finalizan redondeadas en ambos extremos usando "round" .....               | 210 |
| Ilustración 38: Tres líneas con la misma longitud, pero diferente cierre de extremo .....             | 211 |
| Ilustración 39: Dos líneas juntas con un empalme redondeado .....                                     | 212 |
| Ilustración 40: Diferentes formas de juntar dos líneas: "round", "miter", "bevel" .....               | 213 |
| Ilustración 41: Una figura cerrada.....                                                               | 214 |
| Ilustración 42: Figura cerrada y rellena de un color .....                                            | 215 |
| Ilustración 43: Líneas generadas por un ciclo .....                                                   | 216 |
| Ilustración 44: Líneas verticales dibujadas con un ciclo.....                                         | 217 |
| Ilustración 45: Ilusión de curva usando líneas rectas.....                                            | 218 |
| Ilustración 46: Ilusión de dos curvas usando líneas rectas .....                                      | 219 |
| Ilustración 47: La manera en que se dibujará el círculo .....                                         | 220 |
| Ilustración 48: Círculo .....                                                                         | 220 |
| Ilustración 49: Forma en que se dibujarán las curvas .....                                            | 221 |
| Ilustración 50: Curva.....                                                                            | 221 |
| Ilustración 51: Curva Bézier .....                                                                    | 222 |
| Ilustración 52: Combinando curvas .....                                                               | 223 |
| Ilustración 53: Variación del color .....                                                             | 224 |
| Ilustración 54: Variación de varios colores en forma oblicua .....                                    | 225 |
| Ilustración 55: Variación de color en forma vertical .....                                            | 226 |
| Ilustración 56: Variación de color horizontal .....                                                   | 227 |
| Ilustración 57: Variación en círculos .....                                                           | 228 |
| Ilustración 59: Imagen original.....                                                                  | 231 |
| Ilustración 60: Imagen en escala de grises .....                                                      | 231 |
| Ilustración 61: Dibuja un texto .....                                                                 | 232 |
| Ilustración 62: Dibuja un texto con relleno de color.....                                             | 232 |
| Ilustración 63: Texto dibujado, se muestra el borde de cada letra .....                               | 233 |
| Ilustración 64: Dibuja un texto alineado al final .....                                               | 233 |
| Ilustración 65: Muestra el ancho del texto dibujado.....                                              | 234 |
| Ilustración 66: Sombra .....                                                                          | 235 |
| Ilustración 67: Rectángulo semitransparente.....                                                      | 236 |
| Ilustración 68: Uso de librería gráfica .....                                                         | 251 |
| Ilustración 69: Uso de librería gráfica .....                                                         | 252 |
| Ilustración 70: Uso de librería gráfica .....                                                         | 253 |
| Ilustración 71: Uso de librería gráfica .....                                                         | 254 |

Ilustración 72: Uso de librería gráfica .....255

Ilustración 73: Uso de librería gráfica .....256

Ilustración 74: Uso de librería gráfica .....257



## Otros libros del autor

Libro **C# y .NET 10. 2026** <https://github.com/ramsoftware/C-Sharp>: Dividido en varias partes, cada uno es un libro

- Parte 1: IDE, variables, ciclos, si condicional, funciones
- Parte 2: Cadenas (strings)
- Parte 3: Arreglos estáticos
- Parte 4: Programación Orientada a Objetos
- Parte 5: Estructuras de datos dinámicas
- Parte 6: LINQ y Lambda
- Parte 7: Un evaluador de expresiones algebraicas
- Parte 8: Estructuras de datos de bajo nivel
- Parte 9: Simulaciones
- Parte 10: Algoritmos evolutivos
- Parte 11: Redes Neuronales
- Parte 12: Gráficos 2D
- Parte 13: Gráficos en 3D
- Parte 14: Animación
- Parte 15: Imágenes

Libro "Evaluador de expresiones en nueve lenguajes de programación. En C#, C++, Delphi, Java, JavaScript, PHP, Python, TypeScript y Visual Basic .NET". En Colombia 2021. Págs. 158. Libro y código fuente descargable en: <https://github.com/ramsoftware/Evaluador3>

Libro "Segunda parte de uso de algoritmos genéticos para la búsqueda de patrones". En Colombia 2015. Págs. 303. En publicación por la Universidad Libre – Cali.

Libro "Un uso de algoritmos genéticos para la búsqueda de patrones". En Colombia 2013. En publicación por la Universidad Libre – Cali.

Libro "Desarrollo de gráficos para PC, Web y dispositivos móviles" En Colombia 2009. ed.: Artes Gráficas Del Valle Editores Impresores Ltda. ISBN: 978-958-8308-95-1 v. 1 págs. 317

Artículo: "Programación Genética: La regresión simbólica". Entramado ISSN: 1900-3803 ed.: Universidad Libre Seccional Cali v.3 fasc.1 p.76 - 85, 2007

## Página web del autor y canal en Youtube

Investigación sobre Vida Artificial: <http://darwin.50webs.com>

Canal en Youtube: <http://www.youtube.com/user/RafaelMorenoP> (dedicado principalmente al desarrollo en C#)

## Sitio en GitHub

El código fuente se puede descargar en <https://github.com/ramsoftware/>

## Licencia de este libro



## Licencia del software

Todo el software desarrollado aquí tiene licencia LGPL "Lesser General Public License"



Introducción

En el desarrollo de aplicaciones Web, en el lado del cliente, JavaScript [1] [2] es el lenguaje de programación con una aplastante mayoría. Sólo es ver el siguiente gráfico a enero de 2026 de cómo está repartido los lenguajes de programación en el cliente [3]:

El enlace es: [https://w3techs.com/technologies/overview/client\\_side\\_language/all](https://w3techs.com/technologies/overview/client_side_language/all)

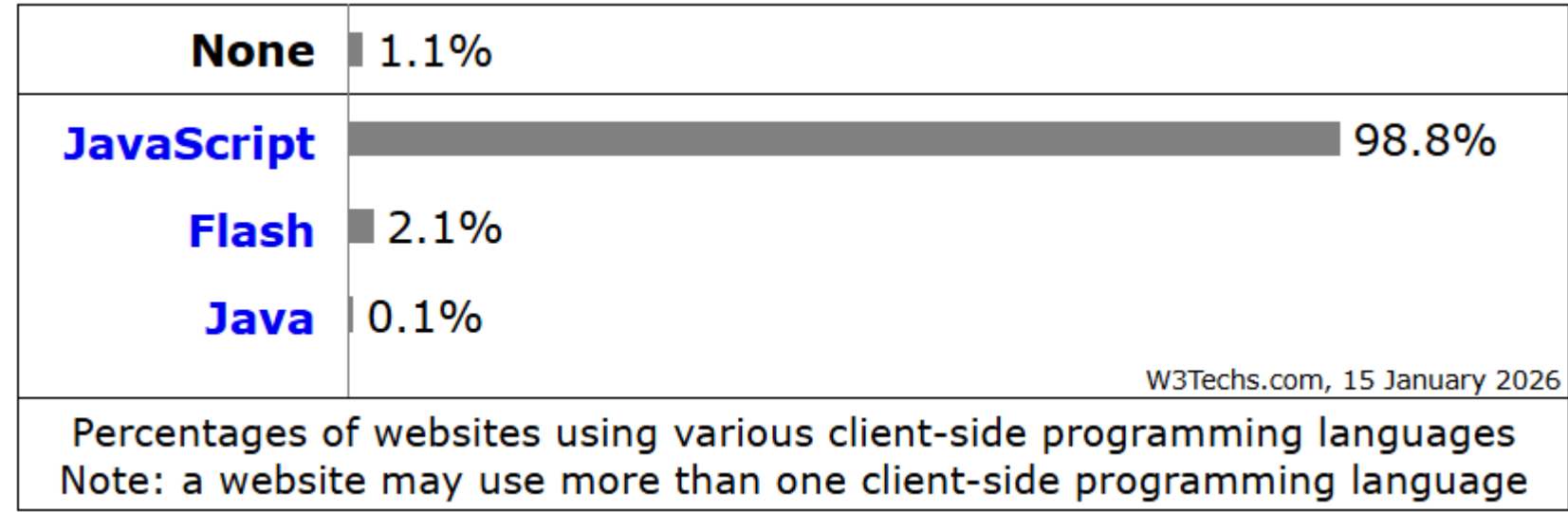


Ilustración 1: Lenguajes de programación para aplicaciones Web en el lado del cliente

JavaScript es el lenguaje de facto, además, se añade que Flash [4] es una tecnología que está abandonada por la empresa que la creó.

Es necesario programar en el lado del cliente para las siguientes tareas:

- 1. Validaciones de datos en el cliente, por ejemplo, que el usuario final sólo ingrese números en un determinado campo y evitar que ingrese letras.
- 2. Mostrar gráficos.
- 3. Mejorar la interactividad de la página web.
- 4. Ejecutar procesos en el lado del cliente.

El presente libro es sobre JavaScript basado en el estándar ECMAScript 2026 [5]

Iniciando

Empezar a programar en JavaScript es fácil y gratuito. Sólo es requerido tener un navegador moderno instalado en el PC (Microsoft Edge, Mozilla Firefox, Opera, Google Chrome, Brave, Vivaldi) y un editor de texto (recomendado Visual Studio Code que es gratuito). Este manual mostrará capturas de pantalla en el ambiente de Microsoft Windows 11, pero lo expuesto es aplicable en otros sistemas operativos porque JavaScript es multiplataforma.

## Navegadores

Enlace para Mozilla Firefox: <https://www.mozilla.org/es-ES/firefox/new/>

Enlace para Opera: <https://www.opera.com/es-419>

Enlace para Vivaldi: <https://vivaldi.com/es/>

Enlace para Brave: <https://brave.com/es/>

Debe considerar el soporte que da a JavaScript los navegadores, ver más en:

[https://www.w3schools.com/js/js\\_2026.asp](https://www.w3schools.com/js/js_2026.asp) [6]

## Editores

Enlace para Visual Studio Code: <https://code.visualstudio.com/>

## Código fuente

El código fuente se puede descargar de: <https://github.com/ramsoftware/JavaScript>

## Preparando el entorno para trabajar con JavaScript

Es posible trabajar con JavaScript directamente con el navegador: Se escribe el código de JavaScript en un archivo con extensión HTML y se ejecuta en el navegador, pero esta técnica conlleva el riesgo de que el desarrollador se equivoque con el uso de las rutas y que al final se le complique poder llevar el programa a una aplicación Web que es el objetivo final: Hacer una aplicación Web dinámica.

Luego la forma más recomendada para trabajar con JavaScript es instalando un servidor de aplicaciones, el cual tiene los siguientes componentes:

1. Un servidor Web [7]
2. Un gestor de base de datos [8]

En el caso del servidor Web se hace uso de Apache y el gestor de base de datos es MariaDB [9] o MySQL [10] (ambos son compatibles). Los dos son software libre, no hay que pagar un licenciamiento por usarlos.

Los dos: Apache y MariaDB/MySQL se pueden instalar por aparte, directamente de los sitios oficiales, pero hay una manera mucho más rápida y fácil y es usando una aplicación que instala esos componentes y los configura automáticamente. Uno muy conocido es XAMPP.

Instalar XAMPP es sencillo, se mostrará paso a paso desde su descarga hasta su instalación y ejecución. El primer paso es ir a <https://www.apachefriends.org/download.html> y descargar el instalador.

Instalar XAMPP es sencillo, se mostrará paso a paso desde su descarga hasta su instalación y ejecución.



Ilustración 2: Sitio web para la descarga de XAMPP

## Inicia la instalación

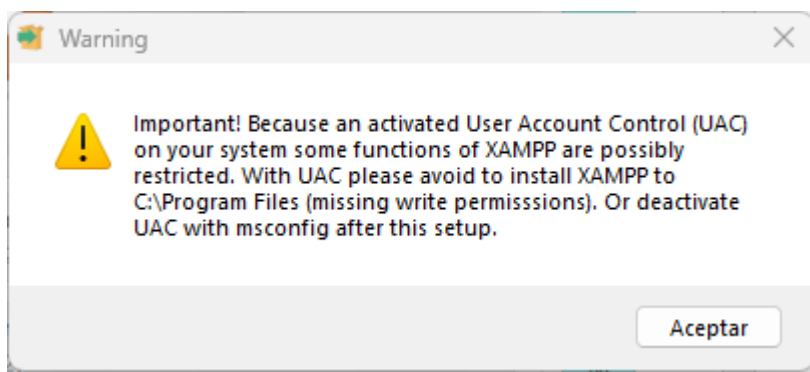


Ilustración 3: Pantalla de advertencia sobre el User Account Control

UAC significa Control de Cuentas de Usuario y es para que las aplicaciones ejecuten en un modo NO administrador del sistema. XAMPP requiere tener acceso de administrador, luego Windows 11 preguntará si va a ser instalarlo, se responde afirmativamente.

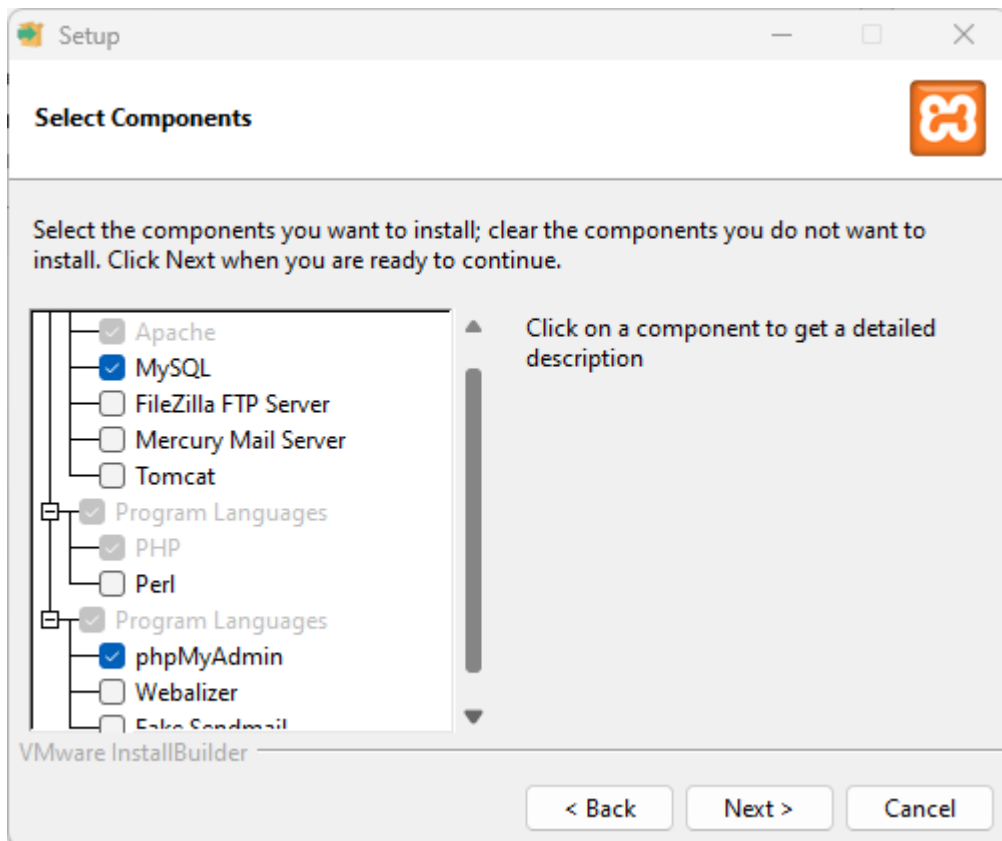


Ilustración 4: Solo Apache, MySQL, PHP y phpMyAdmin.

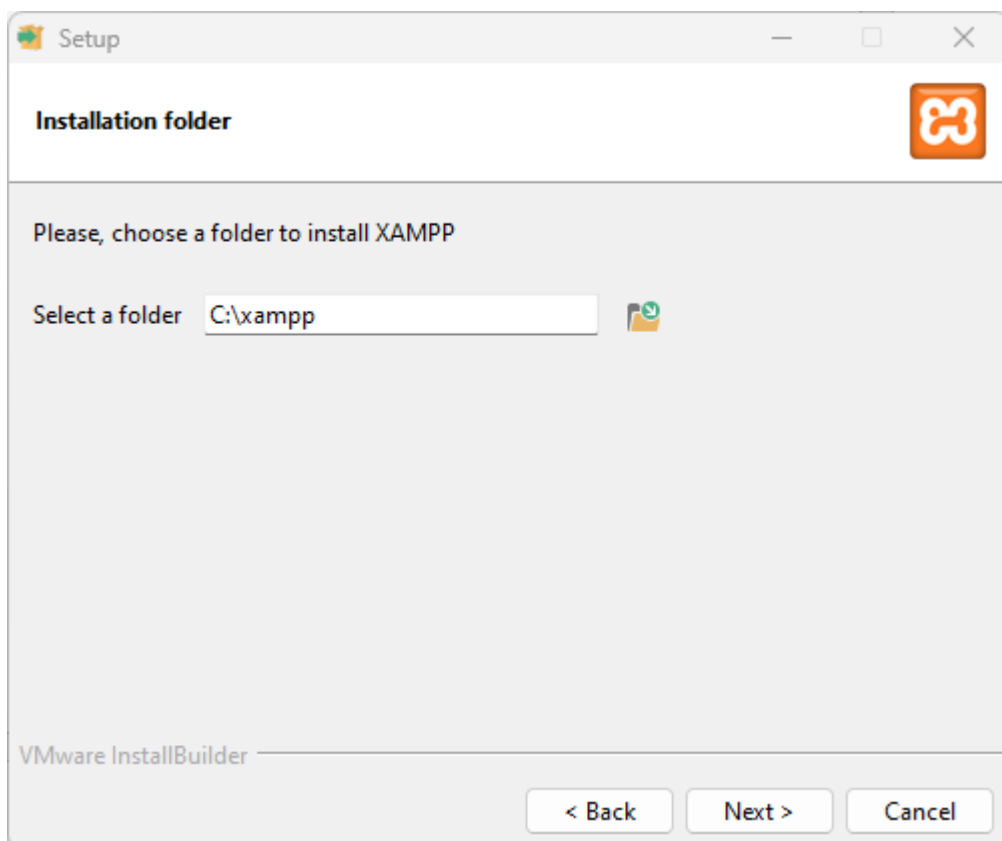


Ilustración 5: El directorio donde se instalará XAMPP.

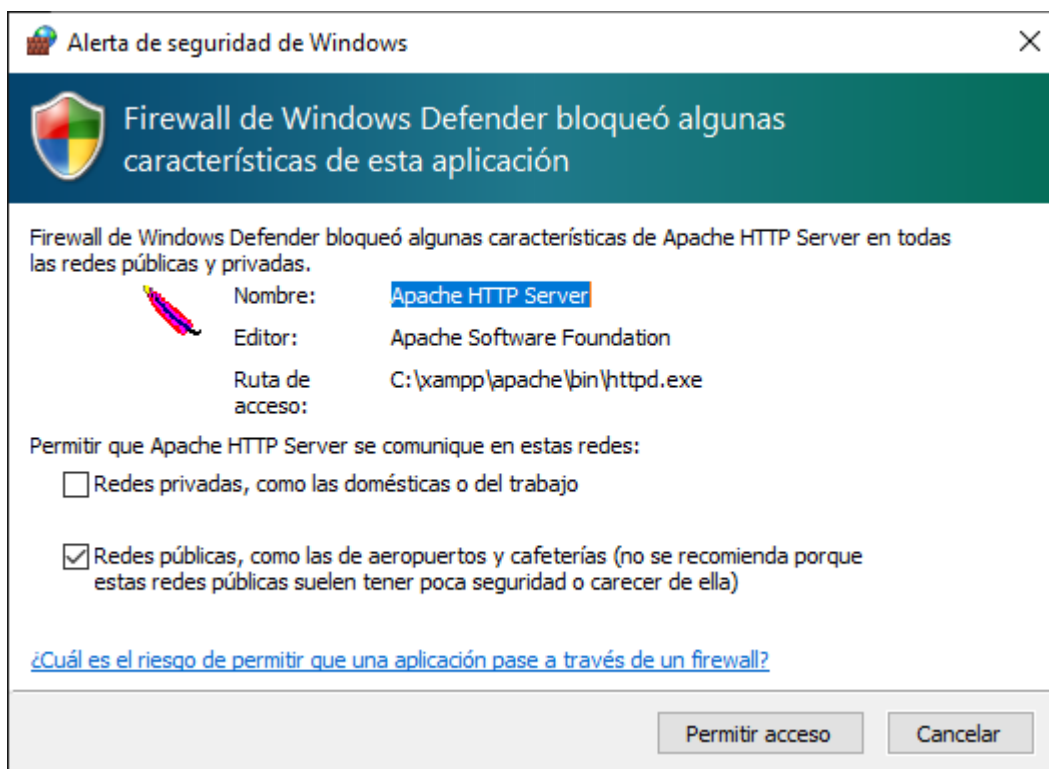


Ilustración 6: Apache HTTP Server requiere permisos del Firewall, se presiona "Permitir acceso"

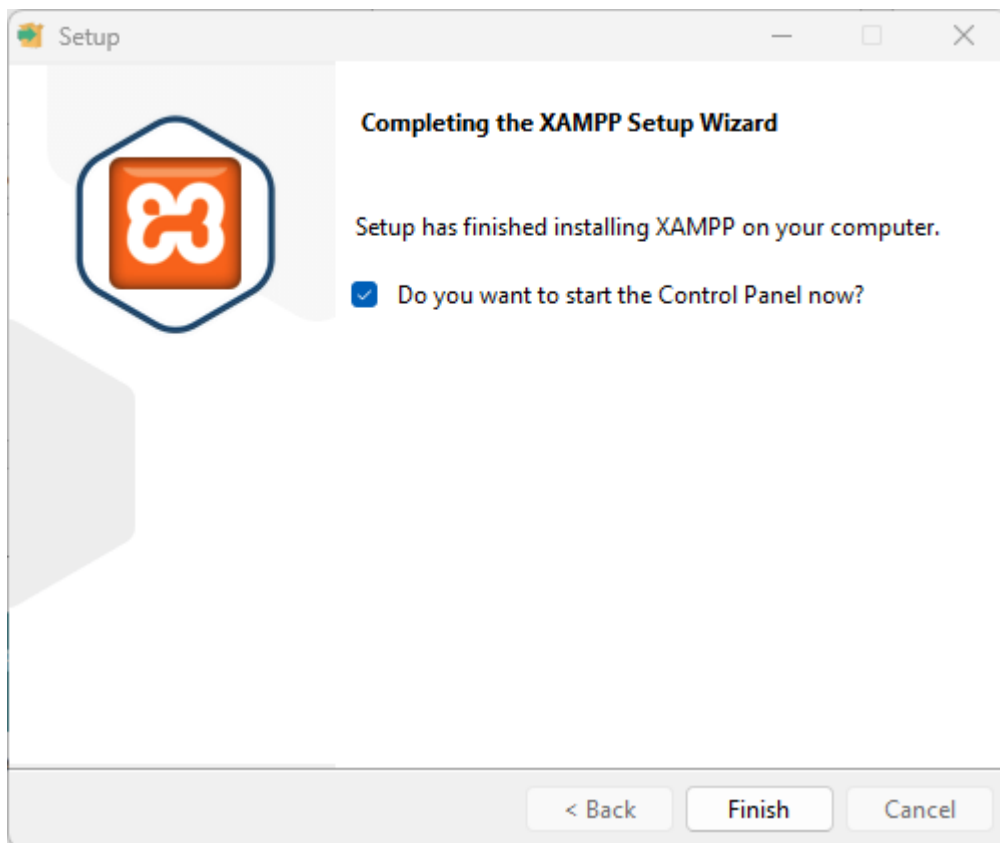


Ilustración 7: Finaliza la instalación

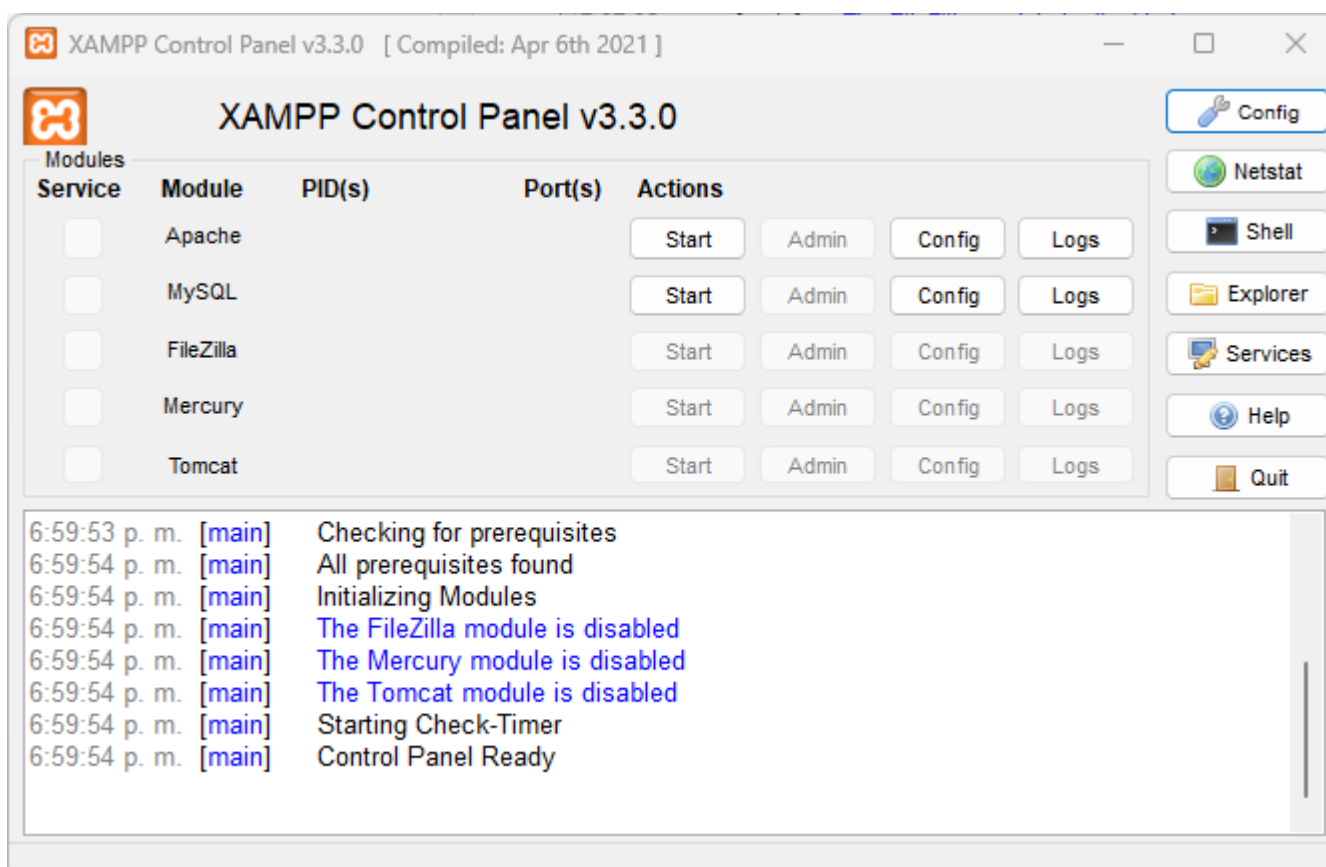


Ilustración 8: Panel de control de XAMPP, los servicios están inactivos.



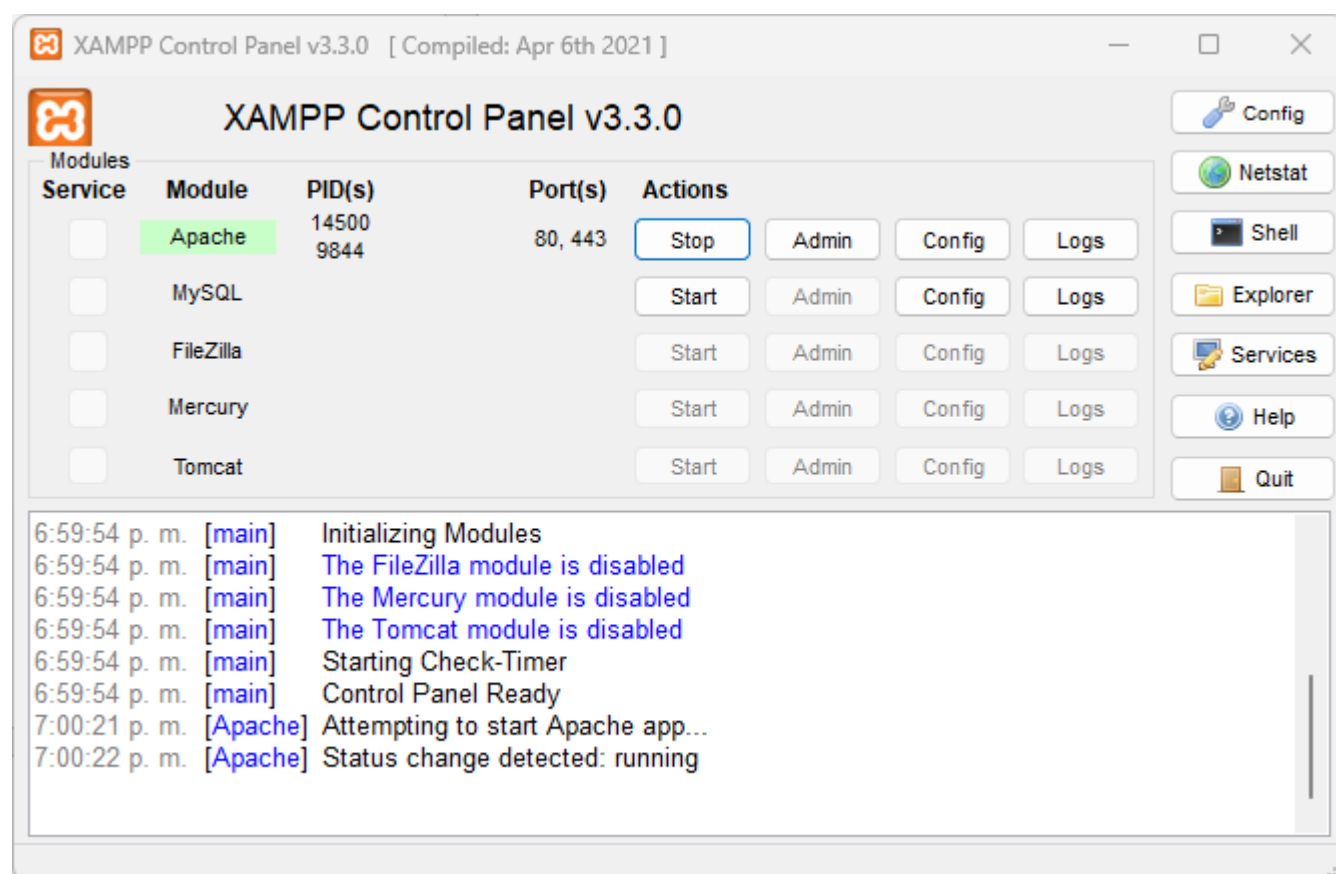


Ilustración 9: Iniciando el servidor web Apache.

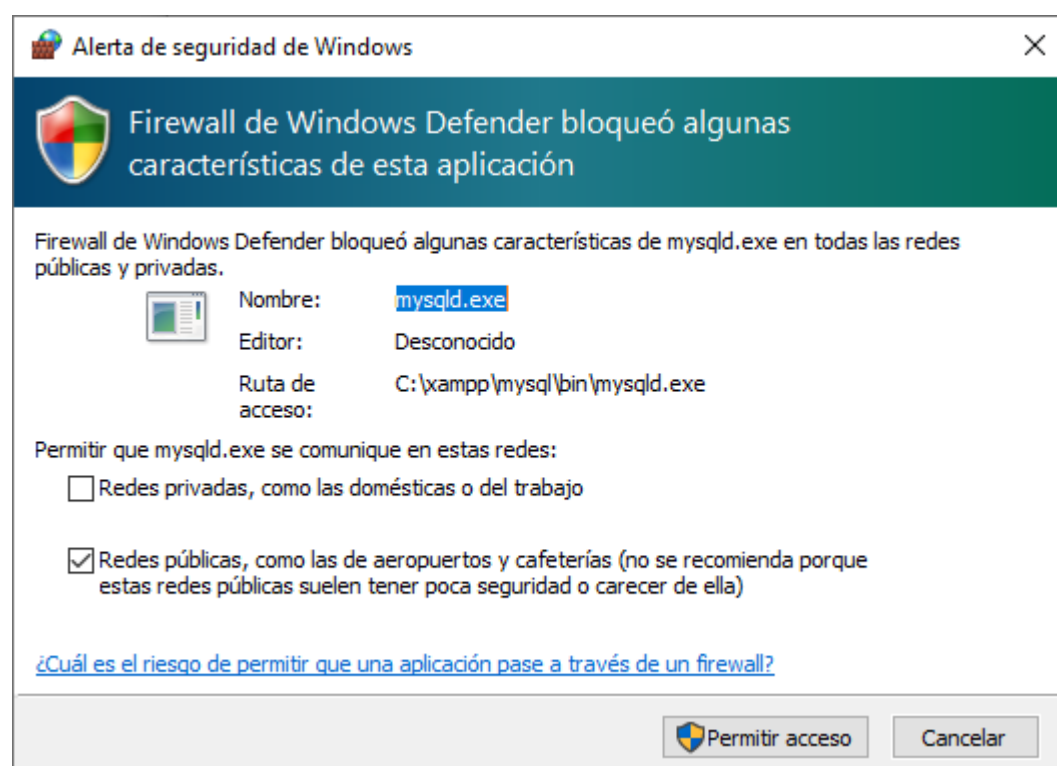


Ilustración 10: MySQL requiere permisos en el firewall, clic en "Permitir acceso".

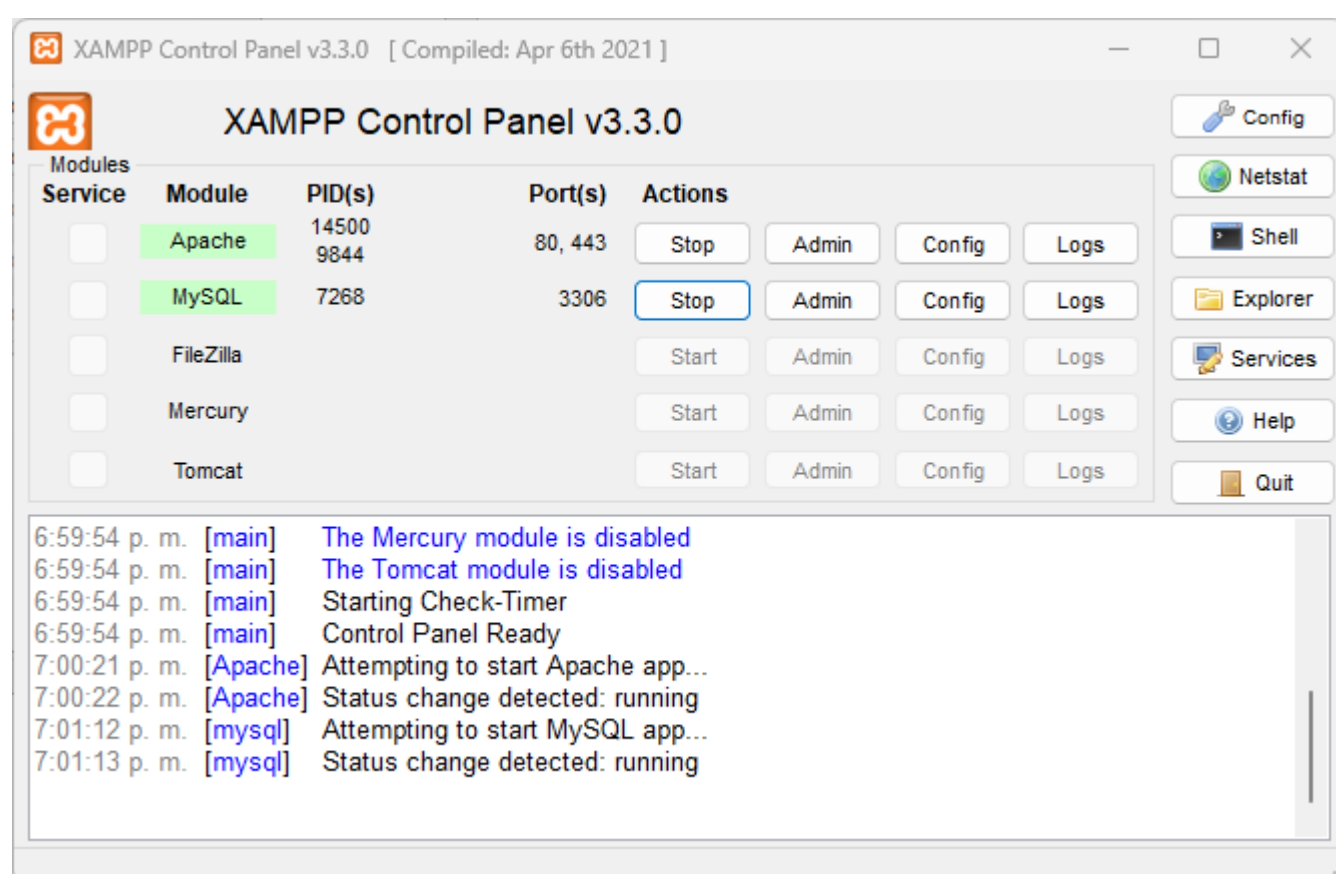


Ilustración 11: Los dos servicios están activos.



Probando que el servidor de aplicaciones funciona bien

Para saber si el servidor de aplicaciones funciona correctamente, se abre un navegador y en <http://localhost> , debería verse esta pantalla:

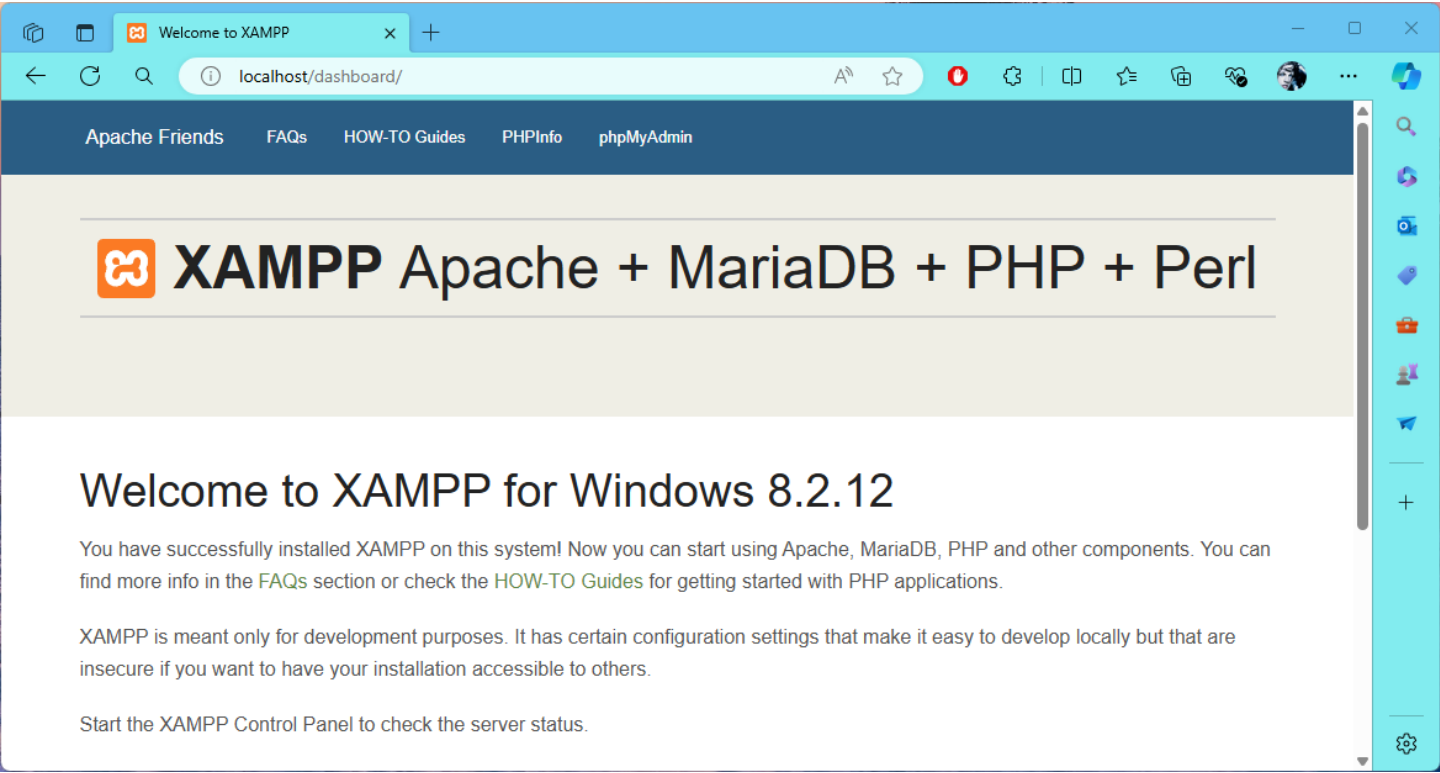


Ilustración 12: ¿Apache funciona? abre un navegador y se escribe <http://localhost>.



## Extensiones útiles en Visual Studio Code

**ESLint**  
Para mostrar errores de sintaxis, cuando no se siguen buenas prácticas, proveer sugerencias para mejorar código y mantener un estilo consistente en todo el código.

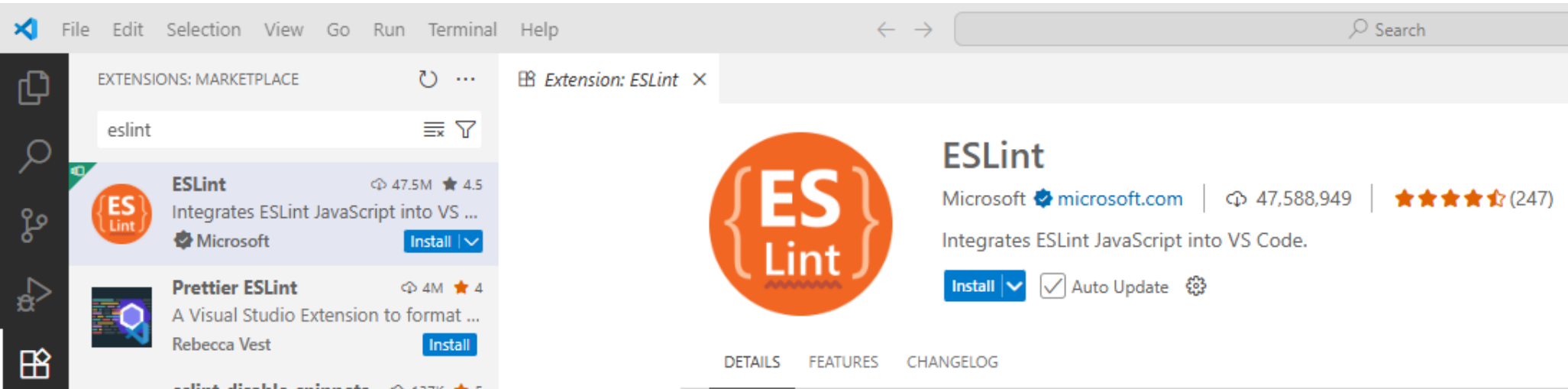


Ilustración 13: ESLint

**Prettier**  
Formateador automático de código.

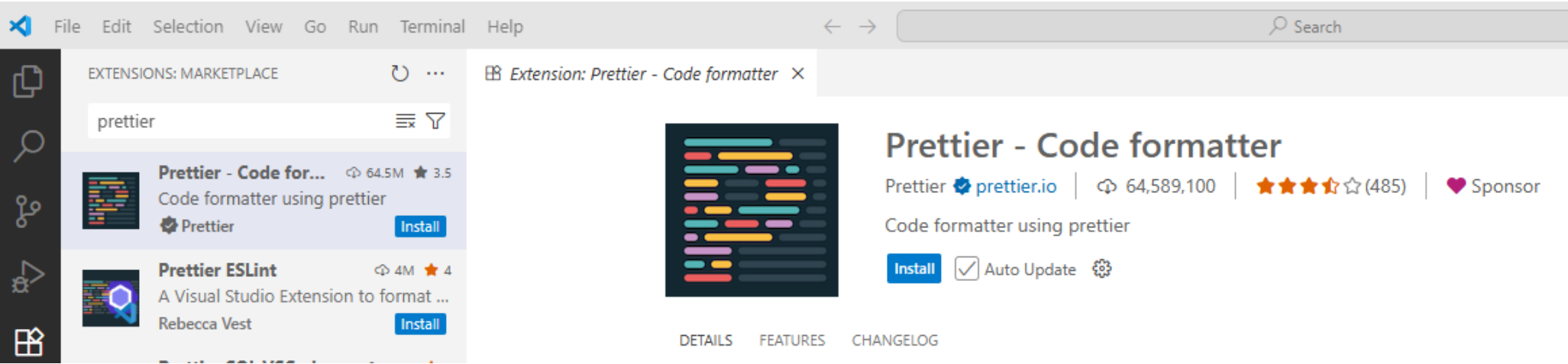


Ilustración 14: Prettier

**JavaScript (ES6) code snippets**  
JavaScript (ES6) code snippets son pequeños fragmentos de código escritos en JavaScript utilizando las características introducidas en la versión ES6 (ECMAScript 2015). Los snippets se usan como ejemplos rápidos, plantillas o bloques reutilizables para mostrar cómo aplicar una función, sintaxis o concepto sin necesidad de escribir un programa completo.

ECMAScript 2015, también conocido como ES6, es la sexta edición del estándar de JavaScript publicada en junio de 2015 por ECMA International. Fue una de las actualizaciones más importantes del lenguaje, porque introdujo muchas características modernas que hoy son la base del desarrollo en JavaScript. Aunque hoy existen versiones posteriores, ES6 es el punto de quiebre que transformó la forma de programar en JavaScript.

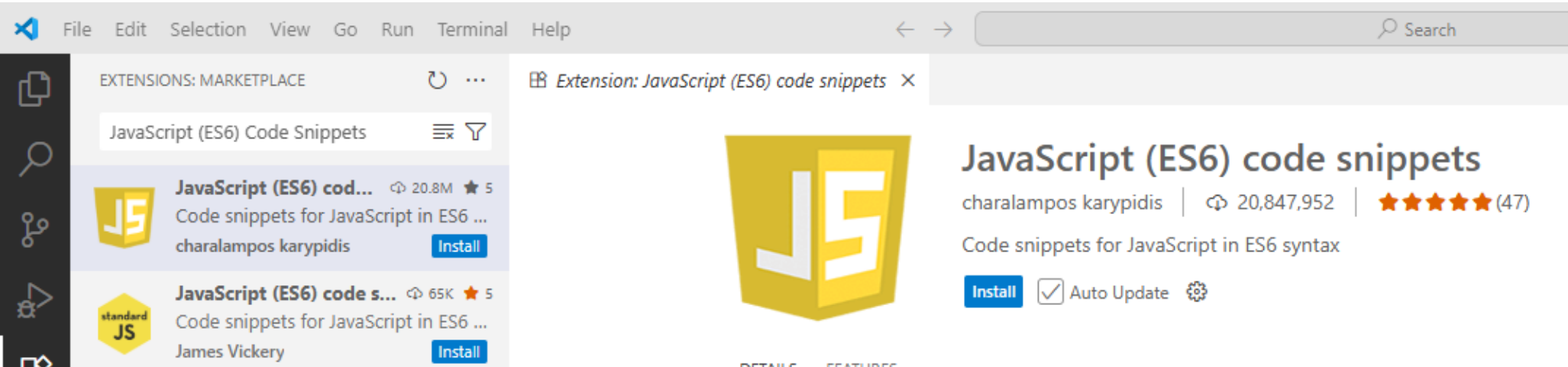


Ilustración 15: JavaScript (ES6) code snippets

**GitLens**  
Es una extensión gratuita y de código abierto que potencia el uso de Git dentro del editor. Permite visualizar fácilmente la autoría del código, explorar el historial de cambios y entender cómo y por qué evolucionó un repositorio.

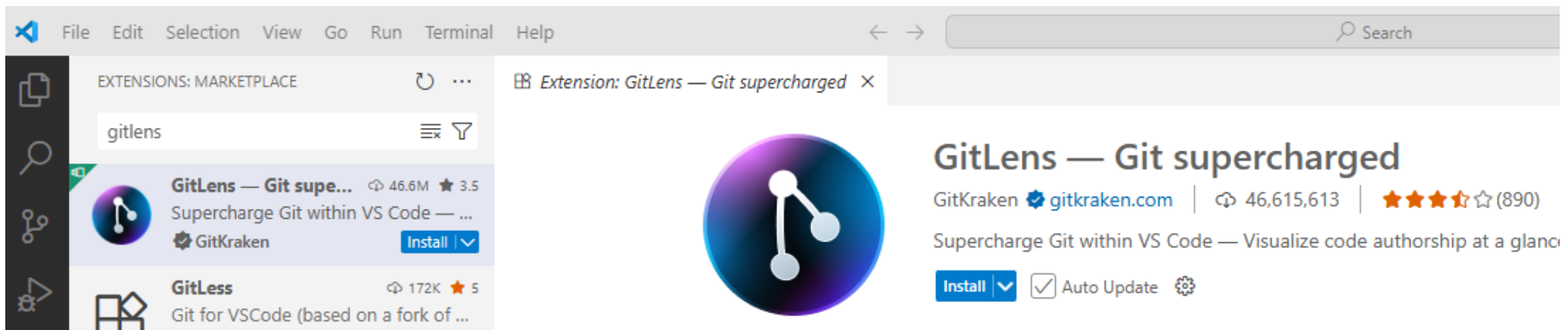


Ilustración 16: GitLens

## Live Server

Es una extensión gratuita que permite lanzar un servidor local para proyectos web y ver los cambios en tiempo real en el navegador sin necesidad de recargar manualmente la página. Es muy usada por desarrolladores front-end para trabajar con HTML, CSS y JavaScript de forma más ágil.

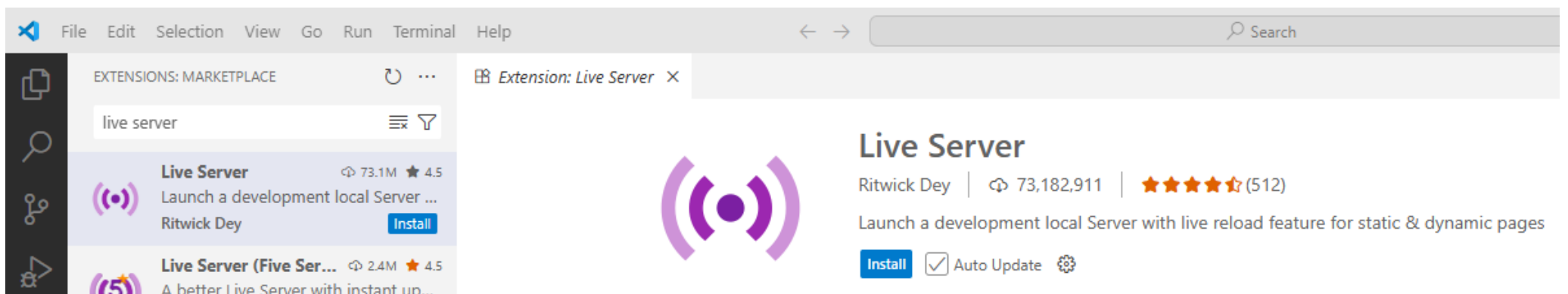


Ilustración 17: Live Server

## GitHub Copilot

Uso de inteligencia artificial para escribir código

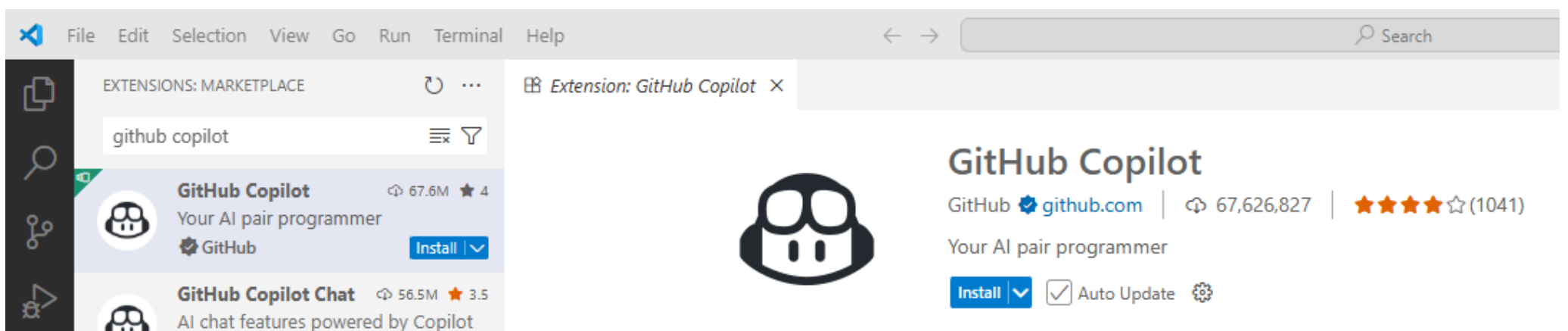
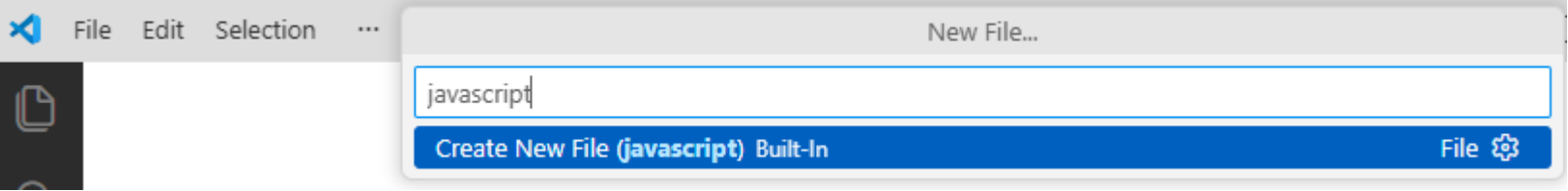
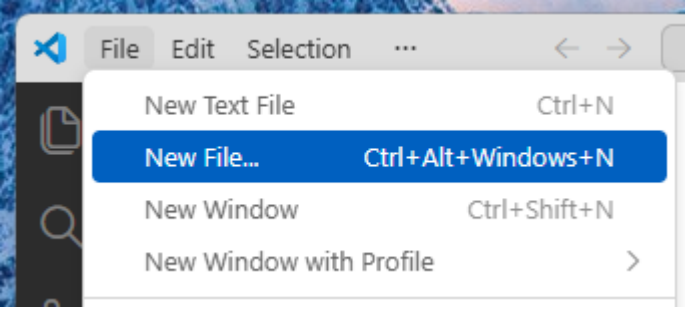


Ilustración 18: GitHub Copilot

Para escribir código fuente se ejecuta Visual Studio Code:



# Capítulo 1: Fundamentos

## Iniciando

Este es el esqueleto en HTML5. Lo mínimo necesario para hacer un programa en JavaScript:

Cap01/001.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>  
  
</script></body></html>
```

La etiqueta `<!DOCTYPE HTML>` informa al navegador que se usará la versión más reciente de HTML, es decir, HTML 5.

Cada etiqueta en HTML se abre y se cierra, ejemplo, `<html>` y su cierre `</html>`

El código en JavaScript se ubica entre `<script>` y su cierre `</script>`



## Comentarios en el código

Los comentarios son clave en la documentación del código, hacen la diferencia entre un software en que es posible hacer mantenimiento y software que hay que desechar. Similar a C++, si el comentario es de una sola línea, se usa // pero si se requieren varias líneas se puede encerrar entre /\* y \*/ . Ver el código:

Cap01/002.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Comentario de una sola línea

/* Comentarios en
   varias
   líneas */
</script></body></html>
```

### ¿Dónde guardar los archivos HTML?

Una vez instalado XAMPP y que esté ejecutando correctamente, se debe ir a **c:\xampp\htdocs** (o **d:\xampp\htdocs**) y crear una carpeta llamada **src**, en el interior de esa carpeta se ponen todos los archivos que descargó de GitHub.

Nota: Es muy recomendado hacer uso de minúsculas en el nombre de los archivos, también evitar los espacios, tildes y Ñs ¿Por qué? Porque muy seguramente la aplicación WEB futura se aloje en un servidor que tenga el sistema operativo Linux. En Linux se diferencia entre minúsculas y mayúsculas en el nombre de los archivos, el resto de las recomendaciones es para minimizar riesgos con el nombre de los archivos.

El siguiente paso es ejecutar la página Web, luego debe abrir el navegador y digitar <http://localhost> y luego ir al directorio src.



El tradicional “Hola Mundo”

La instrucción `document.getElementById("idObjetoHTML").textContent = "que quiere mostrar";` es para mostrar mensajes en el navegador en el interior de un **objeto HTML** que se identifica con el `id = "idObjetoHTML"`.

Las instrucciones deben terminar en punto y coma (;)

Nota: En JavaScript, el punto y coma (;) no es estrictamente necesario al final de cada línea porque el lenguaje tiene una característica llamada Inserción Automática de Punto y Coma (Automatic Semicolon Insertion, o ASI). Esto significa que el intérprete intentará adivinar dónde deberían ir los ; y los insertará si son omitidos. Sin embargo, este sistema **no es infalible**.

Cap01/003.html

```
<!DOCTYPE HTML><html lang="es"><head><meta charset="UTF-8"></head><body>
<p id="mensaje"></p>
<script>
    document.getElementById("mensaje").textContent = "Hola Mundo";
</script>
</body></html>
```

Obsérvese que el objeto HTML es un párrafo (<p>) el cuál se identifica con <p id="mensaje"> y después cierra con </p>, luego JavaScript toma el control de ese objeto HTML y le cambia el interior.

Las comillas dobles pueden ser reemplazadas por comillas simples.

Cap01/004.html

```
<!DOCTYPE HTML><html lang="es"><head><meta charset="UTF-8"></head><body>
<p id="mensaje"></p>
<script>
    document.getElementById("mensaje").textContent = 'Hola Mundo';
</script>
</body></html>
```

Y además es posible poner código HTML en su interior para hacer cambios en el texto

Cap01/005.html

```
<!DOCTYPE HTML><html lang="es"><head><meta charset="UTF-8"></head><body>
<p id="mensaje"></p>
<script>
    document.getElementById("mensaje").innerHTML = "<b>Hola Mundo</b>";
</script>
</body></html>
```

## Uso de variables

JavaScript declara las variables con la palabra reservada let. Es un lenguaje débilmente tipado, es decir, no se definen variables enteras, reales o de cadena. JavaScript detecta el tipo de dato cuando se le asigna el primer valor a la variable. Si se le asigna un texto, se considera que es una variable tipo texto (string).

Cap01/006.html

```
<!DOCTYPE HTML><html lang="es"><head><meta charset="UTF-8"></head><body>
<p id="salida"></p>
<script>
// Variables en JavaScript
let valA = 15;
let cadena = "Esta es una prueba";
let aproxima = -7.126;

// Mostrar resultados de forma moderna y sencilla
document.getElementById("salida").innerHTML =
    valA + "<br>" +
    cadena + "<br>" +
    aproxima;
</script>
</body></html>
```

La instrucción “<br>” es para que haya un salto entre cada dato impreso.

Es posible variar el código para impresión y que quede en una sola línea

Cap01/007.html

```
<!DOCTYPE html><html lang="es"><body>
<p id="salida"></p>
<script>
// Variables en JavaScript
let valA = 78;
let cadena = "Cadena de texto";
let aproxima = -90.23;

// Mostrar resultados de forma moderna y sencilla
document.getElementById("salida").innerHTML =
    valA + "<br>" +
    cadena + "<br>" +
    aproxima;
</script>
</body></html>
```

O todo en una línea

Cap01/008.html

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>

  <script>
    // Variables en JavaScript
    let val = -901;
    let cad = "Manual de JavaScript";
    let aprox = 864.5;

    // Mostrar resultados de forma moderna y sencilla
    document.getElementById("salida").innerHTML =
      val + "<br>" +
      cad + "<br>" +
      aprox;
  </script>
</body></html>
```

## Operaciones matemáticas

Están disponibles suma(+), resta(-), multiplicación(\*), división(/), potencia(\*\*) y división modular(%). La división modular retorna el **residuo** de la división. Este es el código:

Cap01/009.html

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>

  <script>
    // Aritmética
    let valA = 12 + 45;
    let valB = 55 - 89;
    let valC = 7 * 8;
    let valD = 21 / 3;
    let valE = 5 ** 3; // 5 al cubo
    let valF = 5671 % 2; // Determina el residuo

    // Mostrar resultados de forma moderna y sencilla
    document.getElementById("salida").innerHTML =
      valA + "<br>" +
      valB + "<br>" +
      valC + "<br>" +
      valD + "<br>" +
      valE + "<br>" +
      valF + "<br>";
  </script>
</body></html>
```

## Asignación

Se utiliza el operador = para asignar un valor a una variable. Tiene algunas variaciones para acotar operaciones

Cap01/010.html

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>

  <script>
    // Asignación
    let valA = 8;
    let valB = valA * 2;

    // Mostrar resultados de forma moderna y sencilla
    document.getElementById("salida").innerHTML =
      valA + " y " + valB + "<br>";

    valA += 10; // Sumar 10
    valB -= 5; // Restar 5
    document.getElementById("salida").innerHTML +=
      valA + " y " + valB + "<br>";

    valA *= 3; // Multiplicar por 3
    valB /= 2; // Dividir entre 2
    document.getElementById("salida").innerHTML +=
      valA + " y " + valB + "<br>";
  </script>
</body></html>
```

Operaciones con cadenas

Se utiliza el operador + para concatenar cadenas.

Cap01/011.html

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>

  <script>
    // Uso de cadenas
    let txtA = "Una";
    let txtB = "Cadena";
    let txtC = txtA + " --- " + txtB;

    // Mostrar resultado de forma moderna
    document.getElementById("salida").innerHTML = txtC;
  </script>
</body></html>
```

¿Y qué sucede al combinar números y texto?

Cap01/012.html

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>

  <script>
    // Uso de cadenas
    let txtA = "Una";
    let txtB = 45.98;
    let txtC = txtA + " --- " + txtB;

    // Mostrar resultado de forma moderna
    document.getElementById("salida").innerHTML = txtC;
  </script>
</body></html>
```

Otro cambio, ambas variables con números reales.

Cap01/013.html

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>

  <script>
    // Uso de cadenas
    let txtA = 27.6;
    let txtB = 45.98;
    let txtC = txtA + " --- " + txtB;

    // Mostrar resultado de forma moderna
    document.getElementById("salida").innerHTML = txtC;
  </script>
</body></html>
```

Pero si se hace este cambio (poner a sumar las variables con valor de número real y luego concatenar una cadena):

Cap01/014.html

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>

  <script>
    // Uso de cadenas
    let txtA = 7;
    let txtB = 5;
    let txtC = txtA + txtB + "aaaaa";

    // Mostrar resultado de forma moderna
    document.getElementById("salida").innerHTML = txtC;
  </script>
</body></html>
```

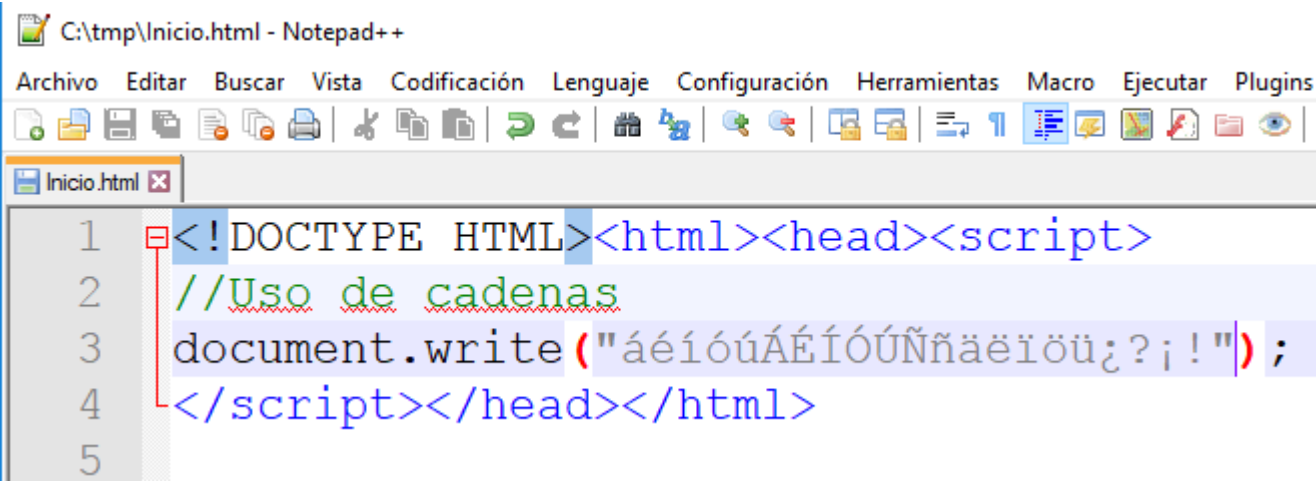
```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>

  <script>
    // Uso de cadenas
    let txtA = 7;
    let txtB = 5;
    let txtC = txtA + txtB + "aaaaa" + txtA + txtB;

    // Mostrar resultado de forma moderna
    document.getElementById("salida").innerHTML = txtC;
  </script>
</body></html>
```



Impresión de caracteres propios del idioma español  
Para imprimir caracteres como tildes, Ñs, abre interrogante o abre admiración. El código a continuación:



```
1 <!DOCTYPE HTML><html><head><script>
2 //Uso de cadenas
3 document.write("áéíóúÁÉÍÓÚÑñäëïöü¿?¡!");
4 </script></head></html>
5
```

Ilustración 19: Imprimir caracteres propios del idioma español

Y al ejecutarlo esto sucede en Google Chrome:

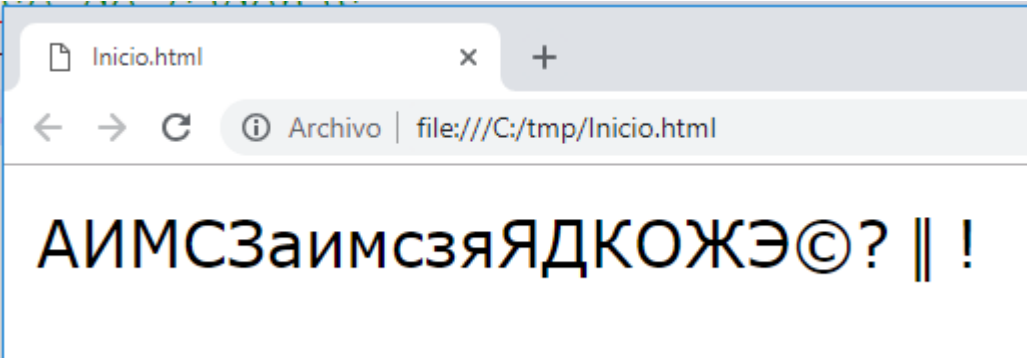


Ilustración 20: En Chrome puede pasar que no se impriman

Funciona correcto en Mozilla Firefox:

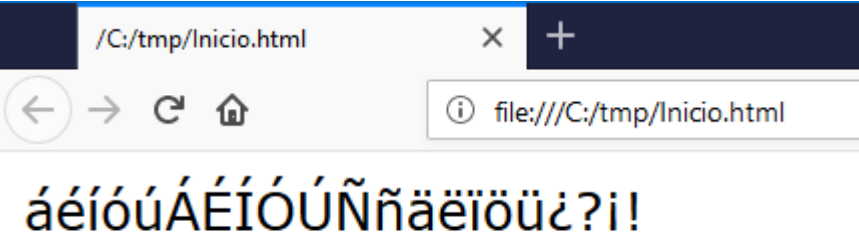


Ilustración 21: Funciona en Mozilla Firefox

¿Por qué en Google Chrome no se muestran correctamente los caracteres? Hay que fijarse en la codificación que tiene Notepad++ sobre el documento: El problema está en que la opción es “Codificar en ANSI”, luego el siguiente paso es dar clic a “Convertir a UTF-8”.

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>
  <script>
    // Librería matemática. Constantes.
    let salida =
      Math.E + "<br>" +           // Número e
      Math.LN2 + "<br>" +         // Logaritmo natural de 2
      Math.LN10 + "<br>" +        // Logaritmo natural de 10
      Math.LOG2E + "<br>" +       // Logaritmo base 2 de e
      Math.LOG10E + "<br>" +      // Logaritmo base 10 de e
      Math.PI + "<br>" +          // Constante PI
      Math.SQRT1_2 + "<br>" +     // Raíz cuadrada de 1/2
      Math.SQRT2 + "<br>";        // Raíz cuadrada de 2

    document.getElementById("salida").innerHTML = salida;
  </script>
</body></html>
```



## Funciones trigonométricas

Las funciones trigonométricas deben ser con ángulos en radianes, no en grados.

Cap01/017.html

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>

  <script>
    // Librería matemática.
    let angGrados = 35;
    let salida = "Grados: " + angGrados;

    let angRadian = angGrados * Math.PI / 180;
    salida += "<br>Radianes: " + angRadian;

    let seno = Math.sin(angRadian);          // Seno
    salida += "<br>Seno: " + seno;

    let coseno = Math.cos(angRadian);        // Coseno
    salida += "<br>Coseno: " + coseno;

    let tangente = Math.tan(angRadian);      // Tangente
    salida += "<br>Tangente: " + tangente;

    let arcoseno = Math.asin(seno);          // Arcoseno
    salida += "<br>Arcoseno: " + arcoseno;

    let arcocoseno = Math.acos(coseno);      // Arcocoseno
    salida += "<br>Arcocoseno: " + arcocoseno;

    let arcotangente = Math.atan(tangente);  // Arcotangente
    salida += "<br>Arcotangente: " + arcotangente;

    document.getElementById("salida").innerHTML = salida;
  </script>
</body></html>
```

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>

  <script>
    // Librería matemática.
    let salida = "Raíz cúbica: " + Math.cbrt(1331);
    salida += "<br>Techo: " + Math.ceil(4.01);
    salida += "<br>Exponencial: " + Math.exp(1);           // e elevado a N
    salida += "<br>Piso: " + Math.floor(4.99);
    salida += "<br>Log: " + Math.log(2);                  // Logaritmo natural
    salida += "<br>Mayor: " + Math.max(5, 9, 7, 3);       // Máximo valor
    salida += "<br>Menor: " + Math.min(5, 9, 7, 3);       // Mínimo valor
    salida += "<br>Potencia: " + Math.pow(2, 5);          // base, potencia
    salida += "<br>Aleatorio: " + Math.random();          // Aleatorio entre 0 y 1
    salida += "<br>Redondeo: " + Math.round(3.7);
    salida += "<br>Redondeo: " + Math.round(3.3);
    salida += "<br>Redondeo: " + Math.round(3.5);
    salida += "<br>Raíz cuadrada: " + Math.sqrt(144);
    salida += "<br>Truncar: " + Math.trunc(8.923);        // Trunca al entero

    document.getElementById("salida").innerHTML = salida;
  </script>
</body></html>
```

```
<!DOCTYPE html><html lang="es">
<body>
  <p id="salida"></p>

  <script>
    // Captura de datos por pantalla
    let texto = prompt("Escriba un mensaje");

    // Mostrar resultado de forma moderna
    document.getElementById("salida").innerHTML =
      "Escribió: " + texto;
  </script>
</body></html>
```

## Captura de datos tipo numérico por pantalla

La instrucción “prompt” captura datos tipo texto (string), luego hay que convertirlo a un valor entero para poder hacer operaciones matemáticas. “parseInt” es la instrucción para convertir de texto a número entero y “parseFloat” es la instrucción para convertir de texto a número real.

Cap01/020.html

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>
  <script>
    // Captura de datos numéricos
    let numEntero = parseInt(prompt("Escriba un valor entero")); // Tipo entero
    let numReal = parseFloat(prompt("Escriba un valor real")); // Tipo real

    let suma = numEntero + numReal;

    // Mostrar resultado
    document.getElementById("salida").innerHTML = suma;
  </script>
</body></html>
```

## Ejemplos de cálculos

### Cálculo del área de un triángulo dada la longitud de sus lados

Cap01/021.html

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>

  <script>
    // Cálculo del área de un triángulo según sus lados (Fórmula de Herón)
    let ladoA = parseFloat(prompt("Longitud Lado A"));
    let ladoB = parseFloat(prompt("Longitud Lado B"));
    let ladoC = parseFloat(prompt("Longitud Lado C"));

    let s = (ladoA + ladoB + ladoC) / 2; // semiperímetro
    let area = Math.sqrt(s * (s - ladoA) * (s - ladoB) * (s - ladoC));

    document.getElementById("salida").innerHTML = "Área es: " + area;
  </script>
</body></html>
```

### Área y perímetro del círculo

Cap01/022.html

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>

  <script>
    // Área y perímetro del círculo
    let radio = parseFloat(prompt("Radio"));
    let area = Math.PI * radio * radio;
    let perimetro = Math.PI * radio * 2;

    document.getElementById("salida").innerHTML =
      "Área es: " + area + "<br>" +
      "Perímetro es: " + perimetro;
  </script>
</body></html>
```

### Área y volumen del cilindro

Cap01/023.html

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>
  <script>
    // Área y volumen del cilindro
    let radio = parseFloat(prompt("Radio del círculo de la tapa"));
    let altura = parseFloat(prompt("Altura del cilindro"));

    let area = 2 * Math.PI * radio * (altura + radio);
    let volumen = Math.PI * Math.pow(radio, 2) * altura;

    document.getElementById("salida").innerHTML =
      "Área es: " + area + "<br>" +
      "Volumen es: " + volumen;
  </script>
</body></html>
```

Manejo de errores con números

isNaN

isNaN significa “is Not a Number”, si lo que entra por parámetro no es un número retorna true. Más información: <https://es.wikipedia.org/wiki/NaN>

Cap01/024.html

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>

  <script>
    // Chequea si el usuario digitó un número correctamente
    let numero = parseInt(prompt("Digite número"));

    if (isNaN(numero)) {
      document.getElementById("salida").innerHTML = "No escribió un número";
    } else {
      document.getElementById("salida").innerHTML = "Número escrito es: " + numero;
    }
  </script>
</body></html>
```

Cap01/025.html

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>

  <script>
    // Uso de isNaN
    let salida = "";

    salida += isNaN(891) + "<br>";           // false
    salida += isNaN(-7.19) + "<br>";          // false
    salida += isNaN(79 + 21) + "<br>";        // false
    salida += isNaN(0) + "<br>";              // false
    salida += isNaN('888') + "<br>";          // false
    salida += isNaN('Probar') + "<br>";       // true
    salida += isNaN('2019/01/13') + "<br>";   // true
    salida += isNaN('') + "<br>";            // false
    salida += isNaN(true) + "<br>";           // false
    salida += isNaN(undefined) + "<br>";      // true
    salida += isNaN('NaN') + "<br>";          // true
    salida += isNaN(NaN) + "<br>";            // true
    salida += isNaN(0 / 0) + "<br>";          // true

    document.getElementById("salida").innerHTML = salida;
  </script>
</body></html>
```

Cap01/026.html

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>

  <script>
    // Chequea si existe el NaN (Not a Number)
    let numero = parseFloat(prompt("Digite número"));
    let valor = Math.asin(numero);

    if (isNaN(valor)) {
      document.getElementById("salida").innerHTML = "Valor erróneo para arcoseno";
    } else {
      document.getElementById("salida").innerHTML = "Arcoseno es: " + valor;
    }
  </script>
</body></html>
```



## isFinite

Una división entre cero daría infinito, luego isFinite lo detecta.

Cap01/027.html

```
<!DOCTYPE html><html lang="es">
<body>
  <p id="salida"></p>

  <script>
    // Verificar si un número es legal o no
    let dividendo = parseInt(prompt("Digite dividendo"));
    let divisor = parseInt(prompt("Digite divisor"));

    let resultado = dividendo / divisor;

    // Verifica si el resultado es un número finito (no Infinity, -Infinity o NaN)
    if (isFinite(resultado)) {
      document.getElementById("salida").innerHTML = "Valor correcto: " + resultado;
    } else {
      document.getElementById("salida").innerHTML = "Número no legal: " + resultado;
    }
  </script>
</body></html>
```

## Try...catch

### Captura el error matemático

Cap01/028.html

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>

  <script>
    // Captura el error
    let expresion = prompt("Escriba expresión algebraica");

    try {
      let valor = eval(expresion);
      document.getElementById("salida").innerHTML = "Valor calculado es: " + valor;
    } catch (errordetectado) {
      document.getElementById("salida").innerHTML = "Mensaje de error: " + errordetectado.message;
    }
  </script>
</body></html>
```



```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>
  <script>
    let valorA;

    let salida = "";
    salida += "<br>1. Tipo de valorA es " + typeof (valorA);

    valorA = 157;
    salida += "<br>2. Tipo de valorA es " + typeof (valorA);

    valorA = "Había una vez...";
    salida += "<br>3. Tipo de valorA es " + typeof (valorA);

    valorA = 'K';
    salida += "<br>4. Tipo de valorA es " + typeof (valorA);

    valorA = true;
    salida += "<br>5. Tipo de valorA es " + typeof (valorA);

    valorA = new Date();
    salida += "<br>6. Tipo de valorA es " + typeof (valorA);

    valorA = 5 / 0; // Infinito
    salida += "<br>7. Tipo de valorA es " + typeof (valorA);

    valorA = Math.asin(39); // NaN
    salida += "<br>8. Tipo de valorA es " + typeof (valorA);

    document.getElementById("salida").innerHTML = salida;
  </script>
</body></html>
```

## Operaciones de bit

### Convertir un entero a su representación binaria

Cap01/030.html

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>
  <script>
    // Convertir un entero a su representación binaria
    let valor = 17;
    let resultado = valor.toString(2);

    let salida = valor + " en binario es: " + resultado + "<br>";

    // Conversión directa
    let convierte = 890..toString(2);
    salida += "En binario el número 890 es " + convierte + "<br>";

    document.getElementById("salida").innerHTML = salida;
  </script>
</body></html>
```

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>
  <script>
    // Operaciones de bit
    let valA = 25;
    let valB = 20;

    let salida = "";
    salida += valA + " en binario es: " + valA.toString(2) + "<br>";
    salida += valB + " en binario es: " + valB.toString(2) + "<br>";

    let valC = valA | valB; // Operación OR
    salida += valC + " en binario es: " + valC.toString(2) + "<br>";

    document.getElementById("salida").innerHTML = salida;
  </script>
</body></html>
```

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>
  <script>
    // Operaciones de bit
    let valA = 25;
    let valB = 20;

    let salida = "";
    salida += valA + " en binario es: " + valA.toString(2) + "<br>";
    salida += valB + " en binario es: " + valB.toString(2) + "<br>";

    let valC = valA & valB; // Operación AND
    salida += valC + " en binario es: " + valC.toString(2) + "<br>";

    document.getElementById("salida").innerHTML = salida;
  </script>
</body></html>
```

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>
  <script>
    // Operaciones de bit
    let valA = 25;
    let valB = 20;

    let salida = "";
    salida += valA + " en binario es: " + valA.toString(2) + "<br>";
    salida += valB + " en binario es: " + valB.toString(2) + "<br>";

    let valC = valA ^ valB; // Operación XOR
    salida += valC + " en binario es: " + valC.toString(2) + "<br>";

    document.getElementById("salida").innerHTML = salida;
  </script>
</body></html>
```

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>
  <script>
    // Operaciones de bit
    let valA = 2;

    let salida = "";
    salida += valA + " en binario es: " + valA.toString(2) + "<br>";

    let valB = valA << 1; // Multiplica por 2
    salida += valB + " en binario es: " + valB.toString(2) + "<br>";

    let valC = valA << 2; // Multiplica por 4
    salida += valC + " en binario es: " + valC.toString(2) + "<br>";

    let valD = valA << 3; // Multiplica por 8
    salida += valD + " en binario es: " + valD.toString(2) + "<br>";

    document.getElementById("salida").innerHTML = salida;
  </script>
</body></html>
```

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>
  <script>
    // Operaciones de bit
    let valA = 64;

    let salida = "";
    salida += valA + " en binario es: " + valA.toString(2) + "<br>";

    let valB = valA >> 1; // Divide entre 2
    salida += valB + " en binario es: " + valB.toString(2) + "<br>";

    let valC = valA >> 2; // Divide entre 4
    salida += valC + " en binario es: " + valC.toString(2) + "<br>";

    let valD = valA >> 3; // Divide entre 8
    salida += valD + " en binario es: " + valD.toString(2) + "<br>";

    document.getElementById("salida").innerHTML = salida;
  </script>
</body></html>
```

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>
  <script>
    let valA = 64;
    let valB = 17;

    let salida = "";
    salida += "valores: " + valA + " , " + valB + "<br>";

    // Para intercambiar los valores de esas variables se puede hacer uso de operadores de bit (XOR)
    valA ^= valB;
    valB ^= valA;
    valA ^= valB;

    salida += "valores: " + valA + " , " + valB + "<br>";

    document.getElementById("salida").innerHTML = salida;
  </script>
</body></html>
```



# Corrigiendo errores en JavaScript

Los navegadores modernos traen herramientas para desarrolladores que permiten, por ejemplo, detectar errores en el código. Al ejecutar en Edge, si el script en JavaScript tiene errores, la pantalla aparece en blanco

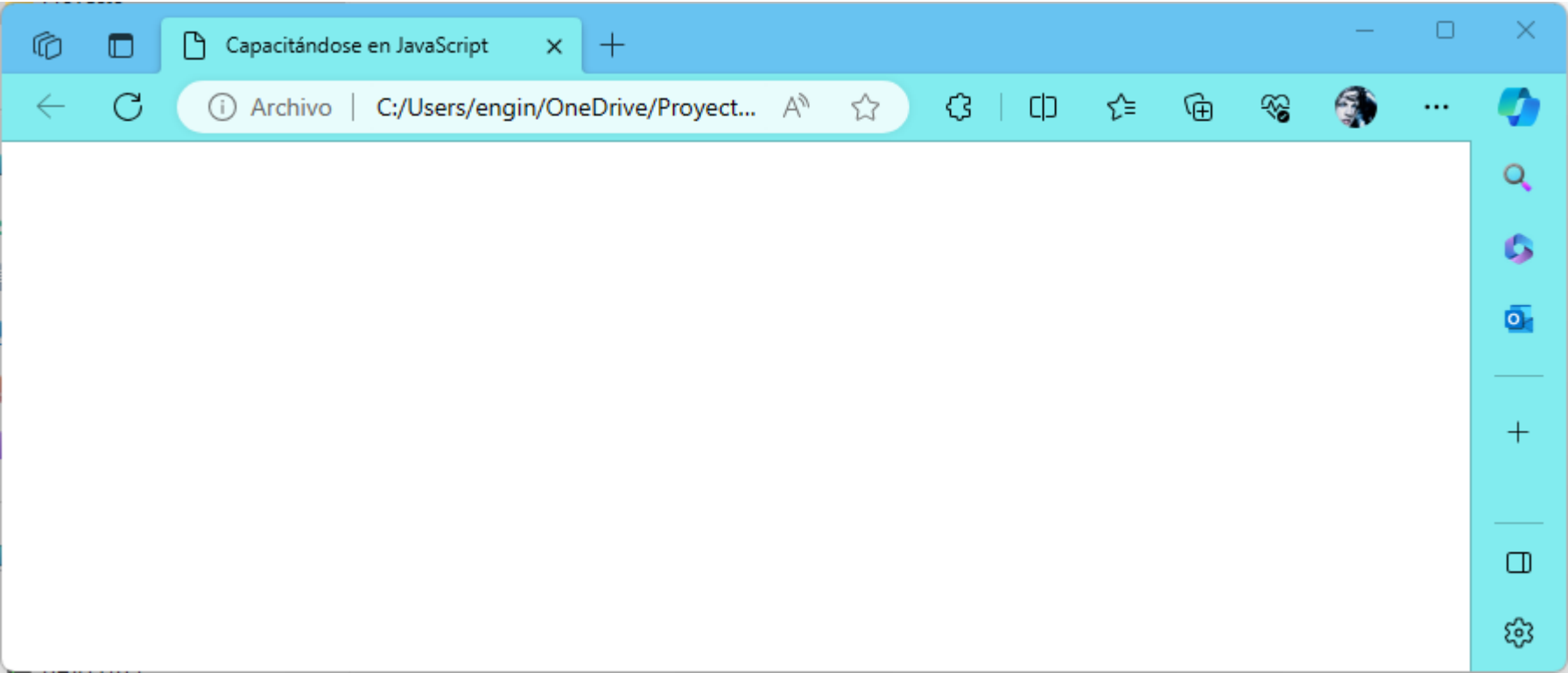


Ilustración 22: Ante errores en JavaScript, el navegador queda en blanco

En ese caso, se presiona la tecla F12 y aparece un menú de desarrollador, se selecciona “Console”

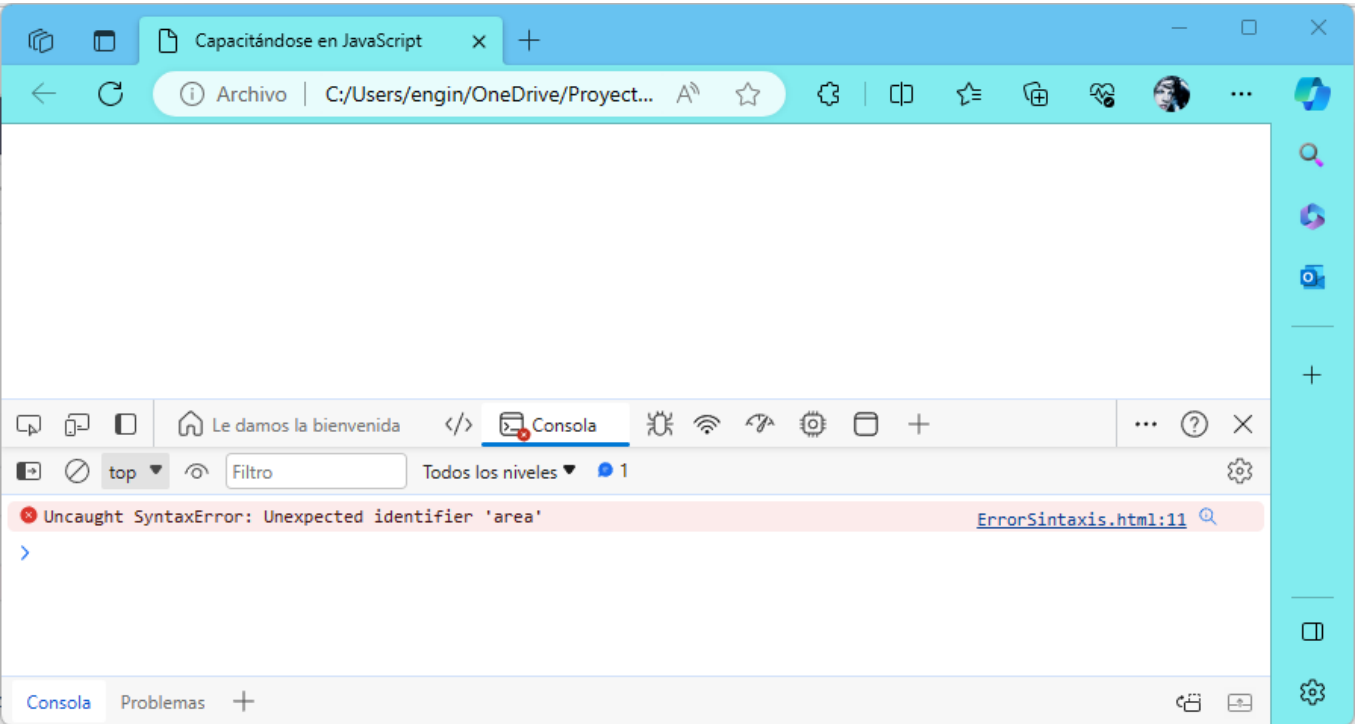


Ilustración 23: Presionando F12 en el navegador Edge y dando clic en Consola se muestra el error

En “Consola” se muestra en qué línea hay un error. Se debe entender el error y proceder a corregirlo. No es esperable una identificación perfecta del error por parte del navegador. En otros navegadores es similar el uso de F12 para mostrar la consola.

## Capítulo 2. Si Condicional



Si Condicional

Similar a C++ o C# es la sintaxis del sí condicional en JavaScript.

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>

  <script>
    // Si condicional
    let valA = parseFloat(prompt("valor A: "));
    let valB = parseFloat(prompt("valor B: "));

    let salida = "";

    if (valA > valB) {
      salida += valA + " es mayor que " + valB;
    } else {
      salida += valA + " es menor o igual que " + valB;
    }

    document.getElementById("salida").innerHTML = salida;
  </script>
</body></html>
```

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>

  <script>
    // Si condicional. Uso de else if
    let valA = parseFloat(prompt("valor A: "));
    let valB = parseFloat(prompt("valor B: "));

    let salida = "";

    if (valA > valB) {
      salida += valA + " es mayor que " + valB;
    } else if (valA === valB) {
      salida += valA + " es igual que " + valB;
    } else {
      salida += valA + " es menor que " + valB;
    }

    document.getElementById("salida").innerHTML = salida;
  </script>
</body></html>
```

Los operadores de comparación son:

| Símbolo | Significado                                                                                                                                                                 |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| >       | Mayor que                                                                                                                                                                   |
| >=      | Mayor o igual que                                                                                                                                                           |
| <       | Menor que                                                                                                                                                                   |
| <=      | Menor o igual que                                                                                                                                                           |
| ==      | Es igual a                                                                                                                                                                  |
| !=      | Es diferente que                                                                                                                                                            |
| ===     | Comparación de igualdad estricta de valor y tipo. Por ejemplo<br>5 === "5" sería falso<br>En cambio<br>5 == "5" sería verdadero (porque == es un comparador menos estricto) |
| !==     | Comparación de diferencia estricta de valor y tipo.                                                                                                                         |



# Operadores lógicos: Conjunción (Y, &&) y Disyunción (O, ||)

El operador AND, Y en JavaScript es &&. El operador OR, O en JavaScript es ||

Cap02/003.html

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>

  <script>
    let ladoA = parseInt(prompt("Lado A:"));
    let ladoB = parseInt(prompt("Lado B:"));
    let ladoC = parseInt(prompt("Lado C:"));

    let salida = "";

    if (ladoA === ladoB && ladoB === ladoC) {
      salida += "Es equilátero";
    } else if (ladoA !== ladoB && ladoA !== ladoC && ladoB !== ladoC) {
      salida += "Es escaleno";
    } else {
      salida += "Es isósceles";
    }

    document.getElementById("salida").innerHTML = salida;
  </script>
</body></html>
```

Cap02/004.html

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>

  <script>
    // Tipo de triángulo
    let ladoA = parseInt(prompt("Lado A:"));
    let ladoB = parseInt(prompt("Lado B:"));
    let ladoC = parseInt(prompt("Lado C:"));

    let salida = "";

    if (ladoA === ladoB && ladoB === ladoC) {
      salida += "Es equilátero";
    } else if (ladoA === ladoB || ladoB === ladoC || ladoA === ladoC) {
      salida += "Es isósceles";
    } else {
      salida += "Es escaleno";
    }

    document.getElementById("salida").innerHTML = salida;
  </script>
</body></html>
```

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>

  <script>
    // Uso del switch
    let valor = parseInt(prompt("Digite un valor"));

    let salida = "";

    switch (valor) {
      case 0:
        salida += "Escribió cero";
        break;
      case 1:
        salida += "Digitó 1";
        break;
      case 2:
        salida += "Tecleó dos";
        break;
      case 3:
        salida += "Presionó 3";
        break;
      default:
        salida += "Ingresó otro valor";
    }

    document.getElementById("salida").innerHTML = salida;
  </script>
</body></html>
```

Se pueden usar varias líneas en el switch. El cierre es la palabra reservada “break”.

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>

  <script>
    // Uso del switch
    let valor = parseInt(prompt("Digite un valor"));

    let salida = "";

    switch (valor) {
      case 0:
        salida += "Escribió cero";
        break;

      case 1:
        salida += "Digitó 1";
        salida += "<br>Escribió uno";
        salida += "<br>Registró uno";
        break;

      case 2:
        salida += "Tecleó dos";
        break;

      case 3:
        salida += "Presionó 3";
        break;

      default:
        salida += "Ingresó otro valor";
    }

    document.getElementById("salida").innerHTML = salida;
  </script>
</body></html>
```

## Capítulo 3. Ciclos

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>
  <script>
    // Ciclo for creciente
    let salida = "";

    for (let cont = 1; cont <= 100; cont++) {
      salida += cont + ", ";
    }

    document.getElementById("salida").innerHTML = salida;
  </script>
</body></html>
```

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>
  <script>
    // Ciclo for decreciente
    let salida = "";

    for (let cont = 100; cont >= 1; cont--) {
      salida += cont + ", ";
    }

    document.getElementById("salida").innerHTML = salida;
  </script>
</body></html>
```

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>
  <script>
    //Ciclo for creciente de 3 en 3
    let salida = "";

    for (let cont = 0; cont <= 300; cont += 3) {
      salida += cont + ", ";
    }

    document.getElementById("salida").innerHTML = salida;
  </script>
</body></html>
```

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>
  <script>
    //Ciclo for creciente de 10 en 10. Se detiene antes de 500
    let salida = "";

    for (let cont = 0; cont < 500; cont += 10) {
      salida += cont + ", ";
    }

    document.getElementById("salida").innerHTML = salida;
  </script>
</body></html>
```

```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>
  <script>
```



```
//Ciclo for creciente aumentando el doble del anterior
let salida = "";

for (let cont = 1; cont <= 5000; cont *= 2) {
    salida += cont + ", ";
}

document.getElementById("salida").innerHTML = salida;
</script>
</body></html>
```

Cap03/006.html

```
<!DOCTYPE html><html lang="es"><body>
<p id="salida"></p>
<script>
//Ciclo for decreciente disminuyendo la mitad del anterior
let salida = "";

for (let cont = 4096; cont >= 1; cont /= 2) {
    salida += cont + ", ";
}

document.getElementById("salida").innerHTML = salida;
</script>
</body></html>
```

Cap03/007.html

```
<!DOCTYPE html><html lang="es"><body>
<p id="salida"></p>
<script>
/* Ciclo for con dos variables. Observar las dos
condiciones que deben darse */
let salida = "";

for (let x = 1, y = 34; x <= 10 && y >= 28; x++, y--) {
    salida += x + ", " + y + "<br>";
}

document.getElementById("salida").innerHTML = salida;
</script>
</body></html>
```

Cap03/008.html

```
<!DOCTYPE html><html lang="es"><body>
<p id="salida"></p>
<script>
/* Ciclo for con dos variables. Observar las dos
condiciones que deben darse */
let salida = "";

for (let x = 1, y = 34; x <= 10 || y >= 28; x++, y--) {
    salida += x + ", " + y + "<br>";
}

document.getElementById("salida").innerHTML = salida;
</script>
</body></html>
```

```
<!DOCTYPE html><html lang="es"><body>
<p id="salida"></p>
<script>
  // Ciclo while creciente
  let salida = "";

  let cont = 1;
  while (cont <= 100) {
    salida += cont + ", ";
    cont++;
  }

  document.getElementById("salida").innerHTML = salida;
</script>
</body></html>
```

```
<!DOCTYPE html><html lang="es"><body>
<p id="salida"></p>
<script>
  //Ciclo while decreciente
  let salida = "";

  let cont = 100;
  while (cont >= 1) {
    salida += cont + ", ";
    cont--;
  }

  document.getElementById("salida").innerHTML = salida;
</script>
</body></html>
```

```
<!DOCTYPE html><html lang="es"><body>
<p id="salida"></p>
<script>
  //Ciclo while creciente de 3 en 3
  let salida = "";

  let cont = 0;
  while (cont <= 300) {
    salida += cont + ", ";
    cont+=3;
  }

  document.getElementById("salida").innerHTML = salida;
</script>
</body></html>
```

```
<!DOCTYPE html><html lang="es"><body>
<p id="salida"></p>
<script>
  //Ciclo while creciente de 10 en 10. Se detiene antes de 500
  let salida = "";

  let cont = 0;
  while (cont < 500) {
    salida += cont + ", ";
    cont+=10;
  }

  document.getElementById("salida").innerHTML = salida;
</script>
</body></html>
```



```

<!DOCTYPE html><html lang="es"><body>
<p id="salida"></p>
<script>
//Ciclo while creciente aumentando el doble del anterior
let salida = "";

let cont = 1;
while (cont <= 5000) {
    salida += cont + ", ";
    cont *= 2;
}

document.getElementById("salida").innerHTML = salida;
//Resultado:1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096
</script>
</body></html>

```

```

<!DOCTYPE html><html lang="es"><body>
<p id="salida"></p>
<script>
//Ciclo while decreciente
let salida = "";

let cont = 4096;
while (cont >= 1) {
    salida += cont + ", ";
    cont /= 2;
}

document.getElementById("salida").innerHTML = salida;
//Resultado: 4096, 2048, 1024, 512, 256, 128, 64, 32, 16, 8, 4, 2, 1,
</script>
</body></html>

```

```

<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
/* Ciclo while con dos variables. Observemos las dos
condiciones que deben darse */
let x = 1;
let y = 34;
let salida = "";
while (x <= 10 && y >= 28) {
    salida += x + ", " + y + "<br>";
    x++;
    y--;
}
document.getElementById("salida").innerHTML = salida;
/* Resultado:
1, 34
2, 33
3, 32
4, 31
5, 30
6, 29
7, 28
*/
</script></body></html>

```

```

<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
/* Ciclo while con dos variables. Observemos las dos
condiciones que deben darse cambiando && por || */
let x = 1;

```

```
let y = 34;
let salida = "";
while (x <= 10 || y >= 28) {
    salida += x + ", " + y + "<br>";
    x++;
    y--;
}
document.getElementById("salida").innerHTML = salida;
/* Resultado:
1, 34
2, 33
3, 32
4, 31
5, 30
6, 29
7, 28
8, 27
9, 26
10, 25
*/
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
/* Ciclo do-while con dos variables. Observemos las dos
   condiciones que deben darse cambiando && por || */
let x = 1;
let y = 34;
let salida = "";
do {
    salida += x + ", " + y + "<br>";
    x++;
    y--;
} while (x <= 10 || y >= 28);
document.getElementById("salida").innerHTML = salida;
/* Resultado:
1, 34
2, 33
3, 32
4, 31
5, 30
6, 29
7, 28
8, 27
9, 26
10, 25
*/
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Ciclo do-while creciente
let cont = 1;
do {
    salida += cont + ", ";
    cont++;
} while (cont <= 100);
document.getElementById("salida").innerHTML = salida;
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Ciclo do-while decreciente
let cont = 100;
let salida = "";
do {
    salida += cont + ", ";
    cont--;
} while (cont >= 1);
document.getElementById("salida").innerHTML = salida;
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Ciclo do-while creciente de 3 en 3
let cont = 0;
let salida = "";
do {
    salida += cont + ", ";
    cont += 3;
} while (cont <= 300);
document.getElementById("salida").innerHTML = salida;
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Ciclo do-while creciente de 10 en 10. Se detiene antes de 500
let cont = 0;
let salida = "";
do {
    salida += cont + ", ";
    cont += 10;
} while (cont < 500);
document.getElementById("salida").innerHTML = salida;
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Ciclo while creciente aumentando el doble del anterior
let cont = 1;
let salida = "";
do {
    salida += cont + ", ";
    cont *= 2;
} while (cont <= 5000);
document.getElementById("salida").innerHTML = salida;
//Resultado:1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Ciclo while decreciente disminuyendo la mitad del anterior
let cont = 4096;
let salida = "";
do {
    salida += cont + ", ";
    cont /= 2;
} while (cont >= 1);
document.getElementById("salida").innerHTML = salida;
//Resultado: 4096, 2048, 1024, 512, 256, 128, 64, 32, 16, 8, 4, 2, 1,
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
/* Ciclo while con dos variables. Observemos las dos
   condiciones que deben darse */
let x = 1;
let y = 34;
let salida = "";
do {
    salida += x + ", " + y + "<br>";
    x++;
    y--;
} while (x <= 10 && y >= 28);
document.getElementById("salida").innerHTML = salida;
/* Resultado:
1, 34
2, 33
3, 32
4, 31
5, 30
6, 29
```

```
7, 28
*/
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Ciclo for creciente, se detiene en 50
let salida = "";
for (let cont = 1; cont <= 100; cont++) {
    salida += cont + ", ";
    if (cont >= 50) break; //Sale del ciclo
}
document.getElementById("salida").innerHTML = salida;
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Ciclo while creciente. Sale cuando cont sea 50
let cont = 1;
let salida = "";
while (cont <= 100) {
    salida += cont + ", ";
    cont++;
    if (cont >= 50) break; //Sale del ciclo
}
document.getElementById("salida").innerHTML = salida;
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Ciclo do-while creciente. Sale cuando cont sea 50
let cont = 1;
let salida = "";
do {
    salida += cont + ", ";
    cont++;
    if (cont >= 50) break; //Sale del ciclo
} while (cont <= 100);
document.getElementById("salida").innerHTML = salida;
</script></body></html>
```



### Uso del continue

La sentencia “continue” hace que el programa haga inmediatamente la siguiente iteración ignorando las instrucciones siguientes dentro del ciclo.

Cap03/028.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Ciclo for creciente, se salta los pares
let salida = "";
for (let cont = 1; cont <= 10; cont++) {
  //Si es par, salta a la siguiente iteración
  if (cont % 2 === 0) continue;
  salida += cont + ", ";
}
document.getElementById("salida").innerHTML = salida;
//Resultado: 1, 3, 5, 7, 9,
</script></body></html>
```

Cap03/029.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Ciclo while creciente. Se salta los pares
let cont = 1;
let salida = "";
while (cont <= 100) {
  cont++;
  if (cont % 2 === 0) continue; //Salta a la siguiente iteración
  salida += cont + ", ";
}
document.getElementById("salida").innerHTML = salida;
</script></body></html>
```

Cap03/030.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Ciclo do-while creciente. Se salta los pares
let cont = 1;
let salida = "";
do {
  cont++;
  if (cont % 2 === 0) continue; //Salta a la siguiente iteración
  salida += cont + ", ";
} while (cont <= 100);
document.getElementById("salida").innerHTML = salida;
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Ciclos anidados
for (let x = 1; x <= 3; x++) {
  for (let y = 45; y <= 50; y++) {
    for (let z = 7; z <= 9; z++) {
      document.write(x + ";" + y + ";" + z + "<br>");
    }
  }
}
</script></body></html>
```

Para salir de ciclos anidados hacemos uso de etiquetas, luego hacemos uso de “break *etiqueta*” y saltaría al lugar de esa etiqueta:

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Saliendo de ciclos anidados. Uso de etiquetas.
saliendo:
  for (let x = 1; x <= 3; x++) {
    for (let y = 45; y <= 50; y++) {
      for (let z = 7; z <= 9; z++) {
        document.write(x + ";" + y + ";" + z + "<br>");
        if (z === 8) break saliendo; //Sale de todos los ciclos
      }
    }
  }
/* Resultado:
1;45;7
1;45;8 */
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Saliendo de ciclos anidados
for (let x = 1; x <= 3; x++) {
  ciclointerno:
    for (let y = 45; y <= 50; y++) {
      for (let z = 7; z <= 9; z++) {
        document.write(x + ";" + y + ";" + z + "<br>");
        if (z === 8) break ciclointerno;
      }
    }
}
/*Resultado:
1;45;7
1;45;8
2;45;7
2;45;8
3;45;7
3;45;8 */
</script></body></html>
```



```
<!DOCTYPE html><html lang="es"><body>
  <p id="salida"></p>

  <script>
    // Números aleatorios
    let numero = Math.random(); // Un número entre 0 y 1. Nunca dará 1.

    document.getElementById("salida").innerHTML = numero;
  </script>
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Números aleatorios. Ciclo.
let salida = "";
for (let num = 1; num <= 100; num++)
  salida += Math.random() + ", ";

document.getElementById("salida").innerHTML = salida;
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
/* Variable aleatoria
   Distribución Normal. Generar una variable aleatoria con media M y desviación D
   variable = M + D * c
   donde c = cos(2*PI*r2)*raizcuadrada(-2*LogarimoNatural(r1))
   r1 y r2 son números aleatorios*/
let M = 100;
let D = 7;
let salida = "";
for (let cont = 1; cont <= 100; cont++) {
  let r1 = Math.random();
  let r2 = Math.random();
  let c = Math.cos(2 * Math.PI * r2) * Math.sqrt(-2 * Math.log(r1));
  let variable = M + D * c;
  salida += variable + ", ";
}
document.getElementById("salida").innerHTML = salida;
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
/* Distribución Triangular. Generar una variable aleatoria con valor mínimo A,
   valor más probable B y valor máximo C
   Si  $r < (B-A)/(C-A)$ 
       Variable =  $A + \sqrt{r \cdot (C-A) \cdot (B-A)}$ 
   de lo contrario
       Variable =  $C - \sqrt{(1-r) \cdot (C-A) \cdot (C-B)}$ 
*/
let A = 150;
let B = 190;
let C = 230;
let variable = 0;
let salida = "";
for (let cont = 1; cont <= 100; cont++) {
    let r = Math.random();
    if (r < (B - A) / (C - A))
        variable = A + Math.sqrt(r * (C - A) * (B - A));
    else
        variable = C - Math.sqrt((1 - r) * (C - A) * (C - B));
    salida += variable + ", ";
}
document.getElementById("salida").innerHTML = salida;
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
/* Variable aleatoria
   Distribución uniforme. Generar un número entero entre A y B
   variable = r * (B - A) + A */
let A = 10;
let B = 50;
let salida = "";
for (let cont = 1; cont <= 100; cont++) {
  let r = Math.random();
  let variable = r * (B - A) + A;
  salida += variable + ", ";
}
document.getElementById("salida").innerHTML = salida;
</script></body></html>
```

## Capítulo 4. Funciones o subrutinas

```

<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
let numEntero = parseInt(prompt("Escriba un valor entero")); //Tipo entero
let Primo = EsPrimo(numEntero); //Llama la función
document.getElementById("salida").innerHTML = numEntero + " es primo: " + Primo;

//Función que retorna true si el número enviado por parámetro es primo
function EsPrimo(num) {
  if (num <= 1) return false;
  if (num === 2) return true;
  if (num % 2 === 0) return false;
  for (let divide = 3; divide <= Math.sqrt(num); divide += 2)
    if (num % divide === 0) return false;
  return true;
}
</script></body></html>

```

El anterior programa pregunta un número por pantalla y responde si el número es primo o no.

Cuando tenemos una buena cantidad de funciones, es recomendable crear una librería. Generamos un archivo con extensión .js

Y este sería el código dentro de ese archivo Libreria.js

Libreria.js

```

//Retorna true si el número es palíndromo
function EsPalindromo(num) {
  return num === InvierteNumero(num);
}

//Retorna las dos últimas cifras del número
function retornadosultimascifras(num) {
  return num % 100;
}

//Dice cuántas cifras de determinado número hay en el número enviado
function cifrashalladas(num, cifra) {
  let acumula = 0;
  while (num !== 0) {
    if (num % 10 === cifra) acumula++;
    num = Math.floor(num / 10);
  }
  return acumula;
}

//Invierte un número
function InvierteNumero(num) {
  let multiplica = Math.pow(10, TotalCifras(num) - 1);
  let acumula = 0;
  while (num !== 0) {
    let cifra = num % 10;
    acumula += cifra * multiplica;
    multiplica /= 10;
    num = Math.floor(num / 10);
  }
  return acumula;
}

//Retorna la primera cifra de un número
function PrimeraCifra(num) {
  let primera = 0;
  while (num !== 0) {
    primera = num % 10;
    num = Math.floor(num / 10);
  }
  return primera;
}

```



```

//Retorna el número de cifras de un número
function TotalCifras(num) {
    let cuenta = 0;
    while (num !== 0) {
        cuenta++;
        num = Math.floor(num / 10);
    }
    return cuenta;
}

//Retorna true si un número es impar
function EsImpar(num) {
    return num % 2 === 1;
}

//Retorna true si un número es par
function EsPar(num) {
    return num % 2 === 0;
}

//Retorna la sumatoria de las cifras de un número
function SumaCifras(num) {
    let acum = 0;
    while (num !== 0) {
        let cifra = num % 10;
        acum += cifra;
        num = Math.floor(num / 10);
    }
    return acum;
}

//Retorna la sumatoria de las cifras pares de un número
function SumaCifrasPares(num) {
    let acum = 0;
    while (num !== 0) {
        let cifra = num % 10;
        if (cifra % 2 === 0) acum += cifra;
        num = Math.floor(num / 10);
    }
    return acum;
}

//Retorna la sumatoria de las cifras impares de un número
function SumaCifrasImpares(num) {
    let acum = 0;
    while (num !== 0) {
        let cifra = num % 10;
        if (cifra % 2 !== 0) acum += cifra;
        num = Math.floor(num / 10);
    }
    return acum;
}

//Retorna el producto de las cifras de un número
function MultiplicaCifras(num) {
    let acum = 1;
    while (num !== 0) {
        let cifra = num % 10;
        acum *= cifra;
        num = Math.floor(num / 10);
    }
    return acum;
}

//Retorna el producto de las cifras impares de un número
function MultiplicaCifrasImpares(num) {
    let acum = 1;
    while (num !== 0) {
        let cifra = num % 10;
        if (cifra % 2 !== 0) acum *= cifra;
        num = Math.floor(num / 10);
    }
    return acum;
}

//Retorna el producto de las cifras pares de un número
function MultiplicaCifrasPares(num) {
    let acum = 1;
    while (num !== 0) {
        let cifra = num % 10;

```

```

        if (cifra % 2 === 0) acum *= cifra;
        num = Math.floor(num / 10);
    }
    return acum;
}

//Retorna la antepenúltima cifra de un número entero
function AntepenultimaCifra(num) {
    return Math.floor(num / 100) % 10;
}

//Retorna la penúltima cifra de un número entero
function PenultimaCifra(num) {
    return Math.floor(num / 10) % 10;
}

//Retorna la última cifra de un número entero
function UltimaCifra(num) {
    return num % 10;
}

//Retorna true si el número enviado por parámetro es primo
function EsPrimo(num) {
    if (num <= 1) return false;
    if (num === 2) return true;
    if (num % 2 === 0) return false;
    for (let divide = 3; divide <= Math.sqrt(num); divide += 2)
        if (num % divide === 0) return false;
    return true;
}

//Retorna true si todas las cifras son pares
function TodasCifrasPares(num) {
    while (num !== 0) {
        let cifra = num % 10;
        if (cifra % 2 !== 0) return false;
        num = Math.floor(num / 10);
    }
    return true;
}

//Retorna true si todas las cifras son impares
function TodasCifrasImpares(num) {
    if (num === 0) return false;
    while (num !== 0) {
        let cifra = num % 10;
        if (cifra % 2 === 0) return false;
        num = Math.floor(num / 10);
    }
    return true;
}

//Retorna el número de cifras pares
function TotalCifrasPares(num) {
    let acum = 0;
    while (num !== 0) {
        let cifra = num % 10;
        if (cifra % 2 === 0) acum++;
        num = Math.floor(num / 10);
    }
    return acum;
}

//Retorna el número de cifras impares
function TotalCifrasImpares(num) {
    let acum = 0;
    while (num !== 0) {
        let cifra = num % 10;
        if (cifra % 2 !== 0) acum++;
        num = Math.floor(num / 10);
    }
    return acum;
}

//Retorna la cifra más alta
function LaCifraMasAlta(num) {
    let cifra = 0;
    while (num !== 0) {
        if (num % 10 > cifra) cifra = num % 10;
        num = Math.floor(num / 10);
    }
}

```



```

    }
    return cifra;
}

//Retorna la cifra más baja
function LaCifraMasBaja(num) {
    let cifra = 9;
    while (num !== 0) {
        if (num % 10 < cifra) cifra = num % 10;
        num = Math.floor(num / 10);
    }
    return cifra;
}

//Retorna true si num tiene sólo cifras menores o iguales de cifra
function SoloCifrasMenorIgual(num, cifra) {
    while (num !== 0) {
        if (num % 10 > cifra) return false;
        num = Math.floor(num / 10);
    }
    return true;
}

//Retorna true si num tiene sólo cifras mayores o iguales de cifra
function SoloCifrasMayorIgual(num, cifra) {
    while (num !== 0) {
        if (num % 10 < cifra) return false;
        num = Math.floor(num / 10);
    }
    return true;
}

//Retorna true si todas las cifras son distintas
function DistintasCifras(num) {
    for (let cifra = 0; cifra <= 9; cifra++) {
        let numero = num;
        let cuenta = 0;
        while (numero !== 0) {
            if (numero % 10 === cifra) cuenta++;
            if (cuenta > 1) return false;
            numero = Math.floor(numero / 10);
        }
    }
    return true;
}

//Retorna si todas las cifras pares son distintas
function DistintasCifrasPares(num) {
    for (let cifra = 0; cifra <= 8; cifra += 2) {
        let numero = num;
        let cuenta = 0;
        while (numero !== 0) {
            if (numero % 10 === cifra) cuenta++;
            if (cuenta > 1) return false;
            numero = Math.floor(numero / 10);
        }
    }
    return true;
}

//Retorna si todas las cifras impares son distintas
function DistintasCifrasImPares(num) {
    for (let cifra = 1; cifra <= 9; cifra += 2) {
        let numero = num;
        let cuenta = 0;
        while (numero !== 0) {
            if (numero % 10 === cifra) cuenta++;
            if (cuenta > 1) return false;
            numero = Math.floor(numero / 10);
        }
    }
    return true;
}

//Elelet al cuadrado cada cifra y retornar la suma
function sumacifrascuadrado(num) {
    let acumula = 0;
    while (num !== 0) {
        acumula += (num % 10) * (num % 10);
        num = Math.floor(num / 10);
    }
}

```

```
}  
    return acumula;  
}
```

Para usar la librería se llama desde el archivo principal con `<script src="Librería.js"></script>`

Cap04/002.html

```
<!DOCTYPE HTML><html lang="es"><head><script src="Libreria.js"></script></head><body>  
<p id="salida"></p>  
<script>  
let numEntero = parseInt(prompt("Escribir un valor entero"));  
let Primo = EsPrimo(numEntero); //Llama la función  
document.getElementById("salida").innerHTML = numEntero + " es primo: " + Primo;  
</script></body></html>
```

De esa forma separa el código JavaScript y hace más legible el código.

```
<!DOCTYPE HTML><html lang="es"><head><script src="Libreria.js"></script></head><body>
<p id="salida"></p>
<script>
let minimo = 1000;
let maximo = 9999;

//Cantidad de números que la suma de las tres últimas cifras es par
let cuenta = 0;
for (let num = minimo; num <= maximo; num++) {
    if (EsPar(AntepenultimaCifra(num) + UltimaCifra(num) + PenultimaCifra(num))) {
        cuenta++;
    }
}
Imprime(minimo, maximo, "Cantidad de números que la suma de las tres últimas cifras es par", cuenta);

//Imprime el resultado en el navegador
function Imprime(minimo, maximo, texto, resultado) {
    document.getElementById("salida").innerHTML = "Entre " + minimo + " y " + maximo + ". " + texto + ": " +
resultado + "<br>";
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head><script src="Libreria.js"></script></head><body>
<p id="salida"></p>
<script>
let minimo = 1000;
let maximo = 9999;

//Cantidad de números que la multiplicación de las tres últimas cifras es igual a la suma de esas tres
últimas cifras
let cuenta = 0;
for (let num = minimo; num <= maximo; num++) {
    if (AntepenultimaCifra(num) * UltimaCifra(num) * PenultimaCifra(num) === AntepenultimaCifra(num) +
UltimaCifra(num) + PenultimaCifra(num)) {
        cuenta = cuenta + 1
    }
}
Imprime(minimo, maximo, "Cantidad de números que la multiplicación de las tres últimas cifras es igual a
la suma de esas tres últimas cifras", cuenta);

//Imprime el resultado en el navegador
function Imprime(minimo, maximo, texto, resultado) {
    document.getElementById("salida").innerHTML = "Entre " + minimo + " y " + maximo + ". " + texto + ": " +
resultado + "<br>";
}
</script></body></html>
```

```

<!DOCTYPE HTML><html lang="es"><head><script src="Libreria.js"></script></head><body>
<p id="salida"></p>
<script>
let minimo = 8000;
let maximo = 9000;

//Cantidad de números que cada una de sus tres últimas cifras es menor de 5
let cuenta = 0;
for (let num = minimo; num <= maximo; num++) {
    if (AntepenultimaCifra(num) < 5 && PenultimaCifra(num) < 5 && UltimaCifra(num) < 5) {
        cuenta = cuenta + 1
    }
}
Imprime(minimo, maximo, "Cantidad de números que cada una de sus tres últimas cifras es menor de 5",
cuenta);

//Imprime el resultado en el navegador
function Imprime(minimo, maximo, texto, resultado) {
    document.getElementById("salida").innerHTML = "Entre " + minimo + " y " + maximo + ". " + texto + ": " +
resultado + "<br>";
}
</script></body></html>

```

```

<!DOCTYPE HTML><html lang="es"><head><script src="Libreria.js"></script></head><body>
<p id="salida"></p>
<script>
let minimo = 8500;
let maximo = 8700;

//Cantidad de números que al juntar la antepenúltima cifra y la última cifra se obtenga un primo
let contar = 0;
for (let num = minimo; num <= maximo; num++) {
    let nuevo = AntepenultimaCifra(num) * 10 + UltimaCifra(num)
    if (EsPrimo(nuevo)) {
        contar++;
    }
}
Imprime(minimo, maximo, "Cantidad de números que al juntar la antepenúltima cifra y la última cifra se
obtenga un primo", contar);

//Imprime el resultado en el navegador
function Imprime(minimo, maximo, texto, resultado) {
    document.getElementById("salida").innerHTML = "Entre " + minimo + " y " + maximo + ". " + texto + ": " +
resultado + "<br>";
}
</script></body></html>

```

```

<!DOCTYPE HTML><html lang="es"><head><script src="Libreria.js"></script></head><body>
<p id="salida"></p>
<script>
let minimo = 4000;
let maximo = 7000;

//Cantidad de números primos que no tengan el número 3 en sus cifras
let contar = 0;
for (let num = minimo; num <= maximo; num++) {
    if (cifrashalladas(num, 3) === 0 && EsPrimo(num)) {
        contar = contar + 1
    }
}
Imprime(minimo, maximo, "Cantidad de números primos que no tengan el número 3 en sus cifras", contar);

//Imprime el resultado en el navegador
function Imprime(minimo, maximo, texto, resultado) {
    document.getElementById("salida").innerHTML = "Entre " + minimo + " y " + maximo + ". " + texto + ": " +
resultado + "<br>";
}

```

```
}  
</script></body></html>
```

Cap04/008.html

```
<!DOCTYPE HTML><html lang="es"><head><script src="Libreria.js"></script></head><body>  
<p id="salida"></p>  
<script>  
let minimo = 1000;  
let maximo = 9999;  
  
//Cantidad de números palíndromos que tengan mínimo dos veces el 7 en sus cifras  
let contar = 0;  
for (let num = minimo; num <= maximo; num++) {  
    if (cifrashalladas(num, 7) >= 2 && EsPalindromo(num)) {  
        contar = contar + 1  
    }  
}  
Imprime(minimo, maximo, "Cantidad de números palíndromos que tengan mínimo dos veces el 7 en sus cifras",  
contar);  
  
//Imprime el resultado en el navegador  
function Imprime(minimo, maximo, texto, resultado) {  
    document.getElementById("salida").innerHTML = "Entre " + minimo + " y " + maximo + ". " + texto + ": " +  
resultado + "<br>";  
}  
</script></body></html>
```

Cap04/009.html

```
<!DOCTYPE HTML><html lang="es"><head><script src="Libreria.js"></script></head><body>  
<p id="salida"></p>  
<script>  
let minimo = 5000;  
let maximo = 9000;  
  
//Cantidad de números que tengan solo cifras pares y al menos un número 4  
let contar = 0;  
for (let num = minimo; num <= maximo; num++) {  
    if (cifrashalladas(num, 4) >= 1 && TodasCifrasPares(num)) {  
        contar = contar + 1  
    }  
}  
Imprime(minimo, maximo, "Cantidad de números que tengan solo cifras pares y al menos un número 4",  
contar);  
  
//Imprime el resultado en el navegador  
function Imprime(minimo, maximo, texto, resultado) {  
    document.getElementById("salida").innerHTML = "Entre " + minimo + " y " + maximo + ". " + texto + ": " +  
resultado + "<br>";  
}  
</script></body></html>
```

Cap04/010.html

```
<!DOCTYPE HTML><html lang="es"><head><script src="Libreria.js"></script></head><body>  
<p id="salida"></p>  
<script>  
let minimo = 7700;  
let maximo = 8000;  
  
//Cantidad de números que la suma de sus cifras es impar y al menos tenga una vez el número 7  
let contar = 0;  
for (let num = minimo; num <= maximo; num++) {  
    if (EsImpar(SumaCifras(num)) && cifrashalladas(num, 7) >= 1) {  
        contar = contar + 1  
    }  
}  
Imprime(minimo, maximo, "Cantidad de números que la suma de sus cifras es impar y al menos tenga una vez  
el número 7", contar);  
  
//Imprime el resultado en el navegador  
function Imprime(minimo, maximo, texto, resultado) {
```

```
document.getElementById("salida").innerHTML = "Entre " + minimo + " y " + maximo + ". " + texto + ": " + resultado + "<br>";
}
</script></body></html>
```

Cap04/011.html

```
<!DOCTYPE HTML><html lang="es"><head><script src="Libreria.js"></script></head><body>
<p id="salida"></p>
<script>
let minimo = 1000;
let maximo = 9999;

//Cantidad de números primos que sus cifras están entre 3 y 7
let contar = 0;
for (let num = minimo; num <= maximo; num++) {
    if (EsPrimo(num) && LaCifraMasBaja(num) >= 3 && LaCifraMasAlta(num) <= 7) {
        contar = contar + 1
    }
}
Imprime(minimo, maximo, "Cantidad de números primos que sus cifras están entre 3 y 7", contar);

//Imprime el resultado en el navegador
function Imprime(minimo, maximo, texto, resultado) {
    document.getElementById("salida").innerHTML = "Entre " + minimo + " y " + maximo + ". " + texto + ": " + resultado + "<br>";
}
</script></body></html>
```

Cap04/012.html

```
<!DOCTYPE HTML><html lang="es"><head><script src="Libreria.js"></script></head><body>
<p id="salida"></p>
<script>
let minimo = 10;
let maximo = 999999;

//Cantidad de números primos y a su vez palíndromos
let contar = 0;
for (let num = minimo; num <= maximo; num++) {
    if (EsPrimo(num) && EsPalindromo(num)) {
        contar = contar + 1
    }
}
Imprime(minimo, maximo, "Cantidad de números primos y a su vez palíndromos", contar);

//Imprime el resultado en el navegador
function Imprime(minimo, maximo, texto, resultado) {
    document.getElementById("salida").innerHTML = "Entre " + minimo + " y " + maximo + ". " + texto + ": " + resultado + "<br>";
}
</script></body></html>
```



```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Algoritmo de bisección
let Xinicial = 1;
let Xfinal = 2;
let Ciclos = 100; //Número de veces que hará la operación de aproximación

//Verifica que entre Xinicial y Xfinal exista un corte
if (ecuacion(Xinicial) * ecuacion(Xfinal) > 0) {
    document.getElementById("salida").innerHTML = "No hay punto de corte en los puntos dados";
} else { //Si hay intersección, entonces llama a la función de Bisección
    let Xresultado = Biseccion(Xinicial, Xfinal, Ciclos);
    document.getElementById("salida").innerHTML = "Punto de corte en: " + Xresultado;
}

//Retorna el punto X entre Xinicial y Xfinal que más se aproxime al punto de corte
function Biseccion(Xinicial, Xfinal, Ciclos) {
    let Xmedio = 0;
    if (ecuacion(Xinicial) === 0) Xmedio = Xinicial;
    else if (ecuacion(Xfinal) === 0) Xmedio = Xfinal;
    else
        for (let cont = 1; cont <= Ciclos; cont++) {
            Xmedio = (Xinicial + Xfinal) / 2;
            if (ecuacion(Xmedio) === 0) break;
            else if (ecuacion(Xinicial) * ecuacion(Xmedio) < 0) Xfinal = Xmedio;
            else Xinicial = Xmedio;
        }
    return Xmedio;
}

function ecuacion(x) { //En esta función se pone la ecuación
    return 4 * x * x * x - 3 * x * x - 5 * x + 2;
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Ámbito (scope) de una variable
let valor = 17;

//Imprime 17
document.getElementById("salida").innerHTML = "<br>Valor antes de la función es: " + valor;

prueba(); //Llama a la función

//Imprime 17
document.getElementById("salida").innerHTML += "<br>Valor después de la función es: " + valor;

function prueba() {
  //Imprime 17
  document.getElementById("salida").innerHTML += "<br>Valor dentro de la función es: " + valor;
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
let valor = 17;

//Imprime 17
document.getElementById("salida").innerHTML = "<br>Valor antes de la función es: " + valor;

prueba(); //Llama a la función

//Imprime 8906
document.getElementById("salida").innerHTML += "<br>Valor después de la función es: " + valor;

function prueba() {
  //Imprime 17
  document.getElementById("salida").innerHTML += "<br>Valor dentro de la función es: " + valor;

  valor = 8906;

  //Imprime 8906
  document.getElementById("salida").innerHTML = "<br>Valor después de cambiar es: " + valor;
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
let valor = 17;

//Imprime 17
document.getElementById("salida").innerHTML = "<br>Valor antes de la función es: " + valor;

prueba(); //Llama a la función

//Imprime 17
document.getElementById("salida").innerHTML = "<br>Valor después de la función es: " + valor;

function prueba() {
  //Imprime undefined
  document.getElementById("salida").innerHTML = "<br>Valor dentro de la función es: " + valor;

  let valor = 8906;

  //Imprime 8906
  document.getElementById("salida").innerHTML = "<br>Valor variable interna: " + valor;
}
```



```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Ámbito (scope) de una variable
let valor = 17;

//Imprime 17
document.getElementById("salida").innerHTML = "<br>Valor antes de la función es: " + valor;

prueba(valor); //Llama a la función enviando por parámetro el valor

//Imprime 17
document.getElementById("salida").innerHTML = "<br>Valor después de la función es: " + valor;

function prueba(valor) {
  //Imprime 17
  document.getElementById("salida").innerHTML = "<br>Valor dentro de la función es: " + valor;

  valor = 8906;

  //Imprime 8906
  document.getElementById("salida").innerHTML = "<br>Valor variable interna: " + valor;
}
</script></body></html>
```

Funciones recursivas

Funciones que se llaman a sí mismas y deben tener un criterio de salida.

Cap04/018.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
/* Función recursiva
  Factorial(5) = 5 * Factorial(4)
  Es decir Factorial(N) = N * Factorial(N-1);
  y Factorial(0) = 1
*/
function factorial(numero) {
  if (numero > 1)
    return numero * factorial(numero - 1)
  else
    return 1;
}

document.getElementById("salida").innerHTML = "5! = " + factorial(5);
</script></body></html>
```

Cap04/019.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
// Función recursiva. Máximo común divisor con algoritmo de Euclides
function maximocomundivisor(numA, numB) {
  if (numA < numB) return maximocomundivisor(numB, numA);
  if (numB === 0) return numA;
  return maximocomundivisor(numB, numA % numB);
}

let numero = maximocomundivisor(1000, 750);
document.getElementById("salida").innerHTML = "Máximo común divisor de 1000 y 750 es " + numero;
</script></body></html>
```

Cap04/020.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
// Función recursiva. Suma las cifras de un número
function sumacifras(num) {
  if (num < 10) return num;
  return num % 10 + sumacifras(Math.floor(num / 10));
}

document.getElementById("salida").innerHTML = "cifras suman: " + sumacifras(701);
</script></body></html>
```

Cap04/021.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Convertir a binario
ConvierteBinario(70);

function ConvierteBinario(numero) {
  if (numero !== 0) {
    ConvierteBinario(Math.floor(numero / 2));
    document.getElementById("salida").innerHTML += numero % 2;
  }
}
</script></body></html>
```

Cap04/022.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
```

```
<p id="salida"></p>
<script>
//Potencia de un número
let valor = potencia(2, 7); //Retorna 2^7
document.getElementById("salida").innerHTML = valor;

function potencia(numero, elevado) {
  if (elevado === 1) return numero;
  return numero * potencia(numero, elevado - 1);
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Uso de eval como evaluador de expresiones algebraicas
let valor = eval("3+2*5");
document.getElementById("salida").innerHTML = valor;
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Uso de eval como evaluador de expresiones algebraicas
let valor = eval("Math.sin(15)-Math.cos(21)");
document.getElementById("salida").innerHTML = valor;
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Uso de eval como evaluador de expresiones algebraicas
let a = 3;
let b = 5;
let c = 2;
let d = 7;
let valor = eval("a+b-c*d");
document.getElementById("salida").innerHTML = valor;
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Funciones que reciben distinto número de parámetros
UnaFuncion();
UnaFuncion("esto", "es", "una", "prueba");
UnaFuncion(1, 2, 3, 4, 5, 6, 7, 8, 9);
UnaFuncion("palabra", 5, "texto", 7, 8.56);

function UnaFuncion() {
    //total parámetros
    document.getElementById("salida").innerHTML += "<br>total parámetros: " + arguments.length;
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Funciones que reciben distinto número de parámetros
FuncionDinamica("paso", "varios", "argumentos", 7, 1, "numeros", 3, "y texto");

function FuncionDinamica() {
    for (let cont = 0; cont < arguments.length; cont++)
        document.getElementById("salida").innerHTML += arguments[cont] + "<br>";
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
document.getElementById("salida").innerHTML = "Imprimirá en 3 segundos:";
setTimeout(Imprime, 3000); //3000ms = 3segundos

//Esta función ejecutará en 3 segundos
function Imprime() {
    document.getElementById("salida").innerHTML += "<br>Esta es una prueba";
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Cada 100 milisegundos se llamará a la función Imprime()
setInterval(Imprime, 100);

function Imprime() {
    document.getElementById("salida").innerHTML += "<br>Esta es una prueba";
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
let contador = 1;

//Cada 500 milisegundos se llamará a la función setInterval
let ejecuta = setInterval(Imprime, 500);

function Imprime() {
    document.getElementById("salida").innerHTML += "<br>" + contador;
    contador++;

    //La función deja de llamarse
    if (contador === 10) clearInterval(ejecuta);
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head>
<p id="salida"></p>
<script>
/* Trata de mostrar los primos desde el 10.000 hasta un millón.
   Eso toma bastante tiempo por lo que la ventana del navegador
   se congela, no muestra nada y saldrá un aviso de si decide
   detener el script. Al detener se muestran los números primos
   que alcanzó a calcular */
let numero = 10000;
while (numero < 1000000) {
    if (EsPrimo(numero)) document.getElementById("salida").innerHTML += numero + "<br>";
    numero++;
}

function EsPrimo(num) { //Retorna true si el número enviado por parámetro es primo
    if (num <= 1) return false;
    if (num === 2) return true;
    if (num % 2 === 0) return false;
    for (let divide = 3; divide <= Math.sqrt(num); divide += 2)
        if (num % divide === 0) return false;
    return true;
}
</script></body></html>
```



```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
/* Muestra los primos desde el 10.000 hasta un millón.
   Eso toma bastante tiempo pero al usar setInterval se
   pueden mostrar poco a poco los números sin congelar
   la pestaña del navegador.  */
let numero = 10000;
let ejecuta = setInterval(Imprime, 100);
let caja = document.getElementById("caja");

function Imprime() {
  if (EsPrimo(numero)) document.getElementById("salida").innerHTML += numero + "<br>";
  numero++;
}

function EsPrimo(num) { //Retorna true si el número enviado por parámetro es primo
  if (num <= 1) return false;
  if (num === 2) return true;
  if (num % 2 === 0) return false;
  for (let divide = 3; divide <= Math.sqrt(num); divide += 2)
    if (num % divide === 0) return false;
  return true;
}
</script></body></html>
```

## Capítulo 5. Arreglos unidimensionales o vectores



JavaScript trae varias funciones para el trabajo con arreglos unidimensionales. En JavaScript los arreglos unidimensionales siempre inician en la posición cero.

Cap05/001.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>

let lenguajes = []; //Vectores o arreglos
lenguajes.push("C++"); //con "push" adiciona elementos al arreglo
lenguajes.push("C#");
lenguajes.push("Visual Basic .Net");
lenguajes.push("PHP");
lenguajes.push("JavaScript");
lenguajes.push("Object Pascal");
document.getElementById("salida").innerHTML = lenguajes + "<br>"; //Imprime el arreglo unidimensional

let Mamiferos = [];
Mamiferos.push("Gato");
Mamiferos.push("Perro");
Mamiferos.push("Caballo");
Mamiferos.push("Oveja");
document.getElementById("salida").innerHTML += Mamiferos + "<br>";

let Valores = [];
Valores.push(7);
Valores.push(3.2);
Valores.push(-9.17);
document.getElementById("salida").innerHTML += Valores;
</script></body></html>
```

Cap05/002.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
let Numeros = [];
for(let num=1; num<=1000; num++){
    Numeros.push(Math.floor(Math.random()*1000));
}

//¿Cuántos números impares hay?
let CuentaImpares = 0;
for (let cont=0; cont < 30; cont++){
    if (Numeros[cont] % 2 == 1)
        CuentaImpares++;
}
document.getElementById("salida").innerHTML += "<br>Total de impares: " + CuentaImpares;

//¿Cuántos números múltiplos de 7 hay?
</script></body></html>
```

Cap05/003.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
let lenguajes = []; //Otra forma de declarar arreglos
lenguajes.push("C++"); //con "push" adiciona elementos al arreglo
lenguajes.push("C#");
lenguajes.push("Visual Basic .Net");
lenguajes.push("PHP");
lenguajes.push("JavaScript");
lenguajes.push("Object Pascal");
document.getElementById("salida").innerHTML = lenguajes; //Imprime el arreglo unidimensional
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Otra forma de declarar arreglos y mostrarlos
let pagar = [1120, 6040, 1570, 5205, 3147, 7361, 166, 1002, 1572, 6567];
document.getElementById("salida").innerHTML = pagar;
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
let lenguajes = []; //Vectores o arreglos
lenguajes.push("C++"); //con "push" adiciona elementos al arreglo
lenguajes.push("C#");
lenguajes.push("Visual Basic .Net");
lenguajes.push("PHP");
lenguajes.push("JavaScript");
lenguajes.push("Object Pascal");
document.getElementById("salida").innerHTML = lenguajes + "<br>"; //Imprime el arreglo unidimensional

//Desde la posición 1 borre 3 ítems
lenguajes.splice(1, 3);
document.getElementById("salida").innerHTML += lenguajes;
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
let valores = [];
valores.push(15000);
valores.push(18000);
valores.push(34000);
valores.push(17000);
valores.push(8000);

//Operación con vectores: acumulado
let acumula = 0;
for (let cont = 0; cont < valores.length; cont++) {
  acumula += valores[cont];
}

document.getElementById("salida").innerHTML = "Acumulado es: " + acumula;
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Operación con vectores: promedio
let valores = [];
valores.push(15000);
valores.push(18000);
valores.push(34000);
valores.push(17000);
valores.push(8000);

//Acumula los valores
let acumula = 0;
for (let cont = 0; cont < valores.length; cont++) {
  acumula += valores[cont];
}

//Calcula el promedio
let promedio = acumula / valores.length;
document.getElementById("salida").innerHTML = "Promedio es: " + promedio;
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Operación con vectores: Buscar el mayor valor
const valores = [];
valores.push(15000);
valores.push(18000);
valores.push(34000);
valores.push(17000);
valores.push(8000);

let mayor = valores[0]; //Inicia con el primer elemento como el mayor
for (let cont = 0; cont < valores.length; cont++) {
  if (valores[cont] > mayor) mayor = valores[cont];
}

document.getElementById("salida").innerHTML = "Mayor es: " + mayor;
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Ordena arreglos con valores alfanuméricos
let lenguajes = [];
lenguajes.push("Gómez");
lenguajes.push("Ayala");
lenguajes.push("Moreno");
lenguajes.push("Ñañes");
lenguajes.push("Ángulo");
lenguajes.push("Zapata");

document.getElementById("salida").innerHTML = "<b>Arreglo original:</b> " + lenguajes;

lenguajes.sort(); //Ordena el arreglo en orden alfabético

document.getElementById("salida").innerHTML += "<br><b>Arreglo ordenado:</b> " + lenguajes;
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Ordenar de menor a mayor el arreglo con valores numéricos
let valores = [];
valores.push(19000);
valores.push(18000);
valores.push(34000);
valores.push(27000);
valores.push(8000);
valores.push(9000);
document.getElementById("salida").innerHTML = "Arreglo numérico original: " + valores;

valores.sort(function (a, b) {
    return a - b
}); //Ordena el arreglo
document.getElementById("salida").innerHTML += "<br>Arreglo numérico ordenado: " + valores;
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Ordena en forma inversa un arreglo con valores alfanuméricos
let lenguajes = [];
lenguajes.push("Visual Basic .Net");
lenguajes.push("PHP");
lenguajes.push("JavaScript");
lenguajes.push("Java");
lenguajes.push("Object Pascal");
lenguajes.push("C#");
document.getElementById("salida").innerHTML = "<b>Arreglo original:</b> " + lenguajes;

lenguajes.sort(); //Ordena el arreglo en orden alfabético
document.getElementById("salida").innerHTML += "<br><b>Arreglo ordenado:</b> " + lenguajes;

lenguajes.reverse(); //Invierte el arreglo
document.getElementById("salida").innerHTML += "<br><b>Arreglo invertido:</b> " + lenguajes;
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Dado un arreglo extraer los valores únicos
let numeros = [5, 8, 1, 3, 3, 2, 7, 8, 1, 9, 9, 1, 1, 2, 4, 7, 2, 3, 8, 4, 3, 1, 1, 3, 2, 8, 8, 7];
let extrae = [];

for (let cont = 0; cont < numeros.length; cont++)
    if (BuscarEnArreglo(numeros[cont], extrae) === false)
        extrae.push(numeros[cont]);

document.getElementById("salida").innerHTML = extrae;

function BuscarEnArreglo(num, arreglo) { //Retorna true si encuentra num en arreglo
    for (let cont = 0; cont < arreglo.length; cont++)
        if (num === arreglo[cont])
            return true;
    return false;
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
// ¿Función genérica para hallar menor y mayor valor?
let pagar = [1120, 6040, 1570, 5205, 3147, 7361, 166, 1002, 1572, 6567];

let menor = BuscaMenorValor(pagar);
let mayor = BuscaMayorValor(pagar);
document.getElementById("salida").innerHTML = "Menor cuantía es: " + menor;
document.getElementById("salida").innerHTML += "<br>Mayor cuantía es: " + mayor;

function BuscaMenorValor(arreglo) { //Función genérica
    let minimo = arreglo[0];
    for (let cont = 1; cont < arreglo.length; cont++)
        if (arreglo[cont] < minimo) minimo = arreglo[cont];
    return minimo;
}

function BuscaMayorValor(arreglo) { //Función genérica
    let maximo = arreglo[0];
    for (let cont = 1; cont < arreglo.length; cont++)
        if (arreglo[cont] > maximo) maximo = arreglo[cont];
    return maximo;
}
</script></body></html>
```



```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
// Separe los elementos pares e impares de una lista, y retorne ambas listas
let numeros = [10, 13, 20, 24, 23, 2, 23, 29, 5, 19, 8, 6, 16, 2, 6, 27];
let pares = retornaPares(numeros);
let impares = retornaImpares(numeros);
document.getElementById("salida").innerHTML = "Pares: " + pares + "<br>";
document.getElementById("salida").innerHTML += "Impares: " + impares + "<br>";

function retornaPares(valores) {
  let par = [];
  for (let cont = 0; cont < valores.length; cont++)
    if (valores[cont] % 2 === 0)
      par.push(valores[cont]);
  return par;
}

function retornaImpares(valores) {
  let impar = [];
  for (let cont = 0; cont < valores.length; cont++)
    if (valores[cont] % 2 !== 0)
      impar.push(valores[cont]);
  return impar;
}
</script></body></html>
```

Implementación de algoritmos de ordenación

Algoritmo de burbuja

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
let letras = ['r', 'v', 'e', 'g', 't', 'u', 'd', 'f', 'f', 'y', 'v'];
document.getElementById("salida").innerHTML = "Original: " + letras + "<br>";
OrdenarBurbuja(letras);
document.getElementById("salida").innerHTML += "Ordenado: " + letras + "<br>";

//Ordenación por el método de burbuja
function OrdenarBurbuja(arreglo) {
  for (let i = 0; i < arreglo.length - 1; i++)
    for (let j = 0; j < arreglo.length - i - 1; j++)
      if (arreglo[j] > arreglo[j + 1]) {
        let aux = arreglo[j];
        arreglo[j] = arreglo[j + 1];
        arreglo[j + 1] = aux;
      }
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Arreglo de números enteros
let datos = [-363, -1793, -2601, -2541, 6376, 6053, 503, 1601, -2994, 6480, 998];
document.getElementById("salida").innerHTML = "Original: " + datos + "<br>";
OrdenarBurbuja(datos);
document.getElementById("salida").innerHTML += "Ordenado: " + datos + "<br>";

//Ordenación por burbuja
function OrdenarBurbuja(arreglo) {
  for (let i = 0; i < arreglo.length - 1; i++)
    for (let j = 0; j < arreglo.length - i - 1; j++)
```

```
        if (arreglo[j] > arreglo[j + 1]) {
            let aux = arreglo[j];
            arreglo[j] = arreglo[j + 1];
            arreglo[j + 1] = aux;
        }
    }
</script></body></html>
```

Cap05/017.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
let datos = [-363, -1793, -2601, -2541, 6376, 6053, 503, 1601, -2994];
document.getElementById("salida").innerHTML = "Original: " + datos + "<br>";
OrdenarBurbujaMejorado(datos);
document.getElementById("salida").innerHTML += "Ordenado: " + datos + "<br>";

//Ordenación por método de burbuja mejorado
function OrdenarBurbujaMejorado(arreglo) {
    let intercambio;
    do {
        intercambio = false;
        for (let i = 0; i < arreglo.length - 1; i++) {
            if (arreglo[i] > arreglo[i + 1]) {
                let temporal = arreglo[i];
                arreglo[i] = arreglo[i + 1];
                arreglo[i + 1] = temporal;
                intercambio = true;
            }
        }
    } while (intercambio);
}
</script></body></html>
```



```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
let letras = ['r', 'v', 'e', 'g', 't', 'u', 'd', 'f', 'f', 'y', 'v'];
document.getElementById("salida").innerHTML = "Original: " + letras + "<br>";
OrdenarSeleccion(letras);
document.getElementById("salida").innerHTML += "Ordenado: " + letras + "<br>";

//Ordenación por selección
function OrdenarSeleccion(arreglo) {
  for (let i = 0; i < arreglo.length - 1; i++) {
    let tmp = i;
    for (let j = i + 1; j < arreglo.length; j++)
      if (arreglo[j] < arreglo[tmp])
        tmp = j;

    let temporal = arreglo[tmp];
    arreglo[tmp] = arreglo[i];
    arreglo[i] = temporal;
  }
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
let datos = [-363, -1793, -2601, -2541, 6376, 6053, 503, 1601, -2994, 648];
document.getElementById("salida").innerHTML = "Original: " + datos + "<br>";
OrdenarSeleccion(datos);
document.getElementById("salida").innerHTML += "Ordenado: " + datos + "<br>";

//Ordenación por selección
function OrdenarSeleccion(arreglo) {
  for (let i = 0; i < arreglo.length - 1; i++) {
    let tmp = i;
    for (let j = i + 1; j < arreglo.length; j++)
      if (arreglo[j] < arreglo[tmp])
        tmp = j;

    let temporal = arreglo[tmp];
    arreglo[tmp] = arreglo[i];
    arreglo[i] = temporal;
  }
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
let letras = ['r', 'v', 'e', 'g', 't', 'u', 'd', 'f', 'f', 'y', 'v', 'm', 'z', 'v'];
document.getElementById("salida").innerHTML = "Original: " + letras + "<br>";
OrdenarInsercion(letras);
document.getElementById("salida").innerHTML += "Ordenado: " + letras + "<br>";

//Algoritmo de ordenación por inserción
function OrdenarInsercion(arreglo) {
  for (let i = 0, j, tmp; i < arreglo.length; ++i) {
    tmp = arreglo[i];
    for (j = i - 1; j >= 0 && arreglo[j] > tmp; --j)
      arreglo[j + 1] = arreglo[j];
    arreglo[j + 1] = tmp;
  }
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
let datos = [-363, -1793, -2601, -2541, 6376, 6053, 503, 1601, -2994, 648];
document.getElementById("salida").innerHTML = "Original: " + datos + "<br>";
OrdenarInsercion(datos);
document.getElementById("salida").innerHTML += "Ordenado: " + datos + "<br>";

//Algoritmo de ordenación por inserción
function OrdenarInsercion(arreglo) {
  for (let i = 0, j, tmp; i < arreglo.length; ++i) {
    tmp = arreglo[i];
    for (j = i - 1; j >= 0 && arreglo[j] > tmp; --j)
      arreglo[j + 1] = arreglo[j];
    arreglo[j + 1] = tmp;
  }
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
let letras = ['r', 'v', 'e', 'g', 't', 'u', 'd', 'f', 'f', 'y', 'v'];
document.getElementById("salida").innerHTML = "Original: " + letras + "<br>";
let ordenado = quicksort(letras);
document.getElementById("salida").innerHTML = "Ordenado: " + ordenado + "<br>";

//Algoritmo de ordenación QuickSort
function quicksort(arreglo) {
  if (arreglo.length === 0) return [];
  let izquierdo = [], derecho = [], pivote = arreglo[0];
  for (let i = 1; i < arreglo.length; i++)
    if (arreglo[i] < pivote)
      izquierdo.push(arreglo[i]);
    else
      derecho.push(arreglo[i]);

  return quicksort(izquierdo).concat(pivote, quicksort(derecho));
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
let datos = [-363, -1793, -2601, -2541, 6376, 6053, 503, 1601, -2994, 648];
document.getElementById("salida").innerHTML = "Original: " + datos + "<br>";
let ordenado = quicksort(datos);
document.getElementById("salida").innerHTML += "Ordenado: " + ordenado + "<br>";

function quicksort(arreglo) {
  if (arreglo.length === 0) return [];
  let izquierdo = [], derecho = [], pivote = arreglo[0];
  for (let i = 1; i < arreglo.length; i++)
    if (arreglo[i] < pivote)
      izquierdo.push(arreglo[i]);
    else
      derecho.push(arreglo[i]);

  return quicksort(izquierdo).concat(pivote, quicksort(derecho));
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
let letras = ['r', 'v', 'e', 'g', 't', 'u', 'd', 'f', 'f', 'y', 'v'];
document.getElementById("salida").innerHTML = "Original: " + letras + "<br>";
let ordenado = mergesort(letras);
document.getElementById("salida").innerHTML += "Ordenado: " + ordenado + "<br>";

function mergesort(arreglo) {
  if (arreglo.length <= 1) return arreglo;
  let mitad = Math.floor(arreglo.length / 2);
  let izquierdo = arreglo.slice(0, mitad);
  let derecho = arreglo.slice(mitad, arreglo.length);
  return merge(mergesort(izquierdo), mergesort(derecho));
}

function merge(izquierdo, derecho) {
  let ordenado = [];
  while (izquierdo && izquierdo.length > 0 && derecho && derecho.length > 0) {
    let b = izquierdo[0] <= derecho[0];
    if (b) ordenado.push(izquierdo[0]); else ordenado.push(derecho[0]);
    if (b) izquierdo.splice(0, 1); else derecho.splice(0, 1);
  }
  return ordenado.concat(izquierdo, derecho);
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
let datos = [-363, -1793, -2601, -2541, 6376, 6053, 503, 1601, -2994, 648];
document.getElementById("salida").innerHTML = "Original: " + datos + "<br>";
let ordenado = mergesort(datos);
document.getElementById("salida").innerHTML += "Ordenado: " + ordenado + "<br>";

function mergesort(arreglo) {
  if (arreglo.length <= 1) return arreglo;
  let mitad = Math.floor(arreglo.length / 2);
  let izquierdo = arreglo.slice(0, mitad);
  let derecho = arreglo.slice(mitad, arreglo.length);
  return merge(mergesort(izquierdo), mergesort(derecho));
}

function merge(izquierdo, derecho) {
  let ordenado = [];
  while (izquierdo && izquierdo.length > 0 && derecho && derecho.length > 0) {
    let b = izquierdo[0] <= derecho[0];
    if (b) ordenado.push(izquierdo[0]); else ordenado.push(derecho[0]);
    if (b) izquierdo.splice(0, 1); else derecho.splice(0, 1);
  }
  return ordenado.concat(izquierdo, derecho);
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Mostrar la fecha actual
let fecha = new Date(); //Crea una variable tipo fecha
let dia = fecha.getDate(); //Trae el día
let mes = fecha.getMonth() + 1; //Trae el mes (comienza en 0 para Enero)
let anyo = fecha.getFullYear(); //Trae el año
document.getElementById("salida").innerHTML = "Hoy es " + dia + "/" + mes + "/" + anyo;
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
let meses = ["enero", "febrero", "marzo", "abril", "mayo", "junio", "julio", "agosto", "septiembre",
"octubre", "noviembre", "diciembre"];
let fecha = new Date();
let dia = fecha.getDate();
let mes = fecha.getMonth();
let anyo = fecha.getFullYear();
let mesnombre = meses[mes]; //Convierte el valor numérico en nombre de mes
document.getElementById("salida").innerHTML = dia + " de " + mesnombre + " de " + anyo;
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
let meses = ["enero", "febrero", "marzo", "abril", "mayo", "junio", "julio", "agosto", "septiembre",
"octubre", "noviembre", "diciembre"];
let diasemana = ["domingo", "lunes", "martes", "miércoles", "jueves", "viernes", "sábado"];
let fecha = new Date();
let dia = fecha.getDate();
let mes = fecha.getMonth();
let anyo = fecha.getFullYear();
let diadelasemana = fecha.getDay();
let mesnombre = meses[mes]; //Convierte el valor numérico en nombre de mes
let dianombre = diasemana[diadelasemana]; //Convierte el valor numérico en nombre de día
document.getElementById("salida").innerHTML = dianombre + ", " + dia + " de " + mesnombre + " de " + anyo;
</script></body></html>
```

## Capítulo 6. Manejo de cadenas o strings



```

<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Concatenar cadenas
let cadena1 = "Dragón";
let cadena2 = " de ";
let cadena3 = "Komodo";
let cadena4 = cadena1 + cadena2 + cadena3;
document.getElementById("salida").innerHTML = cadena4;
</script></body></html>

```

```

<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Concatenar cadenas y valores numéricos
let cadena1 = 3;
let cadena2 = "abcde";
let cadena3 = 5;
let cadena4 = cadena1 + cadena2 + cadena3 + "efg";
document.getElementById("salida").innerHTML = cadena4;
</script></body></html>

```

```

<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
/* Extraer una letra de una cadena. Las cadenas se comportan como arreglos unidimensionales, el primer
elemento está en la posición 0 */
let cadena = "Esta es una prueba";
let letra = cadena[0]; //Trae la primera letra
document.getElementById("salida").innerHTML = letra;
</script></body></html>

```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Longitud de una cadena (incluye los espacios)
let cadena = "    Esta es una prueba    ";
let longitud = cadena.length;
document.getElementById("salida").innerHTML = longitud;
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Convertir a mayúsculas y minúsculas
let cadena = "EsTa Es UNa PrUEBa dE CAmbio de MAYúscULas y MINúscULas";
let mayuscula = cadena.toUpperCase(); //Convierte a mayúsculas
let minuscula = cadena.toLowerCase(); //Convierte a minúsculas
document.getElementById("salida").innerHTML = mayuscula + "<br>" + minuscula;
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Comparación de cadenas
let cadena1 = "FraseA";
let cadena2 = "FraseA";
if (cadena1 === cadena2) //Comparación estricta al usar ===
    document.getElementById("salida").innerHTML = "iguales";
else
    document.getElementById("salida").innerHTML = "diferentes";
</script></body></html>
```



```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Invierte la cadena
let cadena = "Rafael Alberto Moreno Parra";
let nueva = "";
for (let cont = 0; cont < cadena.length; cont++) nueva = cadena[cont] + nueva;

document.getElementById("salida").innerHTML = "Original: " + cadena + "<br>";
document.getElementById("salida").innerHTML += "Invierte: " + nueva + "<br>";
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Quita los espacios de una cadena
let cadena = "  a b  c  d e f ";
let nueva = "";
for (let cont = 0; cont < cadena.length; cont++)
    if (cadena[cont] !== ' ')
        nueva += cadena[cont];

document.getElementById("salida").innerHTML = "Original: [" + cadena + "<br>";
document.getElementById("salida").innerHTML = "Sin espacios : [" + nueva + "<br>";
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Quitar vocales (minúsculas, mayúsculas, tildadas)
let cadena = prompt("Escriba un texto");
let nueva = "";
for (let cont = 0; cont < cadena.length; cont++)
    if (!EsVocal(cadena[cont])) nueva += cadena[cont];
document.getElementById("salida").innerHTML = "Original: " + cadena + "<br>";
document.getElementById("salida").innerHTML = "Sin vocales : " + nueva + "<br>";

function EsVocal(caracter) { //Retorna true si el caracter es una vocal
    let vocales = "aeiouAEIOUÁÉÍÓÚáéíóúäëïöü";
    for (let letra = 0; letra < vocales.length; letra++)
        if (caracter === vocales[letra])
            return true;

    return false;
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Retirar caracteres no permitidos de una cadena
let cadena = "Esto es una <b>prueba</b>";
let nueva = "";
for (let cont = 0; cont < cadena.length; cont++)
    if (EsPermitido(cadena[cont])) nueva += cadena[cont];

document.getElementById("salida").innerHTML = "Original: " + cadena + "<br>";
document.getElementById("salida").innerHTML = "Con los caracteres permitidos: " + nueva + "<br>";

//Retorna true si el caracter está permitido
function EsPermitido(caracter) {
    let permite = "ÁÉÍÓÚáéíóúäëïöü";
    permite += "abcdefghijklmnopqrstuvwxyz";
    permite += "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
    permite += " 1234567890";
    for (let letra = 0; letra < permite.length; letra++)
        if (caracter === permite[letra])
            return true;
    return false;
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Cifrado sencillo
let cadena = prompt("Escriba un texto");
let cifrada = Cifrar(cadena);
document.getElementById("salida").innerHTML = "Original: " + cadena + "<br>";
document.getElementById("salida").innerHTML += "Cifrado: " + cifrada + "<br>";

let descifra = DesCifrar(cifrada);
document.getElementById("salida").innerHTML += "Descifrado: " + descifra + "<br>";

//Cifra la cadena moviendo un caracter hacia adelante
function Cifrar(cadena) {
    let cifrada = "";
    for (let cont = 0; cont < cadena.length; cont++) {
        let num = cadena[cont].charCodeAt(); //Trae el valor ASCII de la letra
        num++; //Incrementa en uno ese valor
        cifrada += String.fromCharCode(num); //Convierte el valor a su respectivo caracter
    }
    return cifrada;
}

//DesCifra la cadena cifrada moviendo un caracter hacia atrás
function DesCifrar(cadena) {
    let descifrada = "";
    for (let cont = 0; cont < cadena.length; cont++) {
        let num = cadena[cont].charCodeAt(); //Trae el valor ASCII de la letra
        num--; //Incrementa en uno ese valor
        descifrada += String.fromCharCode(num); //Convierte el valor a su respectivo caracter
    }
    return descifrada;
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Cifrado sencillo con clave de cifrado
let cadena = prompt("Escriba texto");
let clave = prompt("Clave de cifrado");

let cifrada = Cifrar(cadena, clave);
document.getElementById("salida").innerHTML = "Original: " + cadena + "<br>";
document.getElementById("salida").innerHTML += "Cifrado: " + cifrada + "<br>";

let descifra = Descifrar(cifrada, clave);
document.getElementById("salida").innerHTML += "Descifrado: " + descifra + "<br>";

function Cifrar(cadena, clave) {
    let cifrada = "";
    let clavecont = 0;
    for (let cont = 0; cont < cadena.length; cont++) {
        //Cada letra de la clave genera un desplazamiento
        let desplaza = clave[clavecont++].charCodeAt() % 20;
        if (clavecont >= clave.length) clavecont = 0; //Si se llega al final de la clave
        cifrada += String.fromCharCode(cadena[cont].charCodeAt() + desplaza);
    }
    return cifrada;
}

function Descifrar(cadena, clave) {
    let descifrada = "";
    let clavecont = 0;
    for (let cont = 0; cont < cadena.length; cont++) {
        //Cada letra de la clave genera un desplazamiento
        let desplaza = clave[clavecont++].charCodeAt() % 20;
        if (clavecont >= clave.length) clavecont = 0; //Si se llega al final de la clave
        descifrada += String.fromCharCode(cadena[cont].charCodeAt() - desplaza);
    }
    return descifrada;
}
</script></body></html>
```



```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Suma dos números enteros positivos almacenados en cadenas
let numeroA = "395763487398655335"; //Primer número
let numeroB = "888888888888888888"; //Segundo número
let resultado = ""; //Guarda el resultado
let ultimaPosA = numeroA.length - 1; //Posición del último dígito del primer número
let ultimaPosB = numeroB.length - 1;
let llevar = false; //Se activa si la suma de dígitos es 10 o mas
while (ultimaPosA >= 0 || ultimaPosB >= 0) { //Suma del dígito menos significativo al más significativo
    let suma = 0; //Suma de dígitos
    if (ultimaPosA >= 0 && ultimaPosB >= 0) //Si ambos dígitos existen en esas posiciones
        suma = parseInt(numeroA[ultimaPosA]) + parseInt(numeroB[ultimaPosB]);
    else if (ultimaPosA >= 0) //Si sólo existe dígito en primer número
        suma = parseInt(numeroA[ultimaPosA]);
    else //solo existe dígito en segundo número
        suma = parseInt(numeroB[ultimaPosB]);
    if (llevar === true) suma++; //Si llevaba aumenta en 1 la suma
    llevar = false; //La bandera de llelet se apaga
    if (suma >= 10) {
        llevar = true;
        suma -= 10;
    } //Si la suma es 10 o más, entonces lleva, le quita 10
    resultado = suma + resultado; //Agrega al inicio de la cadena
    ultimaPosA--; //Va hacia la izquierda del primer número
    ultimaPosB--; //Va hacia la izquierda del segundo número
}
if (llevar === true) resultado = "1" + resultado; //Si la suma de los primeros dígitos es 10 o más
document.getElementById("salida").innerHTML = numeroA + " + <br>" + numeroB + "<br>-----"
--<br>" + resultado;
</script></body></html>
```

## Capítulo 7. Arreglos bidimensionales



```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Crea el arreglo bidimensional
let bidiarreglo = new Array(2); //Crea dos filas

//Crea cuatro columnas por fila
for (let fila = 0; fila < bidiarreglo.length; fila++)
    bidiarreglo[fila] = new Array(4);

//Tenemos un arreglo bidimensional de 2 filas * 4 columnas.
//Las coordenadas son [fila][columna]. Empiezan en [0,0] y terminan en [1,3]
bidiarreglo[0][0] = 11;
bidiarreglo[0][1] = 13;
bidiarreglo[0][2] = 15;
bidiarreglo[0][3] = 17;

bidiarreglo[1][0] = 21;
bidiarreglo[1][1] = 23;
bidiarreglo[1][2] = 25;
bidiarreglo[1][3] = 27;

//Imprimir el arreglo bidimensional
for (let fila = 0; fila < bidiarreglo.length; fila++) {
    document.getElementById("salida").innerHTML += "<br>";
    for (let columna = 0; columna < bidiarreglo[fila].length; columna++)
        document.getElementById("salida").innerHTML += bidiarreglo[fila][columna] + " , ";
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Crea el arreglo bidimensional
let tabla = new Array(8); //Crea las filas

//Crea las columnas
for (let fila = 0; fila < tabla.length; fila++)
    tabla[fila] = new Array(20);

//Llena el arreglo bidimensional de puntos
for (let fila = 0; fila < tabla.length; fila++)
    for (let columna = 0; columna < tabla[fila].length; columna++)
        tabla[fila][columna] = '.';

//Pone una X en una posición al azar
let posF = Math.floor(Math.random() * 8);
let posC = Math.floor(Math.random() * 20);
tabla[posF][posC] = 'X';

//Imprime la tabla
for (let fila = 0; fila < tabla.length; fila++) {
    document.getElementById("salida").innerHTML += "<br>";
    for (let columna = 0; columna < tabla[fila].length; columna++)
        document.getElementById("salida").innerHTML += tabla[fila][columna] + " , ";
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Llama a función que genera arreglo bidimensional
let tabla = GenerarArregloBidimensional(8);

//Llena el arreglo bidimensional de puntos
for (let fila = 0; fila < tabla.length; fila++)
  for (let columna = 0; columna < 20; columna++)
    tabla[fila][columna] = '.';

//Pone una X en una posición al azar
let posF = Math.floor(Math.random() * 8);
let posC = Math.floor(Math.random() * 20);
tabla[posF][posC] = 'X';

//Imprime la tabla
for (let fila = 0; fila < tabla.length; fila++) {
  document.getElementById("salida").innerHTML += "<br>";
  for (let columna = 0; columna < tabla[fila].length; columna++)
    document.getElementById("salida").innerHTML += tabla[fila][columna] + " , ";
}

//Función genérica para crear arreglo bidimensional
function GenerarArregloBidimensional(totalfilas) {
  let arreglo = [];
  for (let fila = 0; fila < totalfilas; fila++) arreglo[fila] = [];
  return arreglo;
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
let tabla = GeneraArregloBidimensional(10, 15, '.');
ImprimeArregloBidimensional(tabla);

/* Función genérica para crear arreglos bidimensionales
   con el número de filas y columnas dado y llena cada celda
   de un valor */
function GeneraArregloBidimensional(filas, columnas, valor) {
  let arreglo = [];
  for (let fila = 0; fila < filas; fila++) {
    arreglo[fila] = [];
    for (let columna = 0; columna < columnas; columna++)
      arreglo[fila][columna] = valor;
  }
  return arreglo;
}

//Función genérica para imprimir un arreglo bidimensional
function ImprimeArregloBidimensional(arreglo) {
  for (let fila = 0; fila < arreglo.length; fila++) {
    document.getElementById("salida").innerHTML += "<br>";
    for (let columna = 0; columna < arreglo[fila].length; columna++)
      document.getElementById("salida").innerHTML += arreglo[fila][columna];
  }
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
/* Crear un tablero de ajedrez de 8*8, poner en una posición al
   azar una Torre y mostrar su ataque */

let tabla = GeneraArregloBidimensional(8, 8, '.');
PoneTorreAzar(tabla);
ImprimeArregloBidimensional(tabla);

//Pone una torre al azar y muestra su ataque
function PoneTorreAzar(arreglo) {
  let filaAzar = Math.floor(Math.random() * arreglo.length);
  let coluAzar = Math.floor(Math.random() * arreglo[0].length);
  for (let fil = 0; fil < arreglo.length; fil++) arreglo[fil][coluAzar] = 'x';
  for (let col = 0; col < arreglo[0].length; col++) arreglo[filaAzar][col] = 'x';
  arreglo[filaAzar][coluAzar] = 'T';
}

function GeneraArregloBidimensional(filas, columnas, valor) {
  let arreglo = [];
  for (let fila = 0; fila < filas; fila++) {
    arreglo[fila] = [];
    for (let columna = 0; columna < columnas; columna++)
      arreglo[fila][columna] = valor;
  }
  return arreglo;
}

function ImprimeArregloBidimensional(arreglo) {
  for (let fila = 0; fila < arreglo.length; fila++) {
    document.getElementById("salida").innerHTML += "<p style='font-family:courier new;'>";
    for (let columna = 0; columna < arreglo[fila].length; columna++)
      document.getElementById("salida").innerHTML += arreglo[fila][columna];
    document.getElementById("salida").innerHTML += "</p>";
  }
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
/* Crear un tablero de ajedrez de 8*8, poner en una posición al
   azar el Rey y mostrar su ataque */
let tabla = GeneraArregloBidimensional(8, 8, '.');
PoneReyAzar(tabla);
ImprimeArregloBidimensional(tabla);

//Pone un Rey al azar y muestra su ataque
function PoneReyAzar(arreglo) {
  let filaAzar = Math.floor(Math.random() * arreglo.length);
  let coluAzar = Math.floor(Math.random() * arreglo[0].length);
  if (filaAzar > 0) arreglo[filaAzar - 1][coluAzar] = 'x';
  if (filaAzar > 0 && coluAzar > 0) arreglo[filaAzar - 1][coluAzar - 1] = 'x';
  if (coluAzar > 0) arreglo[filaAzar][coluAzar - 1] = 'x';
  if (coluAzar < arreglo[0].length - 1) arreglo[filaAzar][coluAzar + 1] = 'x';
  if (filaAzar > 0 && coluAzar < arreglo[0].length - 1) arreglo[filaAzar - 1][coluAzar + 1] = 'x';
  if (filaAzar < arreglo.length - 1 && coluAzar < arreglo[0].length - 1) arreglo[filaAzar + 1][coluAzar +
1] = 'x';
  if (filaAzar < arreglo.length - 1) arreglo[filaAzar + 1][coluAzar] = 'x';
  if (filaAzar < arreglo.length - 1 && coluAzar > 0) arreglo[filaAzar + 1][coluAzar - 1] = 'x';
  arreglo[filaAzar][coluAzar] = 'R';
}

function GeneraArregloBidimensional(filas, columnas, valor) {
  let arreglo = [];
  for (let fila = 0; fila < filas; fila++) {
    arreglo[fila] = [];
    for (let columna = 0; columna < columnas; columna++)
      arreglo[fila][columna] = valor;
  }
  return arreglo;
}

function ImprimeArregloBidimensional(arreglo) {
  for (let fila = 0; fila < arreglo.length; fila++) {
    document.getElementById("salida").innerHTML += "<p style='font-family:courier new;'>";
    for (let columna = 0; columna < arreglo[fila].length; columna++)
      document.getElementById("salida").innerHTML += arreglo[fila][columna];
    document.getElementById("salida").innerHTML += "</p>";
  }
}
</script></body></html>
```





```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
/* Crear un tablero de ajedrez de 8*8, poner en una posición al
  azar un Alfil y mostrar su ataque */
let tabla = GeneraArregloBidimensional(8, 8, '.');
PoneAlfilAzar(tabla);
ImprimeArregloBidimensional(tabla);

function PoneAlfilAzar(arreglo) {
  let filaAzar = Math.floor(Math.random() * arreglo.length);
  let coluAzar = Math.floor(Math.random() * arreglo[0].length);
  let tmpFil = filaAzar;
  let tmpCol = coluAzar;
  while (tmpFil >= 0 && tmpCol >= 0) {
    arreglo[tmpFil][tmpCol] = 'x';
    tmpFil--;
    tmpCol--;
  }
  tmpFil = filaAzar;
  tmpCol = coluAzar;
  while (tmpFil >= 0 && tmpCol < 8) {
    arreglo[tmpFil][tmpCol] = 'x';
    tmpFil--;
    tmpCol++;
  }
  tmpFil = filaAzar;
  tmpCol = coluAzar;
  while (tmpFil < 8 && tmpCol < 8) {
    arreglo[tmpFil][tmpCol] = 'x';
    tmpFil++;
    tmpCol++;
  }
  tmpFil = filaAzar;
  tmpCol = coluAzar;
  while (tmpFil < 8 && tmpCol >= 0) {
    arreglo[tmpFil][tmpCol] = 'x';
    tmpFil++;
    tmpCol--;
  }
  arreglo[filaAzar][coluAzar] = 'A';
}

function GeneraArregloBidimensional(filas, columnas, valor) {
  let arreglo = [];
  for (let fila = 0; fila < filas; fila++) {
    arreglo[fila] = [];
    for (let columna = 0; columna < columnas; columna++)
      arreglo[fila][columna] = valor;
  }
  return arreglo;
}

function ImprimeArregloBidimensional(arreglo) {
  for (let fila = 0; fila < arreglo.length; fila++) {
    document.getElementById("salida").innerHTML += "<p style='font-family:courier new;'>";
    for (let columna = 0; columna < arreglo[fila].length; columna++)
      document.getElementById("salida").innerHTML += arreglo[fila][columna];
    document.getElementById("salida").innerHTML += "</p>";
  }
}
</script></body></html>
```





```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
/* Crear un tablero de ajedrez de 8*8, poner en una posición al
   azar la Reina y mostrar su ataque */
let tabla = GeneraArregloBidimensional(8, 8, '.');
PoneReinaAzar(tabla);
ImprimeArregloBidimensional(tabla);

function PoneReinaAzar(arreglo) {
  let filaAzar = Math.floor(Math.random() * arreglo.length);
  let coluAzar = Math.floor(Math.random() * arreglo[0].length);
  let tmpFil = filaAzar;
  let tmpCol = coluAzar;
  while (tmpFil >= 0 && tmpCol >= 0) {
    arreglo[tmpFil][tmpCol] = 'x';
    tmpFil--;
    tmpCol--;
  }
  tmpFil = filaAzar;
  tmpCol = coluAzar;
  while (tmpFil >= 0 && tmpCol < 8) {
    arreglo[tmpFil][tmpCol] = 'x';
    tmpFil--;
    tmpCol++;
  }
  tmpFil = filaAzar;
  tmpCol = coluAzar;
  while (tmpFil < 8 && tmpCol < 8) {
    arreglo[tmpFil][tmpCol] = 'x';
    tmpFil++;
    tmpCol++;
  }
  tmpFil = filaAzar;
  tmpCol = coluAzar;
  while (tmpFil < 8 && tmpCol >= 0) {
    arreglo[tmpFil][tmpCol] = 'x';
    tmpFil++;
    tmpCol--;
  }
  for (tmpFil = 0; tmpFil < 8; tmpFil++) arreglo[tmpFil][coluAzar] = 'x';
  for (tmpCol = 0; tmpCol < 8; tmpCol++) arreglo[filaAzar][tmpCol] = 'x';
  arreglo[filaAzar][coluAzar] = 'R';
}

function GeneraArregloBidimensional(filas, columnas, valor) {
  let arreglo = [];
  for (let fila = 0; fila < filas; fila++) {
    arreglo[fila] = [];
    for (let columna = 0; columna < columnas; columna++)
      arreglo[fila][columna] = valor;
  }
  return arreglo;
}

function ImprimeArregloBidimensional(arreglo) {
  for (let fila = 0; fila < arreglo.length; fila++) {
    document.getElementById("salida").innerHTML += "<p style='font-family:courier new;'>";
    for (let columna = 0; columna < arreglo[fila].length; columna++)
      document.getElementById("salida").innerHTML += arreglo[fila][columna];
    document.getElementById("salida").innerHTML += "</p>";
  }
}
</script></body></html>
```

Ruta del menor costo

Dada una tabla (arreglo bidimensional) de costos que muestra cuánto cuesta ir de una ciudad a otra. El reto es encontrar una ruta que visite todas las ciudades, sin repetir ninguna y que tenga el menor coste. Si nos ponemos sistemáticos habría que probar  $N!$  ( $N$  factorial) rutas para encontrar la de menor costo, un proceso muy costoso computacionalmente y no terminaría en un tiempo justo. El método mostrado es usando una técnica indeterminista que se acerca a la ruta menos costosa.

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
/* Hay N ciudades a recorrer (0 a N-1). Sólo se puede visitar una ciudad por vez.
   En la tabla aparece cuanto cuesta ir de una ciudad (origen) a otra ciudad (destino).
   ¿Qué ruta tomar para visitar todas las ciudades con el mínimo costo? */

let ciudad = 20; //Número de ciudades
let minValor = 15; //Valor mínimo que tendrá ir de una ciudad a otra
let maxValor = 85; //Valor máximo que tendrá ir de una ciudad a otra

let dest1, dest2; //Ciudad origen a Ciudad destino

//Genera valores de viaje al azar
let valorviajes = inicializavalorviajes(ciudad, minValor, maxValor);

//Imprime los valores
imprime(valorviajes);

//Inicia con una ruta predeterminada 0, 1, 2, 3, .... N ... 0
let ruta = iniciaRuta(ciudad);

//Deduce el costo de esa ruta predeterminada
let costo = DeduceCosto(ruta, valorviajes);
document.getElementById("salida").innerHTML += "<br>" + ruta + "=>" + costo;

//Usando el método Monte Carlo se buscarán otras rutas con menor costo
for (let pruebas = 1; pruebas <= 500000; pruebas++) {
  dest1 = Math.floor(Math.random() * ruta.length);
  do {
    dest2 = Math.floor(Math.random() * ruta.length);
  } while (dest2 === dest1)
  ModificaRuta(ruta, dest1, dest2)
  let costoNuevo = DeduceCosto(ruta, valorviajes);
  if (costoNuevo < costo) {
    costo = costoNuevo;
    document.getElementById("salida").innerHTML += "<br>" + ruta + "=>" + costo;
  } else
    ModificaRuta(ruta, dest1, dest2); //Dejar la ruta como antes
}

//Modifica la ruta de viaje
function ModificaRuta(ruta, dest1, dest2) {
  let tmp = ruta[dest1];
  ruta[dest1] = ruta[dest2];
  ruta[dest2] = tmp;
}

//Inicia el arreglo bidimensional de rutas
function iniciaRuta(limite) {
  const ruta = [];
  for (let cont = 0; cont < limite; cont++) ruta.push(cont);
  return ruta;
}

//Deduce el costo de la ruta de viaje
function DeduceCosto(ruta, costos) {
  let acum = 0;
  for (let cont = 0; cont < ruta.length - 1; cont++)
    acum += costos[ruta[cont]][ruta[cont + 1]];
  return acum;
}

//Llena de valores al azar la tabla de costos de viajes de una ciudad a otra
function inicializavalorviajes(ciudad, minValor, maxValor) {
  let tablero = [];
  for (let fila = 0; fila < ciudad; fila++) {
```

```
    tablero[fila] = [];  
    for (let columna = 0; columna < ciudad; columna++)  
        tablero[fila][columna] = Math.floor(Math.random() * (maxValor - minValor) + minValor);  
}  
for (let cont = 0; cont < ciudad; cont++) tablero[cont][cont] = 0;  
return tablero;  
}  
  
//Imprime el tablero  
function imprime(tablero) {  
    for (let fila = 0; fila < tablero.length; fila++) {  
        for (let col = 0; col < tablero.length; col++)  
            document.getElementById("salida").innerHTML += tablero[fila][col] + " ";  
        document.getElementById("salida").innerHTML += "<br>";  
    }  
}  
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
let sudoku = /* El Sudoku como es planteado. Los ceros(0) son las casillas vacías a completar */
  [[5, 3, 0, 0, 7, 0, 0, 0, 0],
   [6, 0, 0, 1, 9, 5, 0, 0, 0],
   [0, 9, 8, 0, 0, 0, 0, 6, 0],
   [8, 0, 0, 0, 6, 0, 0, 0, 3],
   [4, 0, 0, 8, 0, 3, 0, 0, 1],
   [7, 0, 0, 0, 2, 0, 0, 0, 6],
   [0, 6, 0, 0, 0, 0, 2, 8, 0],
   [0, 0, 0, 4, 1, 9, 0, 0, 5],
   [0, 0, 0, 0, 8, 0, 0, 7, 9]
  ];

let finalizo = false; /* Mantiene el ciclo hasta que resuelva el Sudoku */
let ciclos = 0; /* Lleva el número de iteraciones */

let solucion = []; /* Tablero en el que se trabaja */
for (let fila = 0; fila < 9; fila++) {
  solucion[fila] = [];
  for (let columna = 0; columna < 9; columna++)
    solucion[fila][columna] = 0;
}
let DESTRUYE = 700; /* Cada cuantos ciclos borra números para destrabar */

/* Ciclo que llenará el sudoku completamente */
do {
  for (let fila = 0; fila < 9; fila++) { /* Se copia el sudoku sobre el tablero a evaluar */
    for (let columna = 0; columna < 9; columna++)
      if (sudoku[fila][columna] !== 0) solucion[fila][columna] = sudoku[fila][columna];
  }

  let numValido = true;
  do { /* Busca un número al azar para colocar en alguna celda */
    let posX = Math.floor(Math.random() * 9); /* Una posición X de 0 a 8 */
    let posY = Math.floor(Math.random() * 9); /* Una posición Y de 0 a 8 */
    let numero = Math.floor(Math.random() * 9) + 1; /* Un número al azar de 1 a 9 */
    numValido = true; /* Chequea si el número no se repite ni vertical ni horizontalmente */
    for (let cont = 0; cont < 9; cont++) if (solucion[cont][posY] === numero || solucion[posX][cont] ===
numero) numValido = false;
    if (numValido) solucion[posX][posY] = numero; /* Si el número no se repite entonces lo coloca en el
tablero */
  } while (!numValido);

  /* Chequea que NO se viole la regla de que cada uno de los 9 cuadros internos no repita número */
  for (let cuadroX = 0; cuadroX <= 6; cuadroX += 3)
    for (let cuadroY = 0; cuadroY <= 6; cuadroY += 3) {
      let numRepite = 0;
      for (let valor = 1; valor <= 9; valor++) {
        numRepite = 0;
        for (let posX = 0; posX < 3; posX++)
          for (let posY = 0; posY < 3; posY++) {
            if (solucion[cuadroX + posX][cuadroY + posY] === valor) numRepite++;
            if (numRepite > 1) break;
          }

        if (numRepite > 1) /* Si detecta repetición, entonces borra todos los números repetidos */
          for (let posX = 0; posX < 3; posX++)
            for (let posY = 0; posY < 3; posY++)
              if (solucion[cuadroX + posX][cuadroY + posY] === valor) solucion[cuadroX +
posX][cuadroY + posY] = 0;
      }
    }

  finalizo = true; /* Chequea si se completó el sudoku completamente */
  for (let posX = 0; posX < 9; posX++)
    for (let posY = 0; posY < 9; posY++)
      if (solucion[posX][posY] === 0) finalizo = false;

  ciclos++; /* Cada ciertos ciclos para destrabar, borra la tercera parte de lo completado */
  if (ciclos % DESTRUYE === 0)
```

```

        for (let posX = 0; posX < 9; posX++)
            for (let posY = 0; posY < 9; posY++)
                if (Math.floor(Math.random() * 3) === 0) solucion[posX][posY] = 0;
    } while (!finalizo);

document.getElementById("salida").innerHTML = "<p style='font-family:courier new;'>";
document.getElementById("salida").innerHTML += "Ciclos totales: " + ciclos + "</p>";
for (let posX = 0; posX < 9; posX++) {
    document.getElementById("salida").innerHTML += "<p style='font-family:courier new;'>";
    for (let posY = 0; posY < 9; posY++)
        document.getElementById("salida").innerHTML += solucion[posX][posY] + " ";
    document.getElementById("salida").innerHTML += "</p>";
}
</script></body></html>

```



Ilustración 24: Sudoku resuelto



```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
/* Hacer un algoritmo que ponga los 5 barcos
   al azar en el tablero del juego de batalla naval */

//Crear el tablero del juego
let tableroJuego = inicializaTablero(10, 10);

//Poner cada barco: arreglo, tipo barco, número de huecos
poneBarco(tableroJuego, 'P', 5);
poneBarco(tableroJuego, 'G', 4);
poneBarco(tableroJuego, 'C', 3);
poneBarco(tableroJuego, 'S', 3);
poneBarco(tableroJuego, 'D', 2);
imprime(tableroJuego);

//Genera el tablero
function inicializaTablero(filas, columnas) {
  let tablero = [];
  for (let fila = 0; fila < filas; fila++) {
    tablero[fila] = [];
    for (let columna = 0; columna < columnas; columna++)
      tablero[fila][columna] = '.';
  }
  return tablero;
}

function imprime(tablero) { //Imprime el tablero
  for (let fila = 0; fila < tablero.length; fila++) {
    document.getElementById("salida").innerHTML += "<p style='font-family:courier new;'>";
    for (let col = 0; col < tablero[0].length; col++)
      document.getElementById("salida").innerHTML += tablero[fila][col] + " ";
    document.getElementById("salida").innerHTML += "</p>";
  }
  document.getElementById("salida").innerHTML += "</p>";
}

//Pone el barco en una posición al azar
function poneBarco(tablero, simbolo, huecos) {
  let orienta, fil, col, posValida;
  do {
    /* ¿Vertical u horizontal?
       0 a 0.4999 es vertical,
       0.5 a 1 es horizontal */
    orienta = Math.random();

    fil = Math.floor(Math.random() * tablero.length);
    col = Math.floor(Math.random() * tablero[0].length);
    posValida = true; //La posición del barco es válida

    //Chequea si no se sale del tablero
    if (fil + huecos >= tablero.length && orienta < 0.5)
      posValida = false; //La posición del barco es inválida
    else if (col + huecos >= tablero[0].length && orienta >= 0.5)
      posValida = false; //La posición del barco es inválida
    else { //Chequea si ya ha sido ocupada esa parte
      if (orienta < 0.5)
        for (let cont = 0; cont < huecos; cont++) {
          if (tablero[fil + cont][col] !== '.')
            posValida = false;
        }
      else
        for (let cont = 0; cont < huecos; cont++) {
          if (tablero[fil][col + cont] !== '.')
            posValida = false;
        }
    }
  } while (posValida === false);

  if (orienta < 0.5)
    for (let cont = 0; cont < huecos; cont++)
```

```
        tablero[fil + cont][col] = simbolo;
    else
        for (let cont = 0; cont < huecos; cont++)
            tablero[fil][col + cont] = simbolo;
    }
</script></body></html>
```



## Capítulo 8. Programación orientada a objetos

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Definir una clase
class Gato {
  //Constructor de la clase
  constructor(nombre, sexo, fechanace, raza) {
    this.nombre = nombre; //define el atributo y le da un valor
    this.sexo = sexo;
    this.fechanace = fechanace;
    this.raza = raza;
  }
}

//Instanciar un objeto de esa clase
let prueba = new Gato("Sally", "F", "10 de junio de 2010", "Criollo");

document.getElementById("salida").innerHTML = prueba.nombre + " , " + prueba.sexo + " , ";
document.getElementById("salida").innerHTML += prueba.fechanace + " , " + prueba.raza;
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Definir una clase
class Geometria {
    //Define métodos
    AreaCuadrado(Lado) {
        return Lado * Lado;
    }

    //Define métodos
    AreaTriangulo(Base, Altura) {
        return Base * Altura / 2;
    }
}

//Instanciar un objeto de esa clase
let probar = new Geometria();

document.getElementById("salida").innerHTML = "Area del cuadrado es: " + probar.AreaCuadrado(9);
document.getElementById("salida").innerHTML += "<br>Area del triangulo es: " + probar.AreaTriangulo(3, 4);
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Definir una clase
class Geometria {
    //Método que define el atributo Lado y le da un valor
    ValorLado(Lado) {
        this.Lado = Lado;
    }

    //Método que retorna el valor del área
    AreaCuadrado() {
        return this.Lado * this.Lado;
    }
}

//Instanciar un objeto de esa clase
let probar = new Geometria();
probar.ValorLado(11);
document.getElementById("salida").innerHTML = "Area del Cuadrado es: " + probar.AreaCuadrado();
</script></body></html>
```

## Atributos y métodos privados

Se hace uso del carácter # para definir los atributos y métodos privados

Cap08/004.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Definir una clase
class UnaClase {
    //La convención es poner un # al principio
    //del nombre del método para indicar que es
    //un método privado
    #UnMetodo() {
        document.getElementById("salida").innerHTML += "Método privado<br>";
    }

    #OtroMetodo() {
        document.getElementById("salida").innerHTML += "Otro método privado<br>";
    }
}

//Instanciar un objeto de esa clase
let probar = new UnaClase();
probar.#UnMetodo();
probar.#OtroMetodo();
</script></body></html>
```

Si intenta acceder a un método privado desde la instancia, se genera un mensaje de error.

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
class MiClase {
    #texto; //Declara el atributo privado

    constructor(texto) {
        this.#texto = texto; //Define el atributo "privado"
    }

    //Getter
    get texto() {
        return this.#texto;
    }

    //Setter
    set texto(valor) {
        this.#texto = valor;
    }
}

probar = new MiClase('Grisú y Suini');
document.getElementById("salida").innerHTML = probar.texto + "<br>";
probar.texto = "Sally";
document.getElementById("salida").innerHTML = probar.texto + "<br>";
</script></body></html>
```

Podemos probar que los métodos get y set son llamados

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
class MiClase {
    #texto; //Declara el atributo privado

    constructor(texto) {
        this.#texto = texto; //Define el atributo "privado"
    }

    //Getter
    get texto() {
        document.getElementById("salida").innerHTML += "Retorna valor <br>";
        return this.#texto;
    }

    //Setter
    set texto(valor) {
        document.getElementById("salida").innerHTML += "Cambia valor <br>";
        this.#texto = valor;
    }
}

probar = new MiClase('Grisú y Suini');
document.getElementById("salida").innerHTML += probar.texto + "<br>";
probar.texto = "Sally";
document.getElementById("salida").innerHTML += probar.texto + "<br>";
</script></body></html>
```



```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Clase Madre
class Madre {
}

//Clase Hija (hereda de la madre)
class Hija extends Madre {
}

prueba = new Hija();

document.getElementById("salida").innerHTML = "<br><br>prueba es una instancia de Hija: ";
document.getElementById("salida").innerHTML += prueba instanceof Hija; //Resultado true

document.getElementById("salida").innerHTML += "<br><br>prueba es una instancia de Madre: ";
document.getElementById("salida").innerHTML += prueba instanceof Madre; //Resultado true

test = new Madre();

document.getElementById("salida").innerHTML += "<br><br>test es una instancia de Hija: ";
document.getElementById("salida").innerHTML += test instanceof Hija; //Resultado false

document.getElementById("salida").innerHTML += "<br><br>test es una instancia de Madre: ";
document.getElementById("salida").innerHTML += test instanceof Madre; //Resultado true
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Clase Madre
class Madre {
    constructor() {
        document.getElementById("salida").innerHTML += "<br>Constructor clase madre";
    }
}

//Clase Hija (hereda de la madre)
class Hija extends Madre {
    constructor() {
        super(); //Llama al constructor de la clase madre
        document.getElementById("salida").innerHTML += "<br>Constructor clase hija";
    }
}

prueba = new Hija(); //Imprime "Constructor clase madre" y "Constructor clase hija"
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Clase abuela
class Abuela {
    constructor() {
        document.getElementById("salida").innerHTML += "<br>Constructor clase abuela";
    }
}

//Clase Madre
class Madre extends Abuela {
    constructor() {
        super();
        document.getElementById("salida").innerHTML += "<br>Constructor clase madre";
    }
}

//Clase Hija (hereda de la madre)
class Hija extends Madre {
    constructor() {
        super();
        document.getElementById("salida").innerHTML += "<br>Constructor clase hija";
    }
}

prueba = new Hija();
/* Imprime:
    Constructor clase abuela
    Constructor clase madre
    Constructor clase hija
*/
</script></body></html>
```

¡OJO! Debe hacer uso de la instrucción super() en el constructor de la clase hija (la que hereda) porque de lo contrario obtendrá un error y el script se detendrá:

ReferenceError: must call super constructor before using |this| in Hija class constructor



## Sobrecarga de métodos

**No hay sobrecarga de métodos** como tal en JavaScript. El siguiente script pareciera que se implementa sobrecarga, pero NO funciona. Lo peor, es que no se genera mensaje de error en el navegador.

Cap08/010.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
/* La sobrecarga de métodos NO existe. Este script NO va a funcionar como se espera.
   Y no va a haber mensaje de error por parte del navegador */
class MiClase {
  MetodoA() {
    document.getElementById("salida").innerHTML += "<br>Llama al método A con cero parámetros";
  }

  MetodoA(param1, param2) {
    document.getElementById("salida").innerHTML += "<br>Llama al método A con dos parámetros";
  }

  MetodoA(param1) {
    document.getElementById("salida").innerHTML += "<br>Llama al método A con un parámetro";
  }

  MetodoA(param1, param2, param3) {
    document.getElementById("salida").innerHTML += "<br>Llama al método A con tres parámetros: " + param1
+ ", " + param2 + ", " + param3;
  }
}

prueba = new MiClase();
prueba.MetodoA();
prueba.MetodoA(2, 7);
prueba.MetodoA(8);
prueba.MetodoA(5, 9, 3);

/* El programa ejecutará así:
Llama al método A con tres parámetros: undefined, undefined, undefined
Llama al método A con tres parámetros: 2, 7, undefined
Llama al método A con tres parámetros: 8, undefined, undefined
Llama al método A con tres parámetros: 5, 9, 3 */
</script></body></html>
```

## Capítulo 9. Estructuras de Datos

## Árbol binario

Con tres nodos se puede crear un árbol binario: raíz, rama izquierda, rama derecha

Cap09/001.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Definir una clase
class Nodo {
    //Constructor de la clase
    constructor(valor, izq, der) {
        this.valor = valor;
        this.izquierdo = izq;
        this.derecho = der;
    }
}

let NodoRaiz = new Nodo(1, null, null); //Nodo raíz
let NodoIzq = new Nodo(2, null, null); //Rama izquierda
let NodoDer = new Nodo(3, null, null); //Rama derecha

//El nodo raíz conecta a la izquierda y derecha
NodoRaiz.izquierdo = NodoIzq;
NodoRaiz.derecho = NodoDer;

//Imprime los valores del árbol
document.getElementById("salida").innerHTML = NodoRaiz.valor + "<br>";
document.getElementById("salida").innerHTML = NodoRaiz.izquierdo.valor + "<br>";
document.getElementById("salida").innerHTML = NodoRaiz.derecho.valor + "<br>";
</script></body></html>
```

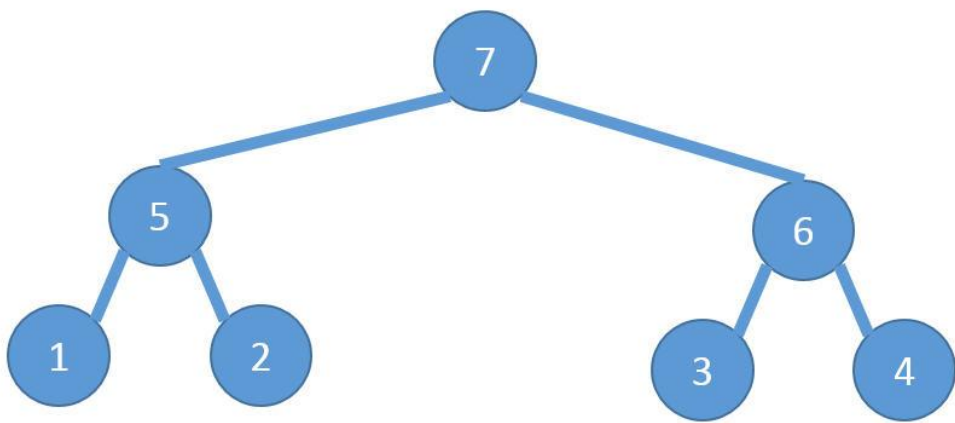


Ilustración 25: Árbol binario

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Definir una clase
class Nodo {
    //Constructor de la clase
    constructor(valor, izq, der) {
        this.valor = valor;
        this.izquierdo = izq;
        this.derecho = der;
    }
}

//Se genera el árbol binario
let Nodo1 = new Nodo(1, null, null);
let Nodo2 = new Nodo(2, null, null);
let Nodo3 = new Nodo(3, null, null);
let Nodo4 = new Nodo(4, null, null);
let Nodo5 = new Nodo(5, Nodo1, Nodo2);
let Nodo6 = new Nodo(6, Nodo3, Nodo4);
let Nodo7 = new Nodo(7, Nodo5, Nodo6);

//Lo recorre en preorden
preorden(Nodo7); //7,5,1,2,6,3,4

function preorden(nodo) {
    if (nodo === null) return;
    document.getElementById("salida").innerHTML += " , " + nodo.valor;
    preorden(nodo.izquierdo);
    preorden(nodo.derecho);
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Definir una clase
class Nodo {
    //Constructor de la clase
    constructor(valor, izq, der) {
        this.valor = valor;
        this.izquierdo = izq;
        this.derecho = der;
    }
}

//Se genera el árbol binario
let Nodo1 = new Nodo(1, null, null);
let Nodo2 = new Nodo(2, null, null);
let Nodo3 = new Nodo(3, null, null);
let Nodo4 = new Nodo(4, null, null);
let Nodo5 = new Nodo(5, Nodo1, Nodo2);
let Nodo6 = new Nodo(6, Nodo3, Nodo4);
let Nodo7 = new Nodo(7, Nodo5, Nodo6);

inorden(Nodo7); //1,5,2,7,3,6,4

function inorden(nodo) {
    if (nodo === null) return;
    inorden(nodo.izquierdo);
    document.getElementById("salida").innerHTML += " , " + nodo.valor;
    inorden(nodo.derecho);
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Definir una clase
class Nodo {
    //Constructor de la clase
    constructor(valor, izq, der) {
        this.valor = valor;
        this.izquierdo = izq;
        this.derecho = der;
    }
}

//Se genera el árbol binario
let Nodo1 = new Nodo(1, null, null);
let Nodo2 = new Nodo(2, null, null);
let Nodo3 = new Nodo(3, null, null);
let Nodo4 = new Nodo(4, null, null);
let Nodo5 = new Nodo(5, Nodo1, Nodo2);
let Nodo6 = new Nodo(6, Nodo3, Nodo4);
let Nodo7 = new Nodo(7, Nodo5, Nodo6);

postorden(Nodo7); //1,2,5,3,4,6,7

function postorden(nodo) {
    if (nodo === null) return;
    postorden(nodo.izquierdo);
    postorden(nodo.derecho);
    document.getElementById("salida").innerHTML += " , " + nodo.valor;
}
</script></body></html>
```



```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Ordenación usando árbol binario
class Nodo {
    constructor(valor, izq, der) {
        this.valor = valor;
        this.izquierdo = izq;
        this.derecho = der;
    }
}

//Un arreglo con valores no ordenados
let arreglo = [15, 21, 17, 32, 48, 13, 29, 44, 19, 16, 3, -1, 9, 0];

//Genera el árbol binario
let NodoRaiz = new Nodo(arreglo[0], null, null);
for (let cont = 1; cont < arreglo.length; cont++) {
    let pasea = NodoRaiz;
    while (pasea !== null) {
        if (arreglo[cont] < pasea.valor) { //Pone valor en la rama izquierda
            if (pasea.izquierdo !== null)
                pasea = pasea.izquierdo;
            else { //Crea nodo con el nuevo valor
                pasea.izquierdo = new Nodo(arreglo[cont], null, null);
                break;
            }
        } else { //Pone valor en la rama derecha
            if (pasea.derecho !== null)
                pasea = pasea.derecho;
            else { //Crea nodo con el nuevo valor
                pasea.derecho = new Nodo(arreglo[cont], null, null);
                break;
            }
        }
    } //Cierra el while que navega por el árbol binario
} //Cierra el for que pasea por todo el arreglo unidimensional no ordenado

//Recorre en in-orden el árbol binario y los datos están ordenados
inorden(NodoRaiz);

function inorden(nodo) {
    if (nodo === null) return;
    inorden(nodo.izquierdo);
    document.getElementById("salida").innerHTML += ", " + nodo.valor;
    inorden(nodo.derecho);
}
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Primer ejemplo de uso de JSON
let dato = {"id": 13, "nombre": "Rafael Alberto Moreno", "tiposangre": "O+", "altura": 1.80}
document.getElementById("salida").innerHTML = dato.nombre + "<br>" + dato.tiposangre + "<br>" + dato.id +
"<br>";
document.getElementById("salida").innerHTML += dato.altura + "<br>";

/* Imprime
Rafael Alberto Moreno
O+
13
1.8
*/
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Segundo ejemplo de uso de JSON
let dato = {
  "mensaje": function () {
    document.getElementById("salida").innerHTML = "Hola Mundo";
  }
}
dato.mensaje();

/* Imprime:
  Hola Mundo
*/
</script></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Tercer ejemplo de uso de JSON
let dato = {
  "nombre": "Rafael", "mensaje": function () {
    document.getElementById("salida").innerHTML = dato.nombre;
  }
}
dato.mensaje();

/* Imprime:
  Rafael
*/
</script></body></html>
```



```

<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Cuarto ejemplo de uso de JSON
let animal =
{
    "Aves": [
        {"nombre": "Cuervo", "esperanzavida": 70},
        {"nombre": "Loro Gris", "esperanzavida": 80},
        {"nombre": "Zopilote de Turquía", "esperanzavida": 119}]
    };
document.getElementById("salida").innerHTML = animal.Aves[2].esperanzavida;

/* Imprime:
    119
*/
</script></body></html>

```

```

<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Quinto ejemplo de uso de JSON
let animal =
{
    "Aves": [
        {"nombre": "Cuervo", "esperanzavida": 70},
        {"nombre": "Loro Gris", "esperanzavida": 80},
        {"nombre": "Zopilote de Turquía", "esperanzavida": 118}],

    "Reptiles": [
        {"nombre": "Tortuga radiada", "esperanzavida": 100},
        {"nombre": "Cocodrilo", "esperanzavida": 50},
        {"nombre": "Dragón de Komodo", "esperanzavida": 30}]
    };
document.getElementById("salida").innerHTML = animal.Aves[0].nombre + "<br>";
document.getElementById("salida").innerHTML += animal.Reptiles[1].nombre;

/* Imprime:
    Cuervo
    Cocodrilo
*/
</script></body></html>

```

```

<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Sexto ejemplo de uso de JSON
let animal =
{
    "Aves": [
        {"nombre": "Cuervo", "esperanzavida": 70},
        {"nombre": "Loro Gris", "esperanzavida": 80},
        {"nombre": "Zopilote de Turquía", "esperanzavida": 118}],

    "Reptiles": [
        {"nombre": "Tortuga radiada", "esperanzavida": 100},
        {"nombre": "Cocodrilo", "esperanzavida": 50},
        {"nombre": "Dragón de Komodo", "esperanzavida": 30}]
    };

for (let cont = 0; cont < animal.Aves.length; cont++) {
    document.getElementById("salida").innerHTML = animal.Aves[cont].nombre + " vive hasta ";
    document.getElementById("salida").innerHTML += animal.Aves[cont].esperanzavida + " años <br>";
}

/* Imprime:
Cuervo vive hasta 70 años
Loro Gris vive hasta 80 años
Zopilote de Turquía vive hasta 118 años
*/
</script></body></html>

```

```

<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Séptimo ejemplo de uso de JSON
let clima = [
    {"temperaturas": [23, 27, 29, 21]},
    {"temperaturas": [14, 18, 15, 11]},
    {"temperaturas": [33, 32, 35, 30]}};

//Va de grupo en grupo de temperaturas
for (let cont = 0; cont < clima.length; cont++) {

    //va de ítem en ítem de cada grupo de temperaturas
    for (let pos = 0; pos < clima[cont].temperaturas.length; pos++)
        document.getElementById("salida").innerHTML += clima[cont].temperaturas[pos] + ", ";
    document.getElementById("salida").innerHTML += "<br>";
}

/* Imprime:
23, 27, 29, 21,
14, 18, 15, 11,
33, 32, 35, 30,
*/
</script></body></html>

```

```

<!DOCTYPE HTML><html lang="es"><head></head><body>
<p id="salida"></p>
<script>
//Octavo ejemplo de uso de JSON
let Datos =
{
    "cadenaA": "esta es una cadena de ejemplo",
    "valorA": 17,
    "valorB": true,
    "valorC": [29, 31, 43, 47, 51],
    "otros": {
        "cadenaB": "Segundo ejemplo",
        "valores": ["abcdefg", "xyzwqr", "kqxrsft"]
    }
};

document.getElementById("salida").innerHTML = Datos.cadenaA + "<br>" + Datos.valorA + "<br>";
document.getElementById("salida").innerHTML += Datos.valorB + "<br>" + Datos.valorC + "<br>";
document.getElementById("salida").innerHTML += Datos.otros.cadenaB + "<br>";
document.getElementById("salida").innerHTML += Datos.otros.valores + "<br>";
document.getElementById("salida").innerHTML += Datos.otros.valores[1] + "<br>";

/* Imprime:
esta es una cadena de ejemplo
17
true
29,31,43,47,51
Segundo ejemplo
abcdefg,xyzwqr,kqxrsft
xyzwqr
*/
</script></body></html>

```

## Capítulo 10. Control sobre la página HTML

JavaScript permite un control completo sobre los objetos que hay en la página HTML.

Al dar clic en el botón muestra una ventana emergente de alerta.

Cap10/001.html

```
<!DOCTYPE html><html lang="es"><head></head><body>
<button id="boton">Tocar</button>
<script>
document.addEventListener("DOMContentLoaded", () => {
    const boton = document.getElementById("boton");
    boton.addEventListener("click", () => {
        alert("Hola mundo");
    });
});
</script></body></html>
```

El siguiente código hace lo mismo haciendo uso de funciones

Cap10/002.html

```
<!DOCTYPE html><html lang="es"><head></head><body>
<button id="boton">Tocar2</button><script>

function aviso() {
    alert("Hola mundo");
}

document.addEventListener("DOMContentLoaded", () => {
    const boton = document.getElementById("boton");
    boton.addEventListener("click", aviso);
});

</script></body></html>
```

Funciona también para enlaces

Cap10/003.html

```
<!DOCTYPE html><html lang="es"><head></head><body>
  <a href="#" id="enlace">De clic aquí</a>
  <script>
    document.addEventListener("DOMContentLoaded", () => {
      const enlace = document.getElementById("enlace");
      enlace.addEventListener("click", (event) => {
        event.preventDefault(); // evita que el enlace recargue la página
        alert("Hola mundo");
      });
    });
  </script>
</body></html>
```

Y se puede separar el código HTML de JavaScript. Primero el código JavaScript en su propio archivo (para este ejemplo es saludo.js)

saludo.js

```
/* Funciones JavaScript en un archivo externo con extensión .js */
function imprimir() {
    alert("Hola mundo");
}

document.addEventListener("DOMContentLoaded", () => {
    const enlace = document.getElementById("enlace");
    enlace.addEventListener("click", (event) => {
        event.preventDefault(); // evita que el enlace intente navegar
        imprimir();
    });
});
```

Y el código HTML que lo llama

Cap10/004.html

```
<!DOCTYPE html><html lang="es"><head>
  <script src="saludo.js" defer></script>
</head>
<body>
  <a href="#" id="enlace">De clic aquí</a>
</body></html>
```



# Captura el evento de dar clic en un <div>

Puede activarse cuando el usuario de clic en el interior de <div>

Cap10/005.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <style>
    .caja {
      width: 230px;
      height: 230px;
      margin: 20px;
      background-color: lightgreen;
      display: inline-block;
    }
  </style>
  <script>
    function clicCaja() {
      alert("Dio clic dentro de un <div>");
    }

    document.addEventListener("DOMContentLoaded", () => {
      const cajas = document.querySelectorAll(".caja");
      cajas.forEach(caja => {
        caja.addEventListener("click", clicCaja);
      });
    });
  </script>
</head>
<body>
  <div id="caja1" class="caja"></div>
  <div id="caja2" class="caja"></div>
  <div id="caja3" class="caja"></div>
  <div id="caja4" class="caja"></div>
</body>
</html>
```

Puede saberse en que <div> el usuario da clic. Cada <div> debe tener su propio id

Cap10/006.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ejemplo Moderno</title>
  <style>
    .caja {
      width: 70px;
      height: 70px;
      margin: 20px;
      background-color: lightgreen;
      display: inline-block;
    }
  </style>
  <script>
    function divclic(event) {
      alert("<div> tocado es: " + event.target.id);
    }

    document.addEventListener("DOMContentLoaded", () => {
      const cajas = document.querySelectorAll(".caja");
      cajas.forEach(caja => {
        caja.addEventListener("click", divclic);
      });
    });
  </script>
</head>
<body>
  <div id="caja1" class="caja"></div>
  <div id="caja2" class="caja"></div>
  <div id="caja3" class="caja"></div>
```

```
<div id="caja4" class="caja"></div>  
</body>  
</html>
```



```
<!DOCTYPE HTML><html lang="es"><head><meta charset="UTF-8">
<style>
  div {
    width: 230px;
    height: 230px;
    margin: 20px;
    background-color: lightgreen;
    cursor: pointer;
  }
</style>
</head>
<body>

<div id="caja1"></div>

<script>
  function divDobleClic() {
    alert("Diste doble clic");
  }

  // Asignar evento
  document.getElementById("caja1").addEventListener("dblclick", divDobleClic);
</script>

</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head>
<style>
  div {
    width: 230px;
    height: 230px;
    margin: 20px;
    background-color: lightgreen;
  }
</style>
</head>
<body>
<script>
function divmouseencima() {
  alert("El ratón pasó por encima");
}
</script>
<div id="caja1" onmouseover="divmouseencima()"></div>
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head>
<style>
  div {
    width: 230px;
    height: 230px;
    margin: 20px;
    background-color: lightgreen;
  }
</style>
</head>
<body>
<script>
function divmousesale() {
  alert("El ratón ha salido del <div>");
}
</script>
</body>
<div id="caja" onmouseout="divmousesale()"></div>
</html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head>
<body onLoad="cargandopagina()">
<script>
function cargandopagina() {
    alert("Página se carga");
}
</script>
<p>Esto es una prueba</p>
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head>
<style>
    div {
        width: 230px;
        height: 230px;
        margin: 20px;
        background-color: lightgreen;
    }
</style>
</head>
<body>
<script>
function divmousepresiona() {
    alert("clic de ratón");
}
</script>
<div id="caja1" onmousedown="divmousepresiona()"></div>
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head>
<style>
  div {
    width: 230px;
    height: 230px;
    margin: 20px;
    background-color: lightgreen;
  }
</style>
</head>
<body>
<script>
function divmousesuelta() {
  alert("Soltó tecla del ratón");
}
</script>
<div id="caja1" onmouseup="divmousesuelta()"></div>
</body></html>
```

# Captura el evento de mover el cursor del ratón

Cambiar el color de fondo de la página mientras movemos el cursor del ratón por el <div>

Cap10/013.html

```
<!DOCTYPE HTML><html lang="es"><head>
<style>
    div {
        width: 130px;
        height: 130px;
        margin: 20px;
        background-color: lightgreen;
    }
</style>
</head>
<body>
<script>
function divmousemueve() {
    document.bgColor = '#' + Math.floor(Math.random() * 16777215).toString(16);
}
</script>
<div id="caja1" onmousemove="divmousemueve()"></div>
</body></html>
```



```
<!DOCTYPE HTML><html lang="es"><head></head>
<body onresize="cambialongitud()" ">
<script>
function cambialongitud() {
    alert("¡Crecer o decrecer!");
}
</script>
<p>Cambie el tamaño de la ventana del navegador</p>
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
function formularioenvia() {
    alert("activo envío");
}
</script>
<form id="formulario" method="post" onsubmit="formularioenvia()">
<input type="submit" id="boton" value="envía"/>
</form>
</body></html>
```



```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
function formularioenvia() {
    alert("activo envío");
}

function entrafocotexto() {
    alert("está en la caja de texto");
    document.getElementById("otro").focus();
}
</script>
<form id="formulario" method="post" onsubmit="formularioenvia()">
<input type="text" id="texto" onfocus="entrafocotexto()" /><br>
<input type="text" id="otro" /><br>
<input type="submit" id="boton" value="envía"/>
</form>
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
function formularioenvia() {
    alert("activo envío");
}

function salefocotexto() {
    alert("salió de la caja de texto");
}
</script>
<form id="formulario" method="post" onsubmit="formularioenvia()">
<input type="text" id="texto" onblur="salefocotexto()"/>
<input type="text" id="segundotexto"/>
<input type="submit" id="boton" value="envía"/>
</form>
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
function formularioenvia() {
    alert("activo envío");
}

function presionatecla() {
    alert("Ha presionado una tecla");
}
</script>
<form id="formulario" method="post" onsubmit="formularioenvia()">
<input type="text" id="texto" onkeydown="presionatecla()" />
<input type="submit" id="boton" value="envía" />
</form>
</body></html>
```

Un efecto similar sucede con “onkeypress”

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
function formularioenvia() {
    alert("activo envío");
}

function presionatecla() {
    alert("Ha presionado una tecla");
}
</script>
<form id="formulario" method="post" onsubmit="formularioenvia()">
<input type="text" id="texto" onkeypress="presionatecla()" />
<input type="submit" id="boton" value="envía" />
</form>
</body></html>
```

El evento onkeydown sucede primero, le sigue el evento onkeypress (y este sucede cuando el carácter ha entrado). Podemos probarlo así:

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
function formularioenvia() {
    alert("activo envío");
}

function abajo() {
    alert("abajo");
}

function presiona() {
    alert("presiona");
}
</script>
<form id="formulario" method="post" onsubmit="formularioenvia()">
<input type="text" id="texto" onkeypress="presiona()" onkeydown="abajo()" />
<input type="submit" id="boton" value="envía" />
</form>
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
function formularioenvia() {
    alert("activo envío");
}

function sueltatecla() {
    alert("Ha soltado una tecla");
}
</script>
<form id="formulario" method="post" onsubmit="formularioenvia()">
<input type="text" id="texto" onkeyup="sue ltatecla()"/>
<input type="submit" id="boton" value="envía"/>
</form>
</body></html>
```

# Captura el evento de cambio del texto en una caja de texto

Si al saltar a otra caja de texto, la anterior ha cambiado su contenido, se dispara la alerta

Cap10/022.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
function formularioenvia() {
    alert("activo envío");
}

function cambiatexto() {
    alert("Ha cambiado el texto");
}
</script>
<form id="formulario" method="post" onsubmit="formularioenvia()">
<p><input type="text" id="texto" onchange="cambiatexto()" /></p>
<p><input type="text" id="otro" /></p>
<p><input type="submit" id="boton" value="envía" /></p>
</form>
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
function formularioenvia() {
    alert("envío");
}

//Muestra el código de la tecla presionada
function presionatecla(event) {
    alert(event.keyCode);
}
</script>
<form id="formulario" method="post" onsubmit="formularioenvia()">
<input type="text" id="texto" onkeyup="presionatecla(event)"/>
<input type="submit" id="boton" value="envía"/>
</form>
</body></html>
```



```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
function navegar() {
    return confirm("¿Irás a esa página?");
}
</script>
<a href="http://darwin.50webs.com" onclick="return navegar()">Enlace</a>
</body></html>
```

## Capítulo 11. DOM (Document Object Model)



```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Al presionar el botón se llama a esta función:

//Esta función toma el hipervínculo y lo cambia
function enlazarmipagina() {
    //Toma el control del objeto hipervínculo
    let enlace = document.getElementById("ejemploenlace");

    //Cambia hacia donde enlaza
    enlace.href = "http://darwin.50webs.com";
}
</script>
<a id="ejemploenlace">Hipervínculo</a>
<input type="button" onclick="enlazarmipagina()" value="Mi página"/>
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Cambia el fondo de un <div>
function cambiarfondo() {
    let caja = document.getElementById("cajaejemplo");
    caja.style.backgroundColor = "green";
    caja.style.height = "100px";
    caja.style.width = "50%";
    caja.style.border = "1px solid black";
}
</script>
<div id="cajaejemplo">
<input type="button" onclick="cambiarfondo()" value="Cambiar fondo"/>
</div>
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head>
<style>
    #caja {
        width: 200px;
        height: 200px;
        background-color: lightgreen;
    }
</style>
</head>
<body>
<script>
//Oculta el <div>
function apagar() {
    let caja = document.getElementById("caja");
    caja.style.visibility = "hidden";
}

//Muestra el <div>
function encender() {
    let caja = document.getElementById("caja");
    caja.style.visibility = "visible";
}
</script>
<div id="caja"></div>
<input type="button" value="Apagar" onclick="apagar() "/>
<input type="button" value="Encender" onclick="encender() "/>
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head>
<style>
    #caja {
        width: 200px;
        height: 200px;
        background-color: lightgreen;
    }
</style>
</head>
<body>
<script>
//Borra contenido del <div>
function borrar() {
    let caja = document.getElementById("caja");
    caja.innerHTML = "";
}

//Pone contenido en el <div>
function texto() {
    let caja = document.getElementById("caja");
    caja.innerHTML = "<b>dentro del DIV</b>";
}
</script>
<div id="caja">Hola Mundo</div>
<input type="button" value="Borrar" onclick="borrar() "/>
<input type="button" value="Mostrar Texto" onclick="texto() "/>
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head>
<style>
  input {
    background-color: lightgreen;
    border: 1px solid gray;
  }

  input.enfoca {
    background-color: lightblue;
    border: 3px solid green;
  }
</style>
</head>
<body>
<input type="text" id="DatoA" onfocus="this.className='enfoca'" onblur="this.className=''" />
<input type="text" id="DatoB" onfocus="this.className='enfoca'" onblur="this.className=''" />
</body></html>
```

Funciona también así:

```
<!DOCTYPE HTML><html lang="es"><head>
<style>
  input {
    background-color: lightgreen;
    border: 1px solid gray;
  }

  input.enfoco {
    background-color: lightblue;
    border: 3px solid green;
  }
</style>
</head>
<body>
<script>
function consigüefoco(entrada) {
  entrada.className = 'enfoco';
}

function fueradelfoco(entrada) {
  entrada.className = '';
}
</script>
<input type="text" id="DatoA" onfocus="consigüefoco(this)" onblur="fueradelfoco(this)" />
<input type="text" id="DatoB" onfocus="consigüefoco(this)" onblur="fueradelfoco(this)" />
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head>
<style>
input.error{ border: 1px solid red; background-color: yellow; }
</style>
</head>
<body>
<script>
//Cuando se salta a otra caja de texto, se valida si
//el dato es un número válido

//Valida si lo que ingresa es un número
function validanumero(entrada) {
    if (entrada.value !== "") {
        if (isNaN(entrada.value))
            entrada.className = 'error';
        else
            entrada.className = '';
    }
}
</script>
<label>Valor: <input type="text" id="Numero" onblur="validanumero(this)"/></label>
<label> Dato: <input type="text" id="Dato"/></label>
</body></html>
```



```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Hace un conteo de objetos con la etiqueta <p> en la página
function ChequeaObjetos() {
    let obtenerobjetos = document.getElementsByTagName("p");
    alert("Total objetos con <p> : " + obtenerobjetos.length);
}
</script>
<p>Primer párrafo</p>
<p>Segundo párrafo</p>
<p>Tercer párrafo</p>
<input type="button" onclick="ChequeaObjetos()" value="Objetos con <p>">
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Obtiene la lista de objetos con <p>, trae el segundo
//y muestra el código HTML interno
function ChequeaObjetos() {
    let obtenerobjetos = document.getElementsByTagName("p");
    alert("Captura objeto <p> : " + obtenerobjetos[1].innerHTML);
}
</script>
<p>Primer párrafo</p>
<p><b>Segundo párrafo</b></p>
<p><b>Tercer párrafo</b></p>
<input type="button" onclick="ChequeaObjetos()" value="Objetos con <p>">
</body></html>
```



```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Obtiene la lista de objetos tipo entrada, trae el primero
//y muestra que valor tiene
function ChequeaObjetos() {
    let obtenerobjetos = document.getElementsByTagName("input");
    alert("Valor escrito es : " + obtenerobjetos[0].value);
}
</script>
<input type="text"><br>
<input type="text"><br>
<input type="button" onclick="ChequeaObjetos()" value="Leer valor">
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Obtiene la lista de objetos tipo <p>
//y muestra el valor que tiene cada uno
function ChequeaObjetos() {
    let arreglo = document.getElementsByTagName("p");
    for (let cont = 0; cont < arreglo.length; cont++)
        alert("Contenido : " + arreglo[cont].innerHTML);
}
</script>
<p>Primer párrafo</p>
<p><b>Segundo párrafo</b></p>
<p>Tercer párrafo</p>
<p>Cuarto párrafo</p>
<input type="button" onclick="ChequeaObjetos()" value="Objetos con <p>">
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Obtiene la lista de objetos tipo <p>
//y cambia la apariencia de cada uno
function CambiaObjetos() {
    let arreglo = document.getElementsByTagName("p");
    for (let cont = 0; cont < arreglo.length; cont++) { //Cambia la apariencia de cada <p>
        arreglo[cont].style.color = "red";
        arreglo[cont].style["font-family"] = "Impact";
        arreglo[cont].style["font-size"] = "40px";
    }
}
</script>
<p>Primer párrafo</p>
<p>Segundo párrafo</p>
<p>Tercer párrafo</p>
<p>Cuarto párrafo</p>
<input type="button" onclick="CambiaObjetos()" value="Objetos con <p>">
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Muestra la ubicación URL de la página
function DarUbicacion() {
    let ubicacion = document.URL;
    alert("URL completa es: " + ubicacion);
}
</script>
<input type="button" onclick="DarUbicacion()" value="¿Dónde estoy?">
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head>
<style>
    .unestilo {
        color: blue;
        font-family: impact;
        font-size: 50px;
    }
</style></head><body>
<script>
//Aplica un atributo al primer <h1>
function AplicarAtributo() {
    let objetoH1 = document.getElementsByTagName("H1")[0];
    let Atributo = document.createAttribute("class");
    Atributo.value = "unestilo";
    objetoH1.setAttributeNode(Atributo);
}
</script>
<h1>Esta es una prueba</h1>
<h1>Segundo texto</h1>
<button onclick="AplicarAtributo()">Aplicar Atributo</button>
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Muestra que codificación tiene la página
function QueCodificacionTiene() {
    let codificacion = document.inputEncoding;
    alert("Codificación es: " + codificacion);
}
</script>
<input type="button" onclick="QueCodificacionTiene()" value="Mi codificación">
</body></html>
```



```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Lee el título de la página
function QueTituloTiene() {
    let titulo = document.title;
    alert("Título es: " + titulo);
}
</script>
<input type="button" onclick="QueTituloTiene()" value="Mi título">
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<pre>
<script>
//Escribe y deja un salto de línea pero requiere usar <pre> </pre>
document.writeln("Esta es una prueba");
document.writeln("de escritura de texto");
document.writeln("en diferentes líneas");
</script>
</pre>
</body></html>
```

El uso de document.write o document.writeln no está recomendado.



```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Muestra la fecha de modificación del documento
function Modificacion() {
    let fecha = document.lastModified;
    alert("Este documento fue modificado: " + fecha);
}
</script>
<input type="button" onclick="Modificacion()" value="¿Última modificación?">
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Muestra el tipo de documento
function Tipo() {
    let tipodocumento = document.doctype.name;
    alert("Tipo de documento: " + tipodocumento);
}
</script>
<input type="button" onclick="Tipo()" value="¿Tipo de documento?">
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Muestra el dominio donde está el documento
function DominioDocumento() {
    let dominio = document.domain;
    alert("Dominio: " + dominio);
}
</script>
<input type="button" onclick="DominioDocumento()" value="¿Dominio?">
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Muestra el estado del documento
function Estado() {
    let estadodocumento = document.readyState;
    alert("Estado del documento: " + estadodocumento);
}
</script>
<input type="button" onclick="Estado()" value="¿Está listo?">
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Muestra las dimensiones del documento
//Nota: Al agregar más líneas con <p> aumenta la altura
function Dimensiones() {
    let altura = document.body.clientHeight;
    let ancho = document.body.clientWidth;
    alert("Altura: " + altura + " Ancho: " + ancho);
}
</script>
<input type="button" onclick="Dimensiones()" value="Tamaños">
<p>Prueba</p>
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Cuenta el número de hipervínculos que hay en el documento
function TotalEnlaces() {
    alert("Enlaces: " + document.links.length);
}
</script>
<a href="http://darwin.50webs.com">Mi página web</a><br>
<a href="https://github.com/ramsoftware/">Mi repositorio</a><br>
<input type="button" onclick="TotalEnlaces()" value="¿Enlaces?">
</body></html>
```



```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Muestra cada hipervínculo que hay en el documento
function MuestraEnlaces() {
    for (let cont = 0; cont < document.links.length; cont++)
        alert("Enlace: " + document.links[cont].href);
}
</script>
</body>
<a href="http://darwin.50webs.com">Mi página web</a><br>
<a href="https://github.com/ramsoftware/">Mi repositorio</a><br>
<input type="button" onclick="MuestraEnlaces()" value="¿Enlaces?">
</html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
function CreaBotones() {
    //Genera un botón
    let boton = document.createElement("button");

    //Genera un texto
    let texto = document.createTextNode("Presionar");

    //Agrega el texto al botón
    boton.appendChild(texto);

    //Agrega el botón al <body>
    document.body.appendChild(boton);
}
</script>
<input type="button" onclick="CreaBotones()" value="Generar Botones">
</body></html>
```



```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Crea etiquetas en tiempo de ejecución
function CreaEtiquetas() {
  //Genera una etiqueta
  let etiqueta = document.createElement("H1");

  //Genera un texto
  let texto = document.createTextNode("Este es un texto");

  //Agrega el texto a la etiqueta
  etiqueta.appendChild(texto);

  //Agrega la etiqueta al <body>
  document.body.appendChild(etiqueta);
}
</script>
<input type="button" onclick="CreaEtiquetas()" value="Generar Etiquetas"></body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Crea etiquetas en tiempo de ejecución
function CreaEtiquetas() {
    let etiqueta = document.createElement("H1");
    let texto = document.createTextNode("Este es un texto");
    etiqueta.appendChild(texto);
    document.body.appendChild(etiqueta);
}

//Cuenta el número de etiquetas qhe hay en el documento
function TotalEtiquetas() {
    let etiquetas = document.getElementsByTagName("H1");
    alert("Total etiquetas: " + etiquetas.length);
}
</script>
<input type="button" onclick="CreaEtiquetas()" value="Generar Etiquetas">
<input type="button" onclick="TotalEtiquetas()" value="Total Etiquetas">
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Trae el número de elementos con un determinado "name"
function TotalElementos() {
    let elementosNombre = document.getElementsByName("prueba");
    alert("Número de elementos: " + elementosNombre.length);
}
</script>
<input name="prueba" type="text" value="valor A"><br>
Selección: <input name="prueba" type="radio" value="valor B"><br>
Botón: <input name="prueba" type="button" value="valor C"><br>
<input type="button" onclick="TotalElementos()" value="¿Total Elementos?">
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Trae el número de formularios
function TotalFormularios() {
    alert("Total formularios: " + document.forms.length);
}
</script>
<form name="Form1"></form>
<form name="Form2"></form>
<form></form>
<input type="button" onclick="TotalFormularios()" value="¿Total Formularios?">
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Trae el total de imágenes en el documento
function TotalImagenes() {
    alert("Total imágenes: " + document.images.length);
}
</script>
<br>
<input type="button" onclick="TotalImagenes()" value="¿Total imágenes?">
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Describe los objetos de la página
function Atributos() {
    let cajatexto = document.getElementsByTagName("p")[0];
    alert("Total atributos del texto: " + cajatexto.attributes.length);
    alert("Nombre atributo 1: " + cajatexto.attributes[0].name);
    alert("Valor del atributo 1: " + cajatexto.attributes[0].value);
    alert("Nombre atributo 2: " + cajatexto.attributes[1].name);
    alert("Valor del atributo 2: " + cajatexto.attributes[1].value);
}
</script>
<p id="identifica" name="textual">Esto es una prueba</p>
<input type="button" onclick="Atributos()" value="¿Atributos del texto?">
</body></html>
```



```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Abre una ventana secundaria con un ancho y largo determinados
function abrirventana() {
    window.open("http://darwin.50webs.com", "Investigación", "width=300,height=200");
}
</script>
<input type="button" value="Abrir ventana" onclick="abrirventana()" />
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
//Abre determinadas ventanas apuntando a diferentes direcciones
let ventana = 1;
let direccion = "https://github.com/ramsoftware";
function abrirventana() {
    switch (ventana) {
        case 1:
            direccion = "http://darwin.50webs.com";
            break;
        case 2:
            direccion = "http://es.wikipedia.org";
            break;
        case 3:
            direccion = "https://www.youtube.com/@RafaelMorenoP";
            break;
    }
    ventana++;
    if (ventana === 4) ventana = 1;
    window.open(direccion, "_blank", "width=400,height=300");
}
</script>
<input type="button" value="Abrir ventana" onclick="abrirventana()"/>
</body></html>
```



Un formulario que ejecuta un algoritmo local al enviar la información

El programa lee los datos que ha digitado el usuario, ejecuta y muestra en la misma página los resultados.

Cap11/034.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
/*¿Cuántos números hay entre limitemin y limitemax que las dos últimas cifras son múltiplo de 7?
  Se muestra un formulario en el que el usuario digita los valores limitemin y limitemax y el
  programa contesta en el mismo formulario */
function calcula() {
  let cuenta = 0;
  let cadena = "";
  let limiteminimo = parseInt(document.getElementById("limitemin").value);
  let limitemaximo = parseInt(document.getElementById("limitemax").value);
  for (let numero = limiteminimo; numero <= limitemaximo; numero++)
    if ((numero % 100) % 7 === 0) {
      cuenta++;
      cadena += numero + ", ";
    }
  document.getElementById("resultado").value = cuenta;
  document.getElementById("datos").value = cadena;
}
</script>
<form id="formulario" onsubmit="calcula(); return false;">
<p>¿Cuántos números hay entre "Mínimo" y "Máximo" que las dos últimas cifras son múltiplo de 7?</p>
Mínimo: <input type="text" id="limitemin"/><br>
Máximo: <input type="text" id="limitemax"/><br>
<input type="submit" id="boton" value="envía"/><br>
</form>
Total: <input type="text" id="resultado"/><br>
Números: <input type="text" size="200" id="datos"/>
</body></html>
```

Una segunda versión del programa modificando el uso del "submit"

Cap11/035.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
/*¿Cuántos números hay entre limitemin y limitemax que las dos últimas cifras son múltiplo de 7?
  Se muestra un formulario en el que el usuario digita los valores limitemin y limitemax y el
  programa contesta en el mismo formulario */
function calcula() {
  let cuenta = 0;
  let cadena = "";
  let limiteminimo = parseInt(document.getElementById("limitemin").value);
  let limitemaximo = parseInt(document.getElementById("limitemax").value);
  for (let numero = limiteminimo; numero <= limitemaximo; numero++)
    if ((numero % 100) % 7 === 0) {
      cuenta++;
      cadena += numero + ", ";
    }
  document.getElementById("resultado").value = cuenta;
  document.getElementById("datos").value = cadena;
}
</script>
<form id="formulario">
<p>¿Cuántos números hay entre "Mínimo" y "Máximo" que las dos últimas cifras son múltiplo de 7?</p>
Mínimo: <input type="text" id="limitemin"/><br>
Máximo: <input type="text" id="limitemax"/><br>
<input type="button" value="Calcular valor" onclick='calcula()' />
</form>
Total: <input type="text" id="resultado"/><br>
Números: <input type="text" size="200" id="datos"/>
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
function validacorreo() { /* Valida un correo con una expresión regular */
    let correo = document.getElementById("direccion").value;
    let exprReg = /^(([^<>() []\.\.,;:\s@""]+(\.[^<>() []\.\.\.,;:\s@""]+)*)|(\".+\"))@((\[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\)|((\[a-zA-Z\-0-9]+\.)+[a-zA-Z]{2,}))$/;
    let resultado = exprReg.test(correo);
    if (resultado)
        document.getElementById("mensaje").value = "La dirección de correo electrónico es correcta";
    else
        document.getElementById("mensaje").value = "Es errónea la dirección";
}
</script>
<form id="formulario" onsubmit="validacorreo(); return false;">
<p>Ingresa dirección de correo electrónico</p>
Correo: <input type="text" id="direccion" size="50"/><br>
<input type="submit" id="boton" value="envía"/><br>
</form>
Resultado: <input type="text" size="50" id="mensaje"/><br>
</body></html>
```

```
<!DOCTYPE HTML><html lang="es"><head></head><body><script>
/* Valida una URL con expresiones regulares */
function validaURL() {
    let url = document.getElementById("direccion").value;
    let re = /^(ht|f)tp(s?)\:\/\/[0-9a-zA-Z]([-.\w]*[0-9a-zA-Z])*(:(0-9)*)*(\/?)([a-zA-Z0-9\-\
\.\?\\,\'\/\\\+&%\$#_]*)?$/;
    let resultado = re.test(url);
    if (resultado)
        document.getElementById("mensaje").value = "La URL es correcta";
    else
        document.getElementById("mensaje").value = "Es errónea la URL";
}
</script>
<form id="formulario" onsubmit="validaURL(); return false;">
<p>Ingrese una URL</p>
URL: <input type="text" size="50" id="direccion"/><br>
<input type="submit" id="boton" value="envía"/><br>
</form>
Mensaje: <input type="text" size="50" id="mensaje"/><br>
</body></html>
```

# Capítulo 12. Gráficos

Con HTML5 tenemos los lienzos (canvas), zonas en las cuales se pueden dibujar líneas, óvalos, polígonos, etc. Con JavaScript se hacen los gráficos.

El primer paso es definir el lienzo (canvas) para dibujar y eso es con HTML5. Se debe especificar un “id” que será necesario para que JavaScript pueda trabajar con este.

Código básico:

Cap12/001.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="400" height="400"></canvas>
<script>
//Funciones para hacer gráficos
</script></body></html>
```

Si se quiere ver dónde está el lienzo, se añade un borde:

Cap12/002.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="200" height="200" style="border:1px solid #000000;"></canvas>
<script>
//Funciones para hacer gráficos
</script></body></html>
```



Ilustración 26:Lienzo (“canvas”) visible usando un borde

# Dibujar líneas

Usamos entonces instrucciones de JavaScript para hacer los gráficos. Se dibuja una línea.

Cap12/003.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="400" height="400"></canvas>
<script>
//Funciones para hacer gráficos

//Captura el lienzo
let lienzo = document.getElementById('milienzo');

//Captura el contexto (siempre es en 2d)
let contexto = lienzo.getContext('2d');

contexto.beginPath(); //Inicia el camino para dibujar
contexto.moveTo(50, 70); //Mueve el cursor a la posición X=50, Y=70
contexto.lineTo(140, 250); //Hace una línea entre (50,70) - (140,250)
contexto.stroke(); //Hace visible la línea
</script></body></html>
```



Ilustración 27: Línea dibujada



```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="400" height="400"></canvas>
<script>
//Funciones para hacer gráficos

//Captura el lienzo
let lienzo = document.getElementById('milienzo');

//Captura el contexto (siempre es en 2d)
let contexto = lienzo.getContext('2d');

contexto.beginPath(); //Inicia el camino para dibujar
contexto.moveTo(50, 70); //Mueve el cursor a la posición X=50, Y=70
contexto.lineTo(140, 250); //Hace una línea entre (50,70) - (140,250)
contexto.lineWidth = 20; //Grosor de esa línea
contexto.stroke(); //Hace visible la línea
</script></body></html>
```

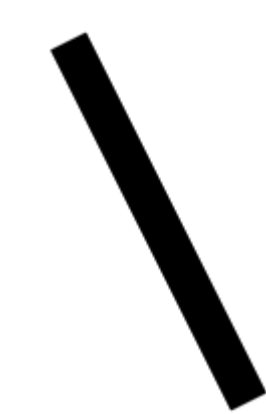


Ilustración 28: Línea gruesa

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="400" height="400"></canvas>
<script>
//Funciones para hacer gráficos

//Captura el lienzo
let lienzo = document.getElementById('milienzo');

//Captura el contexto (siempre es en 2d)
let contexto = lienzo.getContext('2d');

contexto.beginPath(); //Inicia el camino para dibujar
contexto.moveTo(50, 70); //Mueve el cursor a la posición X=50, Y=70
contexto.lineTo(140, 250); //Hace una línea entre (50,70) - (140,250)
contexto.lineWidth = 20; //Grosor de esa línea

contexto.strokeStyle = '#8A2BE2'; //Color de la línea
contexto.stroke(); //Hace visible la línea
</script></body></html>
```



Ilustración 29: Color a la línea



# Color RGB

El color puede ser expresado en RGB (Red, Green Blue). Conociendo los valores de esos colores primarios podemos combinar.

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="400" height="400"></canvas>
<script>
//Funciones para hacer gráficos

//Captura el lienzo
let lienzo = document.getElementById('milienzo');

//Captura el contexto (siempre es en 2d)
let contexto = lienzo.getContext('2d');

contexto.beginPath(); //Inicia el camino para dibujar
contexto.moveTo(50, 70); //Mueve el cursor a la posición X=50, Y=70
contexto.lineTo(140, 250); //Hace una línea entre (50,70) - (140,250)
contexto.lineWidth = 10; //Grosor de esa línea
contexto.strokeStyle = rgbToHexadecimal(19, 163, 204); //Color de la línea
contexto.stroke(); //Hace visible la línea

//Convierte los números RGB (Red, Green, Blue) en su equivalente hexadecimal de color
function rgbToHexadecimal(R, G, B) {
    return "#" + Hexadec(R) + Hexadec(G) + Hexadec(B);
}

//Convierte a Hexadecimal
function Hexadec(num) {
    num = parseInt(num, 10);
    if (isNaN(num)) return "00";
    num = Math.max(0, Math.min(num, 255));
    return "0123456789ABCDEF".charAt((num - num % 16) / 16) + "0123456789ABCDEF".charAt(num % 16);
}
</script></body></html>
```



Ilustración 30: Línea con color modificando los parámetros RGB

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="400" height="400"></canvas>
<script>
//Funciones para hacer gráficos

//Captura el lienzo
let lienzo = document.getElementById('milienzo');

//Captura el contexto (siempre es en 2d)
let contexto = lienzo.getContext('2d');

contexto.beginPath(); //Inicia el camino para dibujar
contexto.lineWidth = 20; //Grosor de esa línea
contexto.strokeStyle = '#8A2BE2'; //Color de la línea
contexto.moveTo(50, 20); //Mueve el cursor a la posición X=50, Y=20
contexto.lineTo(50, 250); //Hace una línea entre (50,20) - (50,250)

//Estilo de inicio y fin de las líneas
contexto.lineCap = 'round'; //Otras opciones son: 'butt' y 'square'

contexto.stroke(); //Hace visible la línea
</script></body></html>
```



Ilustración 31: Las líneas finalizan redondeadas en ambos extremos usando "round"

## Dibujar varias líneas

Requiere llamar tres veces la instrucción `beginPath()`. Dos terminaciones distintas.

Cap12/008.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="400" height="400"></canvas>
<script>
//Captura el lienzo
let lienzo = document.getElementById('milienzo');

//Captura el contexto (siempre es en 2d)
let contexto = lienzo.getContext('2d');

//Línea 1
contexto.beginPath(); //Inicia el camino para dibujar
contexto.lineWidth = 20; //Grosor de esa línea
contexto.strokeStyle = '#8A2BE2'; //Color de la línea
contexto.moveTo(50, 20); //Mueve el cursor a la posición X=50, Y=20
contexto.lineTo(50, 250); //Hace una línea entre (50,20) - (50,250)
contexto.lineCap = 'butt'; //Cierra extremos con butt
contexto.stroke(); //Hace visible la línea

//Línea 2 (misma longitud de Línea 1)
contexto.beginPath(); //Inicia el camino para dibujar
contexto.lineWidth = 20; //Grosor de esa línea
contexto.strokeStyle = '#8A2BE2'; //Color de la línea
contexto.moveTo(80, 20); //Mueve el cursor a la posición X=80, Y=20
contexto.lineTo(80, 250); //Hace una línea entre (80,20) - (80,250)
contexto.lineCap = 'square'; //Cierra extremos con square
contexto.stroke(); //Hace visible la línea

//Línea 3 (misma longitud de Línea 1)
contexto.beginPath(); //Inicia el camino para dibujar
contexto.lineWidth = 20; //Grosor de esa línea
contexto.strokeStyle = '#8A2BE2'; //Color de la línea
contexto.moveTo(110, 20); //Mueve el cursor a la posición X=110, Y=20
contexto.lineTo(110, 250); //Hace una línea entre (110,20) - (110,250)
contexto.lineCap = 'round'; //Cierra extremos con square
contexto.stroke(); //Hace visible la línea
</script></body></html>
```



Ilustración 32:Tres líneas con la misma longitud, pero diferente cierre de extremo

# Juntar líneas

Juntar dos líneas y que el punto de la unión tenga una característica (como ser redondeado)

Cap12/009.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="400" height="400"></canvas>
<script>
let lienzo = document.getElementById('milienzo');
let contexto = lienzo.getContext('2d');

contexto.lineWidth = 28; //Ancho de la línea
contexto.strokeStyle = 'green'; //Color

//Genera dos líneas y las junta
contexto.beginPath();
contexto.moveTo(10, 20);
contexto.lineTo(150, 90); //Línea 1
contexto.lineTo(270, 10); //Línea 2
contexto.lineJoin = 'round'; //miter, round, bevel
contexto.stroke();
</script></body></html>
```



Ilustración 33: Dos líneas juntas con un empalme redondeado

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="400" height="400"></canvas>
<script>
let lienzo = document.getElementById('milienzo');
let contexto = lienzo.getContext('2d');

contexto.lineWidth = 30; //Ancho de la línea
contexto.strokeStyle = 'green'; //Color

//Genera dos líneas y las junta
contexto.beginPath();
contexto.moveTo(10, 20);
contexto.lineTo(150, 90); //Línea 1
contexto.lineTo(270, 10); //Línea 2
contexto.lineJoin = 'round'; //miter, round, bevel
contexto.stroke();

//Genera dos líneas y las junta
contexto.beginPath();
contexto.moveTo(10, 80);
contexto.lineTo(150, 150); //Línea 1
contexto.lineTo(270, 70); //Línea 2
contexto.lineJoin = 'miter'; //miter, round, bevel
contexto.stroke();

//Genera dos líneas y las junta
contexto.beginPath();
contexto.moveTo(10, 140);
contexto.lineTo(150, 210); //Línea 1
contexto.lineTo(270, 130); //Línea 2
contexto.lineJoin = 'bevel'; //miter, round, bevel
contexto.stroke();
</script></body></html>
```



Ilustración 34: Diferentes formas de juntar dos líneas: "round", "miter", "bevel"

# Varias líneas

Con tres líneas hacemos una figura y luego es cerrada (hacemos una línea donde termina la tercera línea y donde empezó la primera línea).

Cap12/011.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="400" height="400"></canvas>
<script>
let lienzo = document.getElementById('milienzo');
let contexto = lienzo.getContext('2d');

contexto.lineWidth = 5; //Ancho de la línea
contexto.strokeStyle = 'green'; //Color

//Hace varias líneas continuas
contexto.beginPath();
contexto.moveTo(20, 30);
contexto.lineTo(250, 190); //Línea 1
contexto.lineTo(170, 50); //Línea 2
contexto.lineTo(280, 120); //Línea 3

//Hace una línea donde termina la línea 3 y empieza la línea 1
contexto.closePath();

contexto.stroke(); //Dibuja
</script></body></html>
```

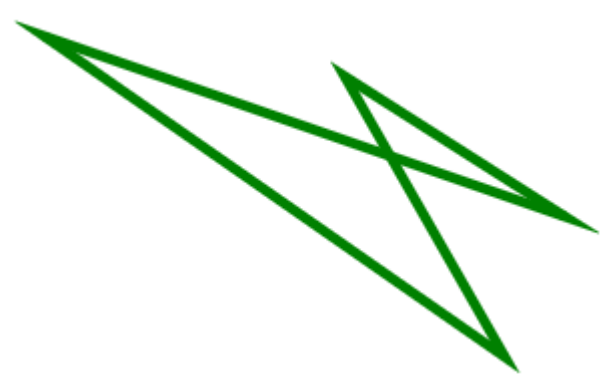


Ilustración 35: Una figura cerrada



```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="400" height="400"></canvas>
<script>
let lienzo = document.getElementById('milienzo');
let contexto = lienzo.getContext('2d');

contexto.lineWidth = 5; //Ancho de la línea
contexto.strokeStyle = 'green'; //Color

//Hace una figura, cerrándola
contexto.beginPath();
contexto.moveTo(20, 30);
contexto.lineTo(250, 190); //Linea 1
contexto.lineTo(170, 50); //Linea 2
contexto.closePath(); //Cierra la figura
contexto.stroke(); //Dibuja la figura

contexto.fillStyle = "red"; //Relleno rojo
contexto.fill(); //Rellena
</script></body></html>
```

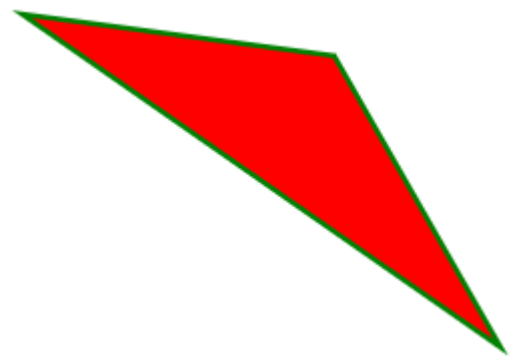


Ilustración 36: Figura cerrada y rellena de un color

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="400" height="400"></canvas>
<script>
let lienzo = document.getElementById('milienzo');
let contexto = lienzo.getContext('2d');

//Dibuja varias líneas horizontales
contexto.beginPath(); //Inicia el camino
for (let coordY = 10; coordY <= 500; coordY += 10) {
    contexto.moveTo(5, coordY);
    contexto.lineTo(600, coordY);
}
contexto.stroke();
</script></body></html>
```

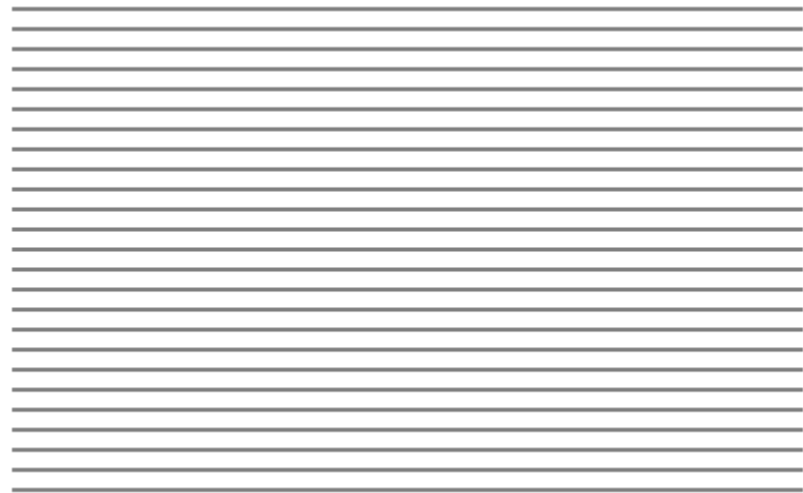


Ilustración 37: Líneas generadas por un ciclo



```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="400" height="400"></canvas>
<script>
let lienzo = document.getElementById('milienzo');
let contexto = lienzo.getContext('2d');

//Dibuja varias líneas verticales
contexto.beginPath(); //Inicia el camino
for (let coordX = 10; coordX <= 500; coordX += 50) {
    contexto.moveTo(coordX, 10);
    contexto.lineTo(coordX, 400);
}
contexto.stroke();
</script></body></html>
```

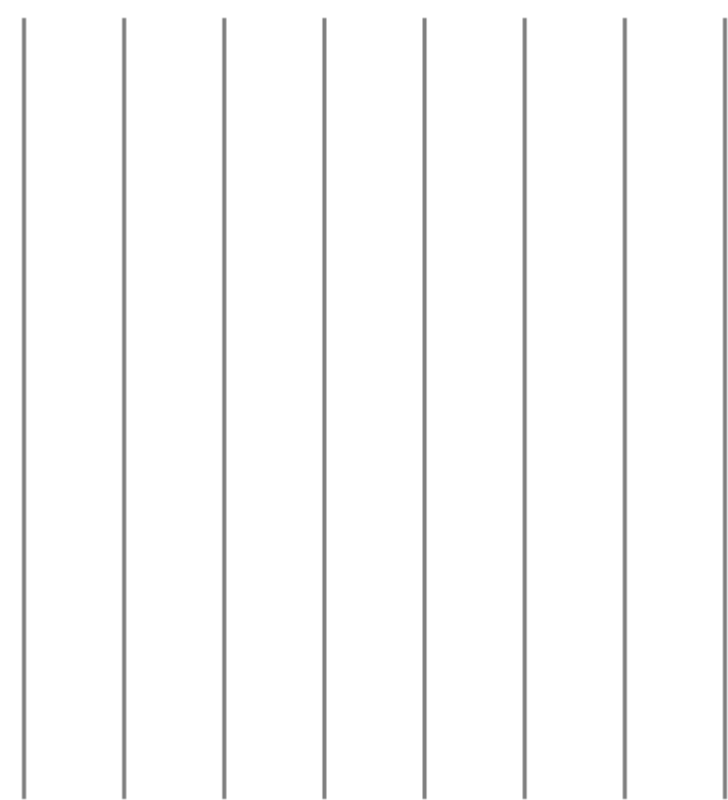


Ilustración 38: Líneas verticales dibujadas con un ciclo

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="600" height="600"></canvas>
<script>
let lienzo = document.getElementById('milienzo');
let contexto = lienzo.getContext('2d');

contexto.beginPath(); //Inicia el camino

//Ilusión de curva con líneas
for (let mover = 0; mover <= 600; mover += 10) {
    contexto.moveTo(0, mover);
    contexto.lineTo(mover, 600);
}
contexto.stroke();
</script></body></html>
```

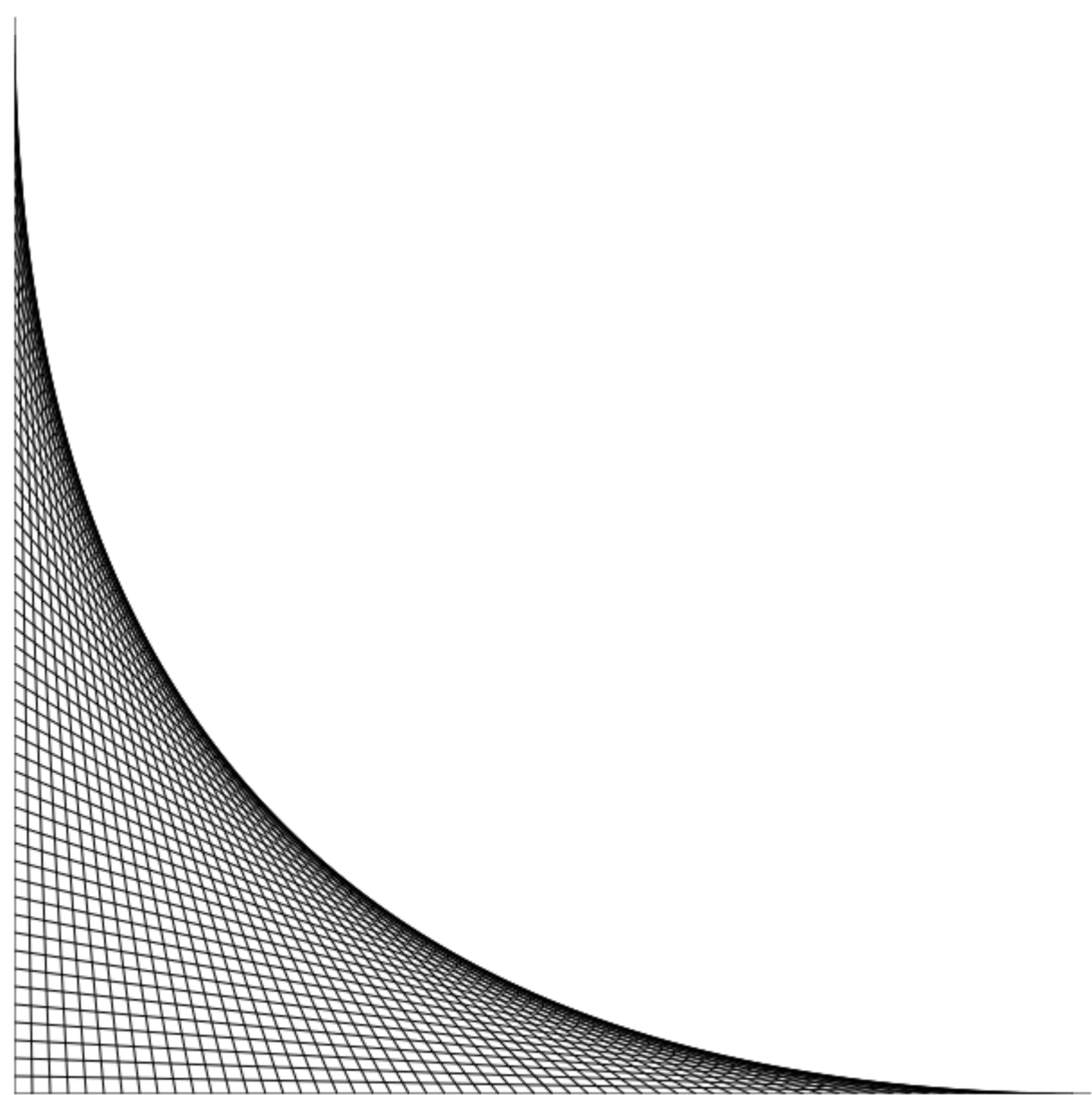


Ilustración 39: Ilusión de curva usando líneas rectas

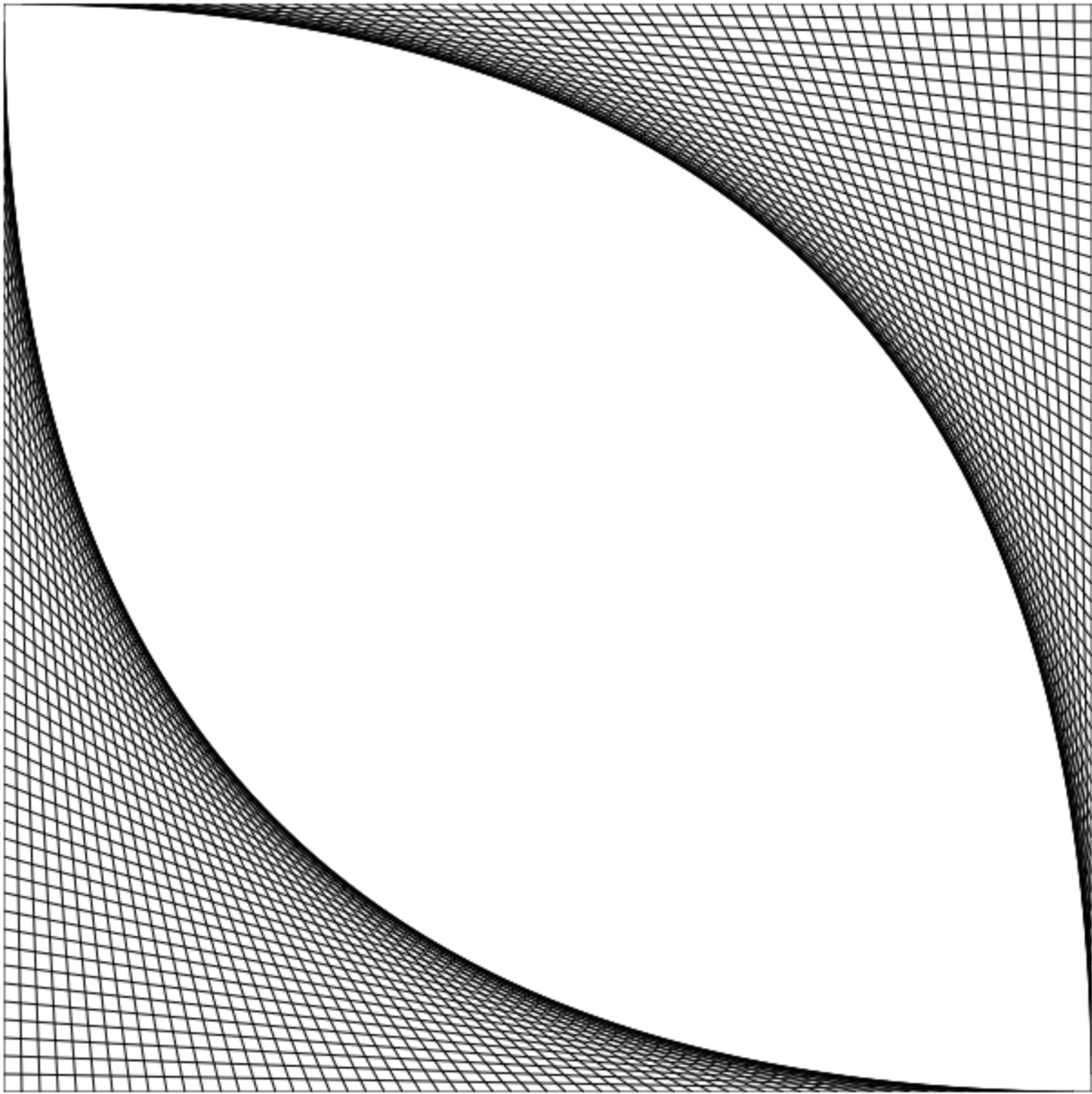
“Curvas” con líneas

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="600" height="600"></canvas>
<script>
let lienzo = document.getElementById('milienzo');
let contexto = lienzo.getContext('2d');

contexto.beginPath(); //Inicia el camino

//"Curvas" usando líneas
for (let mover = 0; mover <= 600; mover += 10) {
    contexto.moveTo(0, mover);
    contexto.lineTo(mover, 600);

    contexto.moveTo(mover, 0);
    contexto.lineTo(600, mover);
}
contexto.stroke();
```



*Ilustración 40: Ilusión de dos curvas usando líneas rectas*



Ilustración 41: La manera en que se dibujará el círculo

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="300" height="300"></canvas>
<script>
let lienzo = document.getElementById('milienzo');
let contexto = lienzo.getContext('2d');

let posX = lienzo.width / 2; //Centro del canvas en X
let posY = lienzo.height / 2; //Centro del canvas en Y
let Radio = 100; //Radio del círculo
let AnguloInicio = 0; //Angulo inicial (0 grados) en radianes
let AnguloFinal = 360 * Math.PI / 180; //Angulo final (360 grados) en radianes
let DibujaContraReloj = false;

contexto.beginPath();
contexto.arc(posX, posY, Radio, AnguloInicio, AnguloFinal, DibujaContraReloj);
contexto.lineWidth = 10; //Ancho de la línea
contexto.strokeStyle = 'green'; //Color
contexto.stroke();
</script></body></html>
```

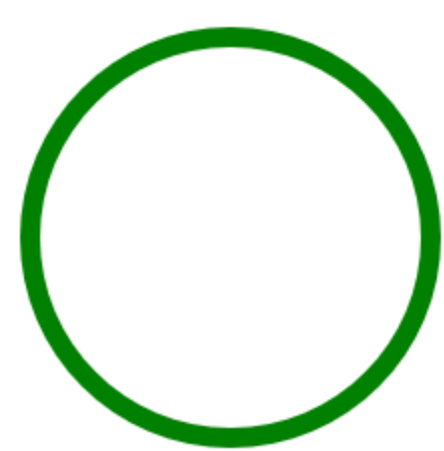


Ilustración 42: Círculo

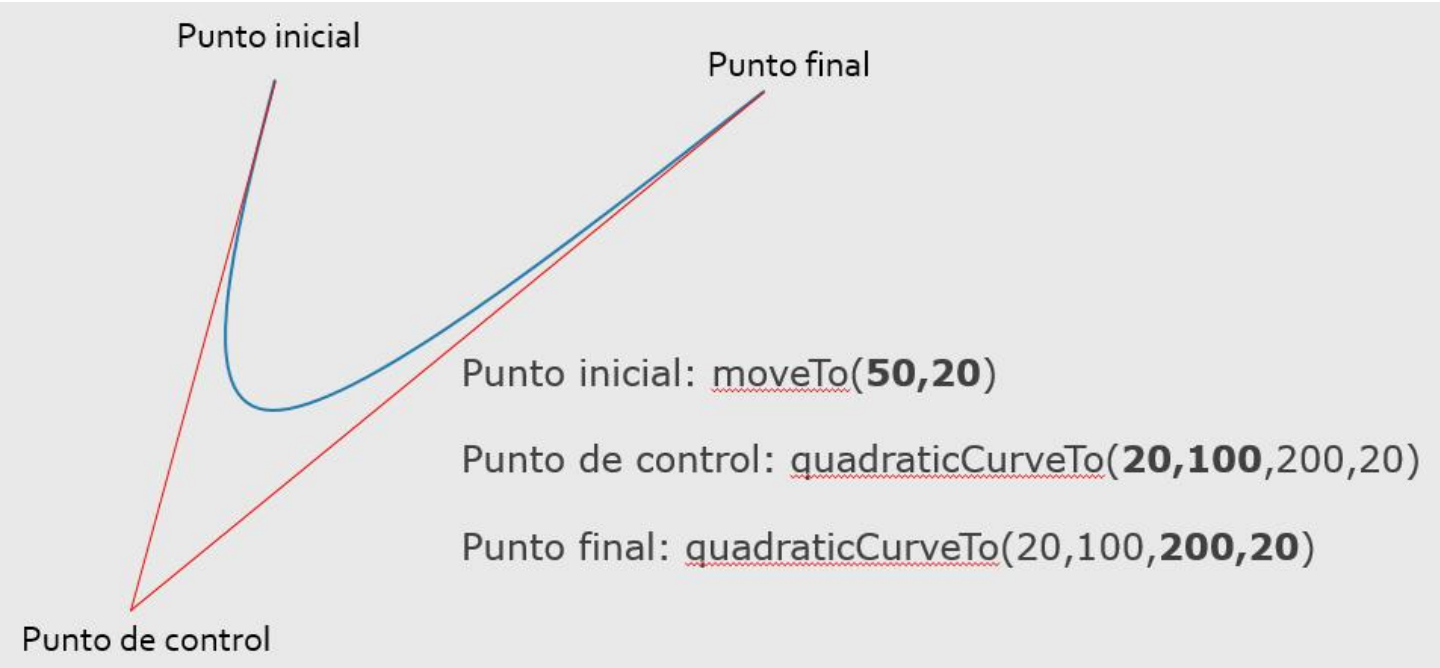


Ilustración 43: Forma en que se dibujarán las curvas

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="300" height="300"></canvas>
<script>
let lienzo = document.getElementById('milienzo');
let contexto = lienzo.getContext('2d');

contexto.beginPath();

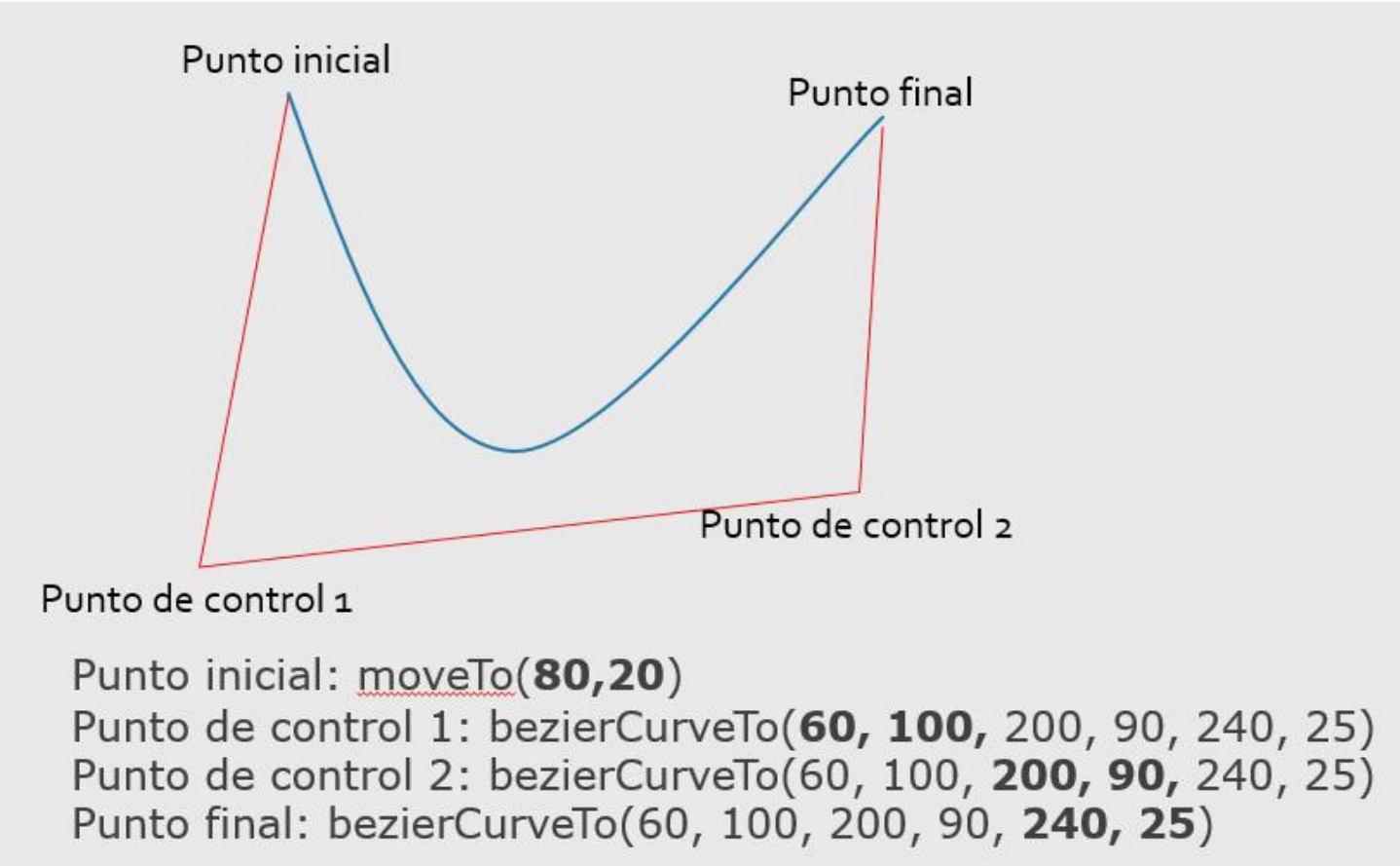
contexto.moveTo(50, 20); //Punto inicial
contexto.quadraticCurveTo(20, 100, 200, 20); //Punto de control y punto final

contexto.lineWidth = 5; //Ancho de la línea
contexto.strokeStyle = 'green'; //Color
contexto.stroke();
</script></body></html>
```



Ilustración 44: Curva

Curva de Bézier



Cap12/019.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="300" height="300"></canvas>
<script>
let lienzo = document.getElementById('milienzo');
let contexto = lienzo.getContext('2d');

contexto.beginPath();

contexto.moveTo(80, 20); //Punto inicial
contexto.bezierCurveTo(60, 100, 200, 90, 240, 25);

contexto.lineWidth = 5; //Ancho de la línea
contexto.strokeStyle = 'green'; //Color
contexto.stroke();
</script></body></html>
```



Ilustración 45: Curva Bézier



```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="300" height="300"></canvas>
<script>
let lienzo = document.getElementById('milienzo');
let contexto = lienzo.getContext('2d');
contexto.lineWidth = 5; //Ancho de la línea

IniciaElementoGrafico(contexto, 'green', 10, 20);
contexto.bezierCurveTo(30, 280, 120, 5, 290, 25);
contexto.stroke();

IniciaElementoGrafico(contexto, 'blue', 290, 25);
contexto.lineTo(150, 90);
contexto.stroke();

IniciaElementoGrafico(contexto, 'black', 150, 90);
contexto.quadraticCurveTo(320, 250, 200, 20);
contexto.stroke();

//Esta función ahorra tener que repetir instrucciones cada vez
//que se quiera dibujar un nuevo objeto
function IniciaElementoGrafico(contexto, quecolor, posX, posY) {
    contexto.beginPath();
    contexto.strokeStyle = quecolor;
    contexto.moveTo(posX, posY);
}
</script></body></html>
```

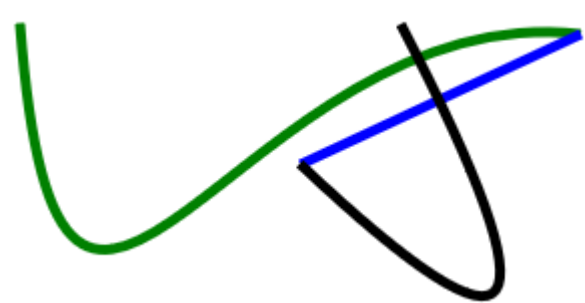


Ilustración 46: Combinando curvas



# Rectángulo y gradiente de color

Se puede tener un relleno de color que vaya cambiando de oscuro a más claro, o de un color a otro.

Cap12/021.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="400" height="400"></canvas>
<script>
let lienzo = document.getElementById('milienzo');
let contexto = lienzo.getContext('2d');

//Hace un rectángulo
contexto.rect(0, 0, lienzo.width, lienzo.height);

//Gradiente de cambio de color
let gradiente = contexto.createLinearGradient(0, 0, lienzo.width, lienzo.height);
gradiente.addColorStop(0, '#000000'); //Punto inicial de primer color 0%
gradiente.addColorStop(1, '#FFFFFF'); //Punto final de último color 100%
contexto.fillStyle = gradiente;
contexto.fill(); //Rellena
</script></body></html>
```

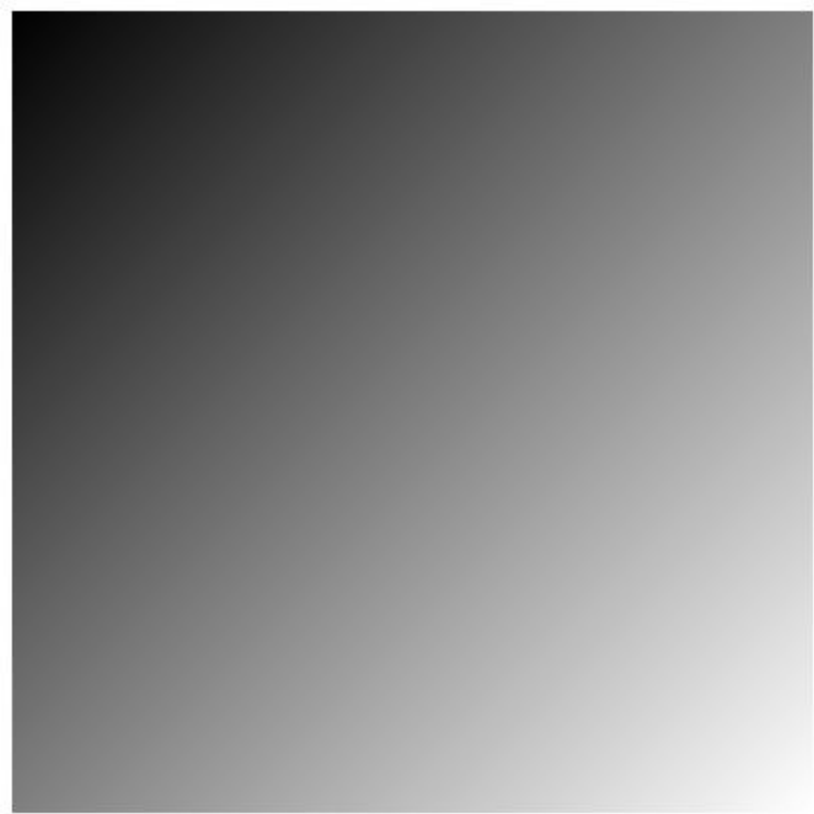


Ilustración 47: Variación del color



```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="400" height="400"></canvas>
<script>
let lienzo = document.getElementById('milienzo');
let contexto = lienzo.getContext('2d');

//Hace un rectángulo
contexto.rect(0, 0, lienzo.width, lienzo.height);

/* Crea un gradiente de color oblicuo: una línea imaginaria del punto
   superior izquierdo al punto inferior derecho. */
let gradiente = contexto.createLinearGradient(0, 0, lienzo.width, lienzo.height);

//Diferentes colores
gradiente.addColorStop(0, '#ABCDEF'); //Punto inicial de primer color
gradiente.addColorStop(0.2, '#BCD123');
gradiente.addColorStop(0.4, '#571ACB');
gradiente.addColorStop(0.6, '#C1B2D3');
gradiente.addColorStop(0.8, '#F1C5C2');
gradiente.addColorStop(1, '#012345'); //Punto final de último color
contexto.fillStyle = gradiente;
contexto.fill(); //Rellena
</script></body></html>
```

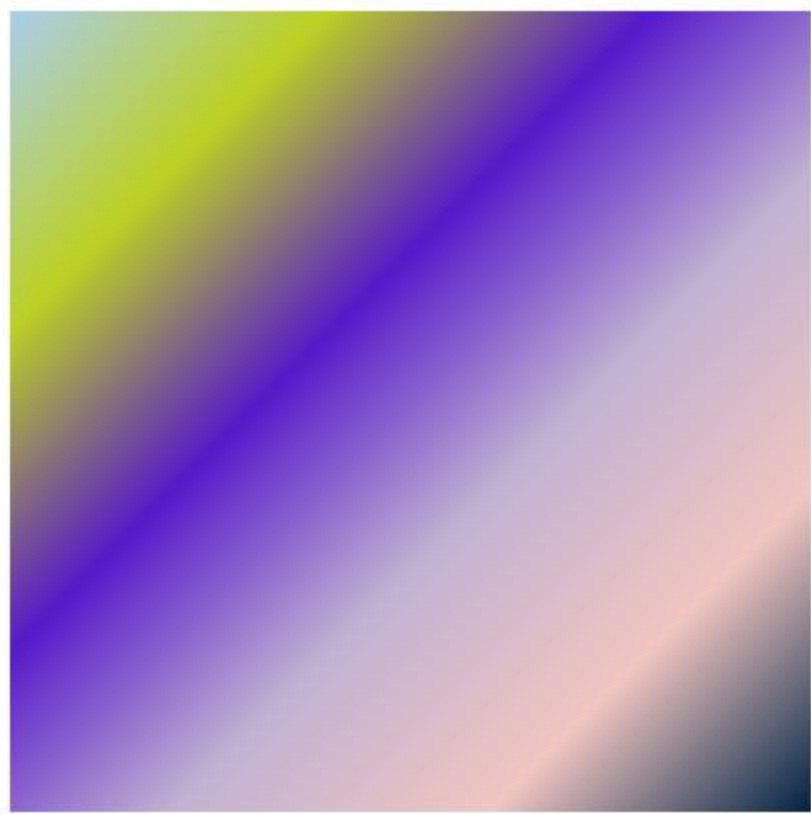


Ilustración 48: Variación de varios colores en forma oblicua

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="400" height="400"></canvas>
<script>
let lienzo = document.getElementById('milienzo');
let contexto = lienzo.getContext('2d');

//Hace un rectángulo
contexto.rect(0, 0, lienzo.width, lienzo.height);

/* Crea un gradiente de color de variación vertical */
let gradiente = contexto.createLinearGradient(0, lienzo.height / 2, lienzo.width, lienzo.height / 2);

//Diferentes colores
gradiente.addColorStop(0, '#ABCDEF'); //Punto inicial de primer color
gradiente.addColorStop(0.2, '#BCD123');
gradiente.addColorStop(0.4, '#571ACB');
gradiente.addColorStop(0.6, '#C1B2D3');
gradiente.addColorStop(0.8, '#F1C5C2');
gradiente.addColorStop(1, '#012345'); //Punto final de último color
contexto.fillStyle = gradiente;
contexto.fill(); //Rellena
</script></body></html>
```

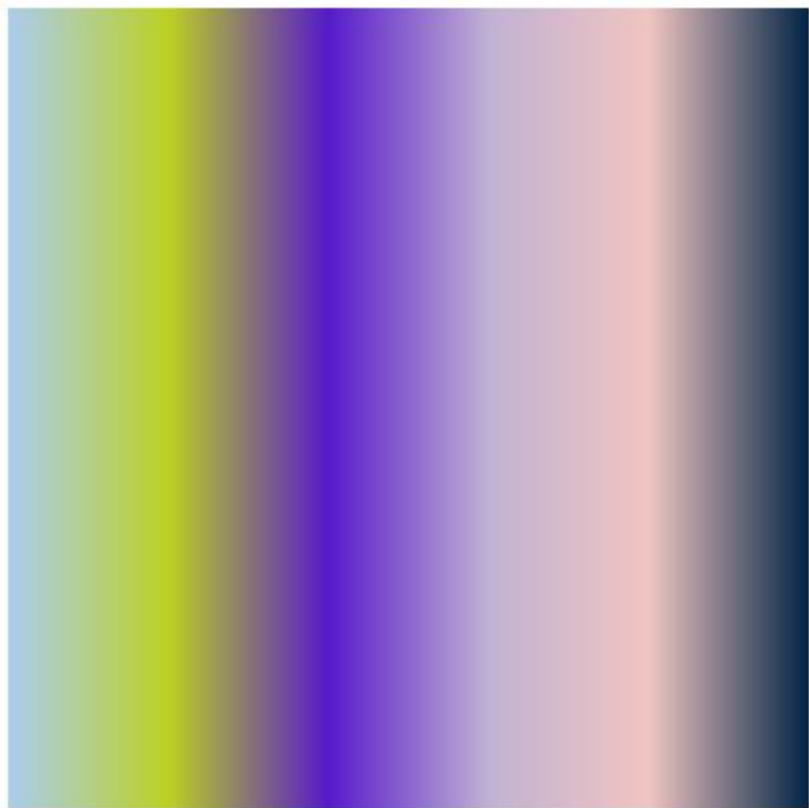


Ilustración 49: Variación de color en forma vertical

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="400" height="400"></canvas>
<script>
let lienzo = document.getElementById('milienzo');
let contexto = lienzo.getContext('2d');

//Hace un rectángulo
contexto.rect(0, 0, lienzo.width, lienzo.height);

/* Crea un gradiente de color de variación horizontal */
let gradiente = contexto.createLinearGradient(lienzo.width / 2, 0, lienzo.width / 2, lienzo.height);

//Diferentes colores
gradiente.addColorStop(0, '#ABCDEF'); //Punto inicial de primer color
gradiente.addColorStop(0.2, '#BCD123');
gradiente.addColorStop(0.4, '#571ACB');
gradiente.addColorStop(0.6, '#C1B2D3');
gradiente.addColorStop(0.8, '#F1C5C2');
gradiente.addColorStop(1, '#012345'); //Punto final de último color
contexto.fillStyle = gradiente;
contexto.fill(); //Rellena
</script></body></html>
```

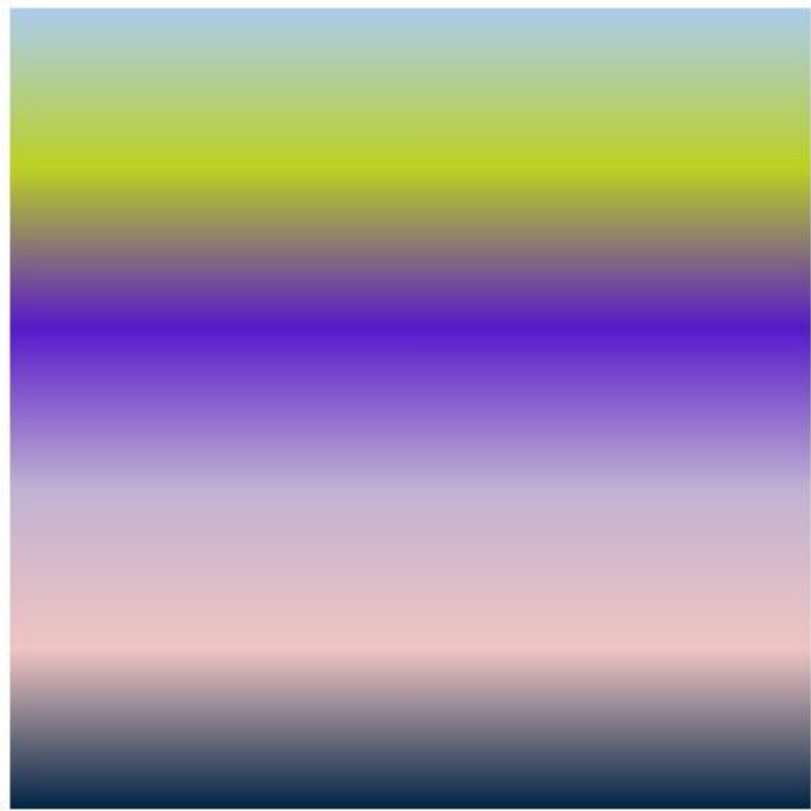


Ilustración 50: Variación de color horizontal

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="400" height="400"></canvas>
<script>
let lienzo = document.getElementById('milienzo');
let contexto = lienzo.getContext('2d');

//Hace un rectángulo
contexto.rect(0, 0, lienzo.width, lienzo.height);

/* Crea un gradiente radial que se define como dos círculos imaginarios, el que
   inicia y el que termina. (posX1, posY1, radiol, posX2, posY2, radio2 */
let gradiente = contexto.createRadialGradient(200, 200, 10, 200, 200, 200);

//Diferentes colores
gradiente.addColorStop(0, '#ABCDEF'); //Punto inicial de primer color
gradiente.addColorStop(0.2, '#BCD123');
gradiente.addColorStop(0.4, '#571ACB');
gradiente.addColorStop(0.6, '#C1B2D3');
gradiente.addColorStop(0.8, '#F1C5C2');
gradiente.addColorStop(1, '#012345'); //Punto final de último color
contexto.fillStyle = gradiente;
contexto.fill(); //Rellena
</script></body></html>
```

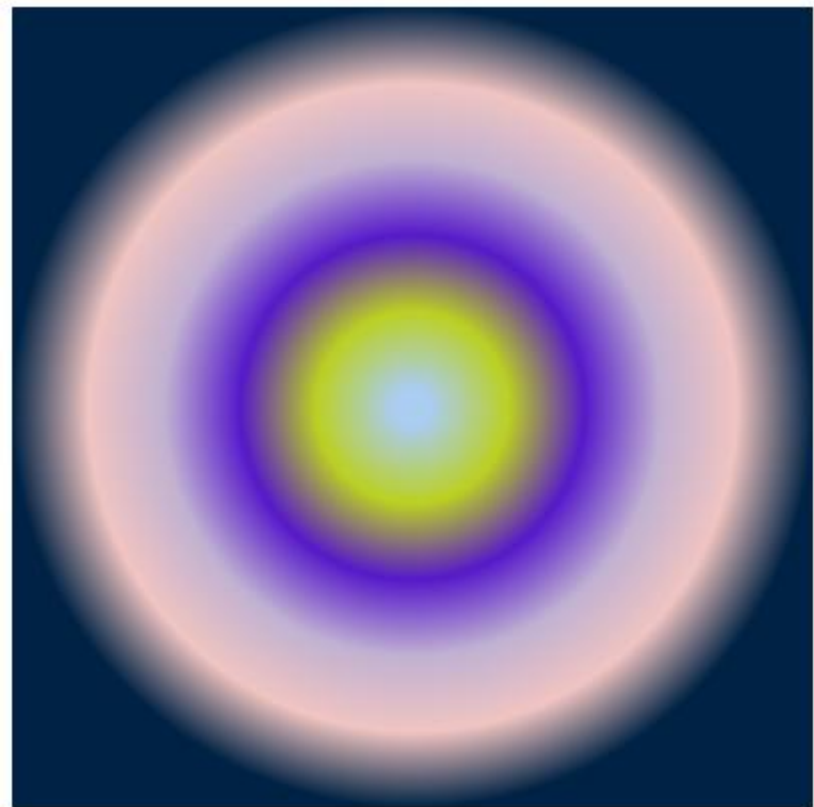


Ilustración 51: Variación en círculos

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="1000" height="1000"></canvas>
<script>
let lienzo = document.getElementById('milienzo');
let contexto = lienzo.getContext('2d');

contexto.rect(0, 0, lienzo.width, lienzo.height); //Un rectángulo

imagen = new Image();
imagen.src = 'logo.jpg'; //La imagen está en un archivo .jpg

//Si la imagen está cargada
imagen.onload = function () {
    contexto.fillStyle = contexto.createPattern(imagen, 'repeat');
    contexto.fill(); //Rellena
}

/* Probar con esto también:
    let patron = contexto.createPattern(imagen, 'repeat-x');
    let patron = contexto.createPattern(imagen, 'repeat-y');
    let patron = contexto.createPattern(imagen, 'no-repeat'); */
</script></body></html>
```

# Variar posición y tamaño de la imagen

Muestra la imagen, se puede variar la posición en el lienzo y su tamaño en ancho y alto

Cap12/027.html

```

<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="400" height="400"></canvas>
<script>
let lienzo = document.getElementById('milienzo');
let contexto = lienzo.getContext('2d');

contexto.rect(0, 0, lienzo.width, lienzo.height); //Un rectángulo

imagen = new Image();
imagen.src = 'logo.jpg'; //La imagen está en un archivo .jpg

//Si la imagen está cargada
imagen.onload = function () {
    //Muestra la imagen, cambiando posición y tamaño de esta
    //posX, posY, ancho y alto
    contexto.drawImage(imagen, 50, 50, 300, 30);
}
</script></body></html>

```



```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="miLienzo" width="595" height="420"></canvas>
<script>
let Imagen = new Image(); //Crea un objeto imagen
Imagen.src = 'Sally.jpg'; //donde se encuentra la imagen

Imagen.onload = function () {
    DibujarImagen(this);
}; //Cuando se cargue el objeto imagen

//Escala de grises
function DibujarImagen(Imagen) {
    let lienzo = document.getElementById('miLienzo');
    let contexto = lienzo.getContext('2d');
    contexto.drawImage(Imagen, 0, 0); //Dibuja la imagen original en 0,0

    //Trae los datos de esa imagen
    let datoImagen = contexto.getImageData(0, 0, Imagen.width, Imagen.height);
    let Datos = datoImagen.data;

    //Se va de pixel en pixel y lo convierte a escala de grises
    for (let i = 0; i < Datos.length; i += 4) {
        let grisaceo = 0.34 * Datos[i] + 0.5 * Datos[i + 1] + 0.16 * Datos[i + 2];
        Datos[i] = grisaceo;
        Datos[i + 1] = grisaceo;
        Datos[i + 2] = grisaceo;
    }
    contexto.putImageData(datoImagen, 0, 0); //Pone la imagen convertida sobre la imagen original
}
</script></body></html>
```



Ilustración 52: Imagen original



Ilustración 53: Imagen en escala de grises

Se pueden imprimir textos como imágenes.

Cap12/029.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="1200" height="800"></canvas>
<script>
let lienzo = document.getElementById('milienzo');
let contexto = lienzo.getContext('2d');

//Tipo de letra, tamaño, fuente
contexto.font = '60pt Courier';

//Texto a imprimir, posición X, posición Y
contexto.fillText('Esta es una prueba', 0, 150);
</script></body></html>
```

Esta es una pru

Ilustración 54: Dibuja un texto

Cap12/030.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="1200" height="800"></canvas>
<script>
let lienzo = document.getElementById('milienzo');
let contexto = lienzo.getContext('2d');

//Tipo de letra, tamaño, fuente
contexto.font = 'bold 45pt Verdana';

//Color que tendrá el texto
contexto.fillStyle = 'blue';

//Texto a imprimir, posición X, posición Y
contexto.fillText('Esta es una prueba', 10, 100);
</script></body></html>
```

Esta es una prueba

Ilustración 55: Dibuja un texto con relleno de color

Cap12/031.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="1200" height="800"></canvas>
<script>
let lienzo = document.getElementById('milienzo');
let contexto = lienzo.getContext('2d');

//Tipo de letra, tamaño, fuente
contexto.font = '50pt Verdana';

//Ancho de la línea
contexto.lineWidth = 1;

//Color de la línea
contexto.strokeStyle = 'orange';

//Texto a imprimir, posición X, posición Y
contexto.strokeText('Esta es una prueba', 10, 100);
</script></body></html>
```



# Esta es una prueba

Ilustración 56: Texto dibujado, se muestra el borde de cada letra

Cap12/032.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="1200" height="800"></canvas>
<script>
let lienzo = document.getElementById('milienzo');
let contexto = lienzo.getContext('2d');

//Dibuja una línea para hacer referencia
contexto.moveTo(350, 0);
contexto.lineTo(350, 350);
contexto.stroke();

//Tipo de letra, tamaño, fuente
contexto.font = 'bold 45pt Verdana';

//Alineación del texto: left, center, right, start, end
contexto.textAlign = 'end';

//Texto a imprimir, posición X, posición Y
contexto.fillText('Alinear', 350, 100);
</script></body></html>
```



Ilustración 57: Dibuja un texto alineado al final

Cap12/033.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="1200" height="800"></canvas>
<script>
let lienzo = document.getElementById('milienzo');
let contexto = lienzo.getContext('2d');

//Tipo de letra, tamaño, fuente
contexto.font = '80pt Courier';

//Texto a imprimir, posición X, posición Y
let texto = "Prueba";
contexto.fillText(texto, 10, 100);

//Tamaño del texto
let tamano = contexto.measureText(texto);
let ancho = tamano.width;
alert(ancho);
</script></body></html>
```



*Ilustración 58: Muestra el ancho del texto dibujado*

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="1200" height="800"></canvas>
<script>
let lienzo = document.getElementById('milienzo');
let contexto = lienzo.getContext('2d');

contexto.rect(50, 50, 200, 200);
contexto.fillStyle = 'green';

contexto.shadowColor = 'blue'; //Color de la sombra
contexto.shadowBlur = 50; //Definición de la sombra
contexto.shadowOffsetX = 40; //Corrimiento sombra en X
contexto.shadowOffsetY = 40; //Corrimiento sombra en Y

contexto.fill();
</script></body></html>
```

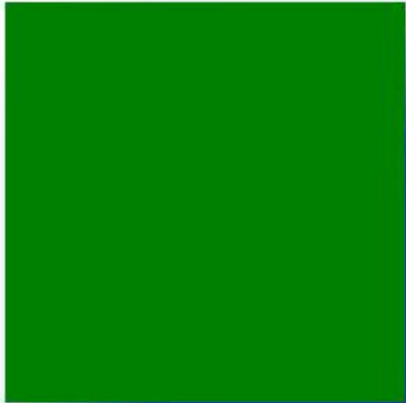


Ilustración 59: Sombra

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="1200" height="800"></canvas>
<script>
let lienzo = document.getElementById('milienzo');
let contexto = lienzo.getContext('2d');

//Cuadrado 1
contexto.beginPath();
contexto.rect(10, 10, 200, 200);
contexto.fillStyle = 'blue';
contexto.fill();

//Cuadrado 2... semitransparente
contexto.globalAlpha = 0.4;
contexto.beginPath();
contexto.rect(100, 100, 200, 200);
contexto.fillStyle = 'gray';
contexto.fill();
</script></body></html>
```

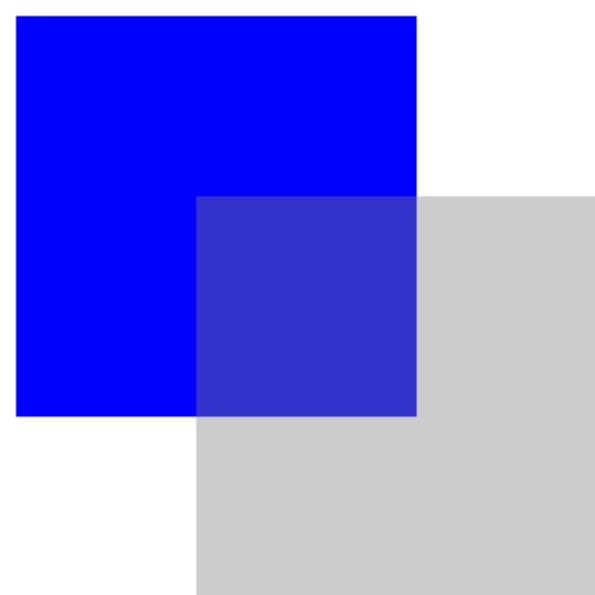


Ilustración 60: Rectángulo semitransparente

# Capítulo 13. Librería Gráfica

Una forma más sencilla de trabajar con gráficos es utilizar la técnica “wrapper” (envoltorio) de las funciones gráficas nativas de JavaScript. A continuación, la librería gráfica:

```
/** Clase con funciones para hacer gráficos
 * @Author Rafael Alberto Moreno Parra
 * @Version 1.1 (07 de julio de 2025)
 * */
class Grafica {
  /**
   * Captura el área gráfica (canvas)
   * @param {HTMLCanvasElement} Lienzo - El lienzo (id del canvas) de HTML5
   */
  constructor (Lienzo){
    this.Lienzo = Lienzo;
    this.Contexto = Lienzo.getContext('2d');
  }

  /**
   * Retorna el alto del área gráfica o lienzo
   * @return {number} Alto
   */
  LienzoAlto(){
    return this.Lienzo.height;
  }

  /**
   * Retorna el ancho del área gráfica o lienzo
   * @return {number} Ancho
   */
  LienzoAncho(){
    return this.Lienzo.width;
  }

  /**
   * Dibuja el perímetro del lienzo
   * @param {number} GrosorBorde - Grosor de la línea del perímetro
   * @param {string} ColorBorde - Color expresado en hexadecimal que tendrá la línea del perímetro
   */
  LienzoBorde(GrosorBorde, ColorBorde){
    this.Rectangulo(0, 0, this.Lienzo.width, this.Lienzo.height, GrosorBorde, ColorBorde);
  }

  /**
   * Poner un color de fondo al lienzo
   * @param {string} ColorFondo - Color expresado en hexadecimal que tendrá el fondo del lienzo
   */
  LienzoFondo(ColorFondo){
    this.RectanguloRelleno(0, 0, this.Lienzo.width, this.Lienzo.height, ColorFondo);
  }

  /**
   * Fondo y borde del lienzo
   * @param {string} ColorFondo - Color expresado en hexadecimal que tendrá el fondo del lienzo
   * @param {number} GrosorBorde - Grosor de la línea del perímetro
   * @param {string} ColorBorde - Color expresado en hexadecimal que tendrá la línea del perímetro
   */
  LienzoFondoBorde(ColorFondo, GrosorBorde, ColorBorde){
    this.LienzoFondo(ColorFondo);
    this.LienzoBorde(GrosorBorde, ColorBorde);
  }

  /**
   * Dibuja una línea recta
   * @param {number} Xinicio - Valor X de la coordenada inicial
   * @param {number} Yinicio - Valor Y de la coordenada inicial
   * @param {number} Xfin - Valor X de la coordenada final
   * @param {number} Yfin - Valor Y de la coordenada final
   * @param {number} GrosorLinea - Grosor de la línea
   * @param {string} ColorLinea - Color expresado en hexadecimal que tendrá la línea
   */
  Linea(Xinicio, Yinicio, Xfin, Yfin, GrosorLinea, ColorLinea){
    this.Contexto.beginPath();
    this.Contexto.lineWidth = GrosorLinea;
    this.Contexto.strokeStyle = ColorLinea;
    this.Contexto.moveTo(Xinicio, Yinicio); //Mueve el cursor a la posición
    this.Contexto.lineTo(Xfin, Yfin); //Hace una línea
    this.Contexto.stroke(); //Hace visible la línea
    this.Contexto.closePath();
  }

  /**
```



```

* Dibuja las líneas que conectan varios puntos
* @param {Punto[]} Puntos - Lista de objetos de la clase Punto
* @param {number} GrosorLinea - Grosor de la línea
* @param {string} ColorLinea - Color expresado en hexadecimal que tendrá la línea
*/
LineasPuntos(Puntos, GrosorLinea, ColorLinea){
    this.Contexto.beginPath();
    this.Contexto.lineWidth = GrosorLinea;
    this.Contexto.strokeStyle = ColorLinea;
    this.Contexto.moveTo(Puntos[0].posX, Puntos[0].posY); //Punto inicial
    for(let cont = 1; cont < Puntos.length; cont++){
        this.Contexto.lineTo(Puntos[cont].posX, Puntos[cont].posY);
    }
    this.Contexto.stroke(); //Hace visible la línea
    this.Contexto.closePath();
}

/**
* Dibuja las líneas que hay en un listado
* @param {Linea[]} LineaLst - Lista de objetos de la clase Linea
*/
LineasListas(LineaLst){
    for(let cont = 0; cont < LineaLst.length; cont++){
        this.LineaObj(LineaLst[cont]);
    }
}

/**
* Dibuja una línea dado el objeto línea
* @param {Linea} obj - Objeto Línea
*/
LineaObj(obj){
    this.Linea(obj.posXini, obj.posYini, obj.posXfin, obj.posYfin, obj.Grosor, obj.Color);
}

/**
* Dibuja un triángulo dada las coordenadas
* @param {number} posX1 - Valor X de la primera coordenada
* @param {number} posY1 - Valor Y de la primera coordenada
* @param {number} posX2 - Valor X de la segunda coordenada
* @param {number} posY2 - Valor Y de la segunda coordenada
* @param {number} posX3 - Valor X de la tercera coordenada
* @param {number} posY3 - Valor Y de la tercera coordenada
* @param {number} GrosorBorde - Grosor del borde del triángulo
* @param {string} ColorBorde - Color expresado en hexadecimal para el borde
*/
Triangulo(posX1, posY1, posX2, posY2, posX3, posY3, GrosorBorde, ColorBorde){
    this.Contexto.beginPath();
    this.Contexto.lineWidth = GrosorBorde;
    this.Contexto.strokeStyle = ColorBorde;
    this.Contexto.moveTo(posX1, posY1); //Mueve el cursor a la posición
    this.Contexto.lineTo(posX2, posY2);
    this.Contexto.lineTo(posX3, posY3);
    this.Contexto.lineTo(posX1, posY1);
    this.Contexto.stroke(); //Hace visible el triángulo
    this.Contexto.closePath();
}

/**
* Dibuja un triángulo relleno dada las coordenadas
* @param {number} posX1 - Valor X de la primera coordenada
* @param {number} posY1 - Valor Y de la primera coordenada
* @param {number} posX2 - Valor X de la segunda coordenada
* @param {number} posY2 - Valor Y de la segunda coordenada
* @param {number} posX3 - Valor X de la tercera coordenada
* @param {number} posY3 - Valor Y de la tercera coordenada
* @param {string} ColorRelleno - Color expresado en hexadecimal para el relleno
*/
TrianguloRelleno(posX1, posY1, posX2, posY2, posX3, posY3, ColorRelleno){
    this.Contexto.beginPath();
    this.Contexto.fillStyle = ColorRelleno;
    this.Contexto.moveTo(posX1, posY1); //Mueve el cursor a la posición
    this.Contexto.lineTo(posX2, posY2);
    this.Contexto.lineTo(posX3, posY3);
    this.Contexto.lineTo(posX1, posY1);
    this.Contexto.fill(); //Rellena con el color
    this.Contexto.closePath();
}

/**
* Dibuja un triángulo relleno de un color y borde de otro color
* @param {number} posX1 - Valor X de la primera coordenada
* @param {number} posY1 - Valor Y de la primera coordenada
* @param {number} posX2 - Valor X de la segunda coordenada
* @param {number} posY2 - Valor Y de la segunda coordenada
* @param {number} posX3 - Valor X de la tercera coordenada
* @param {number} posY3 - Valor Y de la tercera coordenada
* @param {string} ColorRelleno - Color expresado en hexadecimal para el relleno
* @param {number} GrosorBorde - Grosor del borde del triángulo
*/

```

```

    * @param {string} ColorBorde - Color expresado en hexadecimal para el borde
    */
    TrianguloRellenoBorde(posX1, posY1, posX2, posY2, posX3, posY3, ColorRelleno, GrosorBorde, ColorBorde){
        this.TrianguloRelleno(posX1, posY1, posX2, posY2, posX3, posY3, ColorRelleno);
        this.Triangulo(posX1, posY1, posX2, posY2, posX3, posY3, GrosorBorde, ColorBorde);
    }

    /**
     * Dibuja una lista de triángulos
     * @param {Triangulo[]} TrianguloLst - Lista de objetos de la clase Triangulo
     */
    TrianguloLista(TrianguloLst){
        for(let cont = 0; cont < TrianguloLst.length; cont++){
            this.TrianguloObj(TrianguloLst[cont]);
        }

        /**
         * Dibuja un triángulo dado el objeto triángulo
         * @param {Triangulo} obj - Objeto Triangulo
         */
        TrianguloObj(obj){
            switch(obj.Tipo) {
                case 1: this.Triangulo(obj.posX1, obj.posY1, obj.posX2, obj.posY2, obj.posX3, obj.posY3, obj.GrosorBorde,
obj.ColorBorde); break;
                case 2: this.TrianguloRelleno(obj.posX1, obj.posY1, obj.posX2, obj.posY2, obj.posX3, obj.posY3,
obj.ColorRelleno); break;
                case 3: this.TrianguloRellenoBorde(obj.posX1, obj.posY1, obj.posX2, obj.posY2, obj.posX3, obj.posY3,
obj.ColorRelleno, obj.GrosorBorde, obj.ColorBorde); break;
            }
        }

        /**
         * Dibuja un rectángulo
         * @param {number} posX - Posición X de la coordenada superior izquierda del rectángulo
         * @param {number} posY - Posición Y de la coordenada superior izquierda del rectángulo
         * @param {number} Ancho - Ancho del rectángulo
         * @param {number} Alto - Alto del rectángulo
         * @param {number} GrosorBorde - Grosor del borde del rectángulo
         * @param {string} ColorBorde - Color expresado en hexadecimal para el borde
         */
        Rectangulo(posX, posY, Ancho, Alto, GrosorBorde, ColorBorde){
            this.Contexto.beginPath();
            this.Contexto.lineWidth = GrosorBorde;
            this.Contexto.strokeStyle = ColorBorde;
            this.Contexto.rect(posX, posY, Ancho, Alto);
            this.Contexto.stroke(); //Hace visible el rectángulo
            this.Contexto.closePath();
        }

        /**
         * Dibuja un rectángulo relleno
         * @param {number} posX - Posición X de la coordenada superior izquierda del rectángulo
         * @param {number} posY - Posición Y de la coordenada superior izquierda del rectángulo
         * @param {number} Ancho - Ancho del rectángulo
         * @param {number} Alto - Alto del rectángulo
         * @param {string} ColorRelleno - Color expresado en hexadecimal para el relleno
         */
        RectanguloRelleno(posX, posY, Ancho, Alto, ColorRelleno){
            this.Contexto.beginPath();
            this.Contexto.fillStyle = ColorRelleno;
            this.Contexto.fillRect(posX, posY, Ancho, Alto);
            this.Contexto.closePath();
        }

        /**
         * Dibuja un rectángulo relleno con borde
         * @param {number} posX - Posición X de la coordenada superior izquierda del rectángulo
         * @param {number} posY - Posición Y de la coordenada superior izquierda del rectángulo
         * @param {number} Ancho - Ancho del rectángulo
         * @param {number} Alto - Alto del rectángulo
         * @param {string} ColorRelleno - Color expresado en hexadecimal para el relleno
         * @param {number} GrosorBorde - Grosor del borde del rectángulo
         * @param {string} ColorBorde - Color expresado en hexadecimal para el borde
         */
        RectanguloRellenoBorde(posX, posY, Ancho, Alto, ColorRelleno, GrosorBorde, ColorBorde){
            this.RectanguloRelleno(posX, posY, Ancho, Alto, ColorRelleno);
            this.Rectangulo(posX, posY, Ancho, Alto, GrosorBorde, ColorBorde);
        }

        /**
         * Dibuja una lista de rectángulos
         * @param {Rectangulo[]} RectanguloLst - Lista de objetos de la clase Rectangulo
         */
        RectanguloLista(RectanguloLst){
            for(let cont = 0; cont < RectanguloLst.length; cont++){
                this.RectanguloObj(RectanguloLst[cont]);
            }

```



```

/**
 * Dibuja un rectángulo dado el objeto rectángulo
 * @param {Rectangulo} obj - Objeto Rectangulo
 */
RectanguloObj(obj){
    switch(obj.Tipo){
        case 1: this.Rectangulo(obj.posX, obj.posY, obj.Ancho, obj.Alto, obj.GrosorBorde, obj.ColorBorde); break;
        case 2: this.RectanguloRelleno(obj.posX, obj.posY, obj.Ancho, obj.Alto, obj.ColorRelleno); break;
        case 3: this.RectanguloRellenoBorde(obj.posX, obj.posY, obj.Ancho, obj.Alto, obj.ColorRelleno,
obj.GrosorBorde, obj.ColorBorde); break;
    }
}

/**
 * Dibujar un polígono de 4 lados
 * @param {number} posX1 - Valor X de la primera coordenada
 * @param {number} posY1 - Valor Y de la primera coordenada
 * @param {number} posX2 - Valor X de la segunda coordenada
 * @param {number} posY2 - Valor Y de la segunda coordenada
 * @param {number} posX3 - Valor X de la tercera coordenada
 * @param {number} posY3 - Valor Y de la tercera coordenada
 * @param {number} posX4 - Valor X de la cuarta coordenada
 * @param {number} posY4 - Valor Y de la cuarta coordenada
 * @param {number} GrosorBorde - Grosor del borde del rectángulo
 * @param {string} ColorBorde - Color expresado en hexadecimal para el borde
 */
Poligono4Lados(posX1, posY1, posX2, posY2, posX3, posY3, posX4, posY4, GrosorBorde, ColorBorde){
    this.Contexto.beginPath();
    this.Contexto.lineWidth = GrosorBorde;
    this.Contexto.strokeStyle = ColorBorde;
    this.Contexto.moveTo(posX1, posY1); //Mueve el cursor a la posición
    this.Contexto.lineTo(posX2, posY2);
    this.Contexto.lineTo(posX3, posY3);
    this.Contexto.lineTo(posX4, posY4);
    this.Contexto.lineTo(posX1, posY1);
    this.Contexto.stroke(); //Hace visible el polígono
    this.Contexto.closePath();
}

/**
 * Dibujar un polígono de 4 lados relleno
 * @param {number} posX1 - Valor X de la primera coordenada
 * @param {number} posY1 - Valor Y de la primera coordenada
 * @param {number} posX2 - Valor X de la segunda coordenada
 * @param {number} posY2 - Valor Y de la segunda coordenada
 * @param {number} posX3 - Valor X de la tercera coordenada
 * @param {number} posY3 - Valor Y de la tercera coordenada
 * @param {number} posX4 - Valor X de la cuarta coordenada
 * @param {number} posY4 - Valor Y de la cuarta coordenada
 * @param {string} ColorRelleno - Color expresado en hexadecimal para el relleno
 */
Poligono4LadosRelleno(posX1, posY1, posX2, posY2, posX3, posY3, posX4, posY4, ColorRelleno){
    this.Contexto.beginPath();
    this.Contexto.fillStyle = ColorRelleno;
    this.Contexto.moveTo(posX1, posY1); //Mueve el cursor a la posición
    this.Contexto.lineTo(posX2, posY2);
    this.Contexto.lineTo(posX3, posY3);
    this.Contexto.lineTo(posX4, posY4);
    this.Contexto.lineTo(posX1, posY1);
    this.Contexto.fill(); //Rellena con el color
    this.Contexto.closePath();
}

/**
 * Dibujar un polígono de 4 lados relleno
 * @param {number} posX1 - Valor X de la primera coordenada
 * @param {number} posY1 - Valor Y de la primera coordenada
 * @param {number} posX2 - Valor X de la segunda coordenada
 * @param {number} posY2 - Valor Y de la segunda coordenada
 * @param {number} posX3 - Valor X de la tercera coordenada
 * @param {number} posY3 - Valor Y de la tercera coordenada
 * @param {number} posX4 - Valor X de la cuarta coordenada
 * @param {number} posY4 - Valor Y de la cuarta coordenada
 * @param {string} ColorRelleno - Color expresado en hexadecimal para el relleno
 * @param {number} GrosorBorde - Grosor del borde del rectángulo
 * @param {string} ColorBorde - Color expresado en hexadecimal para el borde
 */
Poligono4LadosRellenoBorde(posX1, posY1, posX2, posY2, posX3, posY3, posX4, posY4, ColorRelleno, GrosorBorde,
ColorBorde){
    this.Poligono4LadosRelleno(posX1, posY1, posX2, posY2, posX3, posY3, posX4, posY4, ColorRelleno);
    this.Poligono4Lados(posX1, posY1, posX2, posY2, posX3, posY3, posX4, posY4, GrosorBorde, ColorBorde);
}

/**
 * Dibuja una lista de polígonos de 4 lados
 * @param {Poligono4Lados[]} Poligono4LadosLst - Lista de objetos de la clase Poligono4Lados

```

```

*/
Poligono4LadosLista(Poligono4LadosLst){
    for(let cont = 0; cont < Poligono4LadosLst.length; cont++){
        this.Poligono4LadosObj(Poligono4LadosLst[cont]);
    }
}

/**
 * Dibuja un polígono de 4 lados dado el objeto polígono de 4 lados
 * @param {Poligono4Lados} obj - Objeto Poligono4Lados
 */
Poligono4LadosObj(obj){
    switch(obj.Tipo){
        case 1: this.Poligono4Lados(obj.posX1, obj.posY1, obj.posX2, obj.posY2, obj.posX3, obj.posY3, obj.posX4,
obj.posY4, obj.GrosorBorde, obj.ColorBorde); break;
        case 2: this.Poligono4LadosRelleno(obj.posX1, obj.posY1, obj.posX2, obj.posY2, obj.posX3, obj.posY3,
obj.posX4, obj.posY4, obj.ColorRelleno); break;
        case 3: this.Poligono4LadosRellenoBorde(obj.posX1, obj.posY1, obj.posX2, obj.posY2, obj.posX3, obj.posY3,
obj.posX4, obj.posY4, obj.ColorRelleno, obj.GrosorBorde, obj.ColorBorde); break;
    }
}

/**
 * Dibuja un polígono dados los puntos
 * @param {Punto[]} Puntos - Lista de Puntos
 * @param {number} GrosorBorde - Grosor del borde
 * @param {string} ColorBorde - Color expresado en hexadecimal para el borde
 */
Poligono(Puntos, GrosorBorde, ColorBorde){
    this.Contexto.beginPath();
    this.Contexto.lineWidth = GrosorBorde;
    this.Contexto.strokeStyle = ColorBorde;
    this.Contexto.moveTo(Puntos[0].posX, Puntos[0].posY); //Punto inicial
    for(let cont = 1; cont < Puntos.length; cont++){
        this.Contexto.lineTo(Puntos[cont].posX, Puntos[cont].posY);
    }
    this.Contexto.lineTo(Puntos[0].posX, Puntos[0].posY);
    this.Contexto.stroke(); //Hace visible el polígono
    this.Contexto.closePath();
}

/**
 * Dibuja un polígono relleno dados los puntos
 * @param {Punto[]} Puntos - Lista de Puntos
 * @param {string} ColorRelleno - Color expresado en hexadecimal para el relleno
 */
PoligonoRelleno(Puntos, ColorRelleno){
    this.Contexto.beginPath();
    this.Contexto.fillStyle = ColorRelleno;
    this.Contexto.moveTo(Puntos[0].posX, Puntos[0].posY); //Punto inicial
    for(let cont = 1; cont < Puntos.length; cont++){
        this.Contexto.lineTo(Puntos[cont].posX, Puntos[cont].posY);
    }
    this.Contexto.lineTo(Puntos[0].posX, Puntos[0].posY);
    this.Contexto.fill(); //Rellena con el color
    this.Contexto.closePath();
}

/**
 * Dibujar polígono relleno con borde
 * @param {Punto[]} Puntos - Lista de Puntos
 * @param {string} ColorRelleno - Color expresado en hexadecimal para el relleno
 * @param {number} GrosorBorde - Grosor del borde
 * @param {string} ColorBorde - Color expresado en hexadecimal para el borde
 */
PoligonoRellenoBorde(Puntos, ColorRelleno, GrosorBorde, ColorBorde){
    this.PoligonoRelleno(Puntos, ColorRelleno);
    this.Poligono(Puntos, GrosorBorde, ColorBorde);
}

/**
 * Dibuja una curva Bézier
 * @param {number} IniX - Valor X de la coordenada inicial
 * @param {number} IniY - Valor Y de la coordenada inicial
 * @param {number} ControlX1 - Valor X de la primera coordenada de control
 * @param {number} ControlY1 - Valor Y de la primera coordenada de control
 * @param {number} ControlX2 - Valor X de la segunda coordenada de control
 * @param {number} ControlY2 - Valor Y de la segunda coordenada de control
 * @param {number} FinalX3 - Valor X de la coordenada final
 * @param {number} FinalY3 - Valor Y de la coordenada final
 * @param {number} GrosorLinea - Grosor de la línea
 * @param {string} ColorLinea - Color expresado en hexadecimal para la línea
 */
Bezier(IniX, IniY, ControlX1, ControlY1, ControlX2, ControlY2, FinalX3, FinalY3, GrosorLinea, ColorLinea){
    this.Contexto.beginPath();
    this.Contexto.lineWidth = GrosorLinea;
    this.Contexto.strokeStyle = ColorLinea;
    this.Contexto.moveTo(IniX, IniY); //Punto inicial
    this.Contexto.bezierCurveTo(ControlX1, ControlY1, ControlX2, ControlY2, FinalX3, FinalY3);
    this.Contexto.stroke(); //Hace visible la curva
    this.Contexto.closePath();
}

```

```

}

/**
 * Dibuja un arco
 * @param {number} posX - Valor X de la coordenada central del arco
 * @param {number} posY - Valor Y de la coordenada central del arco
 * @param {number} Radio - Radio que tendrá el arco
 * @param {number} angIni - Ángulo inicial en grados del arco
 * @param {number} angFin - Ángulo final en grados del arco
 * @param {number} GrosorLinea - Grosor de la línea
 * @param {string} ColorLinea - Color expresado en hexadecimal para la línea
 * @param {boolean} DibujaContraReloj - Si se dibuja el arco a contrareloj (true) o siguiendo las manecillas del
reloj (false)
 */
Arco(posX, posY, Radio, angIni, angFin, GrosorLinea, ColorLinea, DibujaContraReloj){
    this.Contexto.beginPath();
    this.Contexto.lineWidth = GrosorLinea;
    this.Contexto.strokeStyle = ColorLinea;
    let AnguloIni = angIni * Math.PI / 180; //Angulo inicial (0 grados) en radianes
    let AnguloFin = angFin * Math.PI / 180; //Angulo final (360 grados) en radianes
    this.Contexto.arc(posX, posY, Radio, AnguloIni, AnguloFin, DibujaContraReloj);
    this.Contexto.stroke(); //Hace visible la curva
    this.Contexto.closePath();
}

/**
 * Dibuja una curva Cuadrática
 * @param {number} IniX - Valor X de la coordenada inicial de la curva
 * @param {number} IniY - Valor Y de la coordenada inicial de la curva
 * @param {number} ControlX1 - Valor X de la coordenada de control de la curva
 * @param {number} ControlY1 - Valor Y de la coordenada de control de la curva
 * @param {number} FinalX2 - Valor X de la coordenada final de la curva
 * @param {number} FinalY2 - Valor Y de la coordenada final de la curva
 * @param {number} GrosorLinea - Grosor de la línea
 * @param {string} ColorLinea - Color expresado en hexadecimal para la línea
 */
CurvaCuadratica(IniX, IniY, ControlX1, ControlY1, FinalX2, FinalY2, GrosorLinea, ColorLinea){
    this.Contexto.beginPath();
    this.Contexto.lineWidth = GrosorLinea;
    this.Contexto.strokeStyle = ColorLinea;
    this.Contexto.moveTo(IniX, IniY); //Punto inicial
    this.Contexto.quadraticCurveTo(ControlX1, ControlY1, FinalX2, FinalY2);
    this.Contexto.stroke(); //Hace visible la curva
    this.Contexto.closePath();
}

/**
 * Dibuja una elipse
 * @param {number} posX - Coordenada X del centro de la elipse
 * @param {number} posY - Coordenada Y del centro de la elipse
 * @param {number} RadioX - Radio en el eje X de la elipse
 * @param {number} RadioY - Radio en el eje Y de la elipse
 * @param {number} RotaElipse - Rotación de la elipse en grados
 * @param {number} angIni - Ángulo inicial en grados de dibujo de la elipse
 * @param {number} angFin - Ángulo final en grados de dibujo de la elipse
 * @param {boolean} ContraReloj - Si se dibuja la elipse a contrareloj (true) o siguiendo las manecillas del
reloj (false)
 * @param {number} GrosorBorde - Grosor del borde
 * @param {string} ColorBorde - Color expresado en hexadecimal para el borde
 */
Elipse(posX, posY, RadioX, RadioY, RotaElipse, angIni, angFin, ContraReloj, GrosorBorde, ColorBorde){
    this.Contexto.beginPath();
    this.Contexto.lineWidth = GrosorBorde;
    this.Contexto.strokeStyle = ColorBorde;
    let AnguloIni = angIni * Math.PI / 180; //Angulo inicial (0 grados) en radianes
    let AnguloFin = angFin * Math.PI / 180; //Angulo final (360 grados) en radianes
    let AnguloRota = RotaElipse * Math.PI / 180;
    this.Contexto.ellipse(posX, posY, RadioX, RadioY, AnguloRota, AnguloIni, AnguloFin, ContraReloj);
    this.Contexto.stroke(); //Hace visible la elipse
    this.Contexto.closePath();
}

/**
 * Dibuja una elipse rellena
 * @param {number} posX - Coordenada X del centro de la elipse
 * @param {number} posY - Coordenada Y del centro de la elipse
 * @param {number} RadioX - Radio en el eje X de la elipse
 * @param {number} RadioY - Radio en el eje Y de la elipse
 * @param {number} RotaElipse - Rotación de la elipse en grados
 * @param {number} angIni - Ángulo inicial en grados de dibujo de la elipse
 * @param {number} angFin - Ángulo final en grados de dibujo de la elipse
 * @param {boolean} ContraReloj - Si se dibuja la elipse a contrareloj (true) o siguiendo las manecillas del
reloj (false)
 * @param {string} ColorRelleno - Color expresado en hexadecimal para el relleno
 */
ElipseRellena(posX, posY, RadioX, RadioY, RotaElipse, angIni, angFin, ContraReloj, ColorRelleno){
    this.Contexto.beginPath();

```

```

        this.Contexto.fillStyle = ColorRelleno;
        let AnguloIni = angIni * Math.PI / 180; //Angulo inicial (0 grados) en radianes
        let AnguloFin = angFin * Math.PI / 180; //Angulo final (360 grados) en radianes
        let AnguloRota = RotaElipse * Math.PI / 180;
        this.Contexto.ellipse(posX, posY, RadioX, RadioY, AnguloRota, AnguloIni, AnguloFin, ContraRelej);
        this.Contexto.fill(); //Rellena con el color
        this.Contexto.closePath();
    }

    /**
     * Dibuja una elipse rellena con borde
     * @param {number} posX - Coordenada X del centro de la elipse
     * @param {number} posY - Coordenada Y del centro de la elipse
     * @param {number} RadioX - Radio en el eje X de la elipse
     * @param {number} RadioY - Radio en el eje Y de la elipse
     * @param {number} RotaElipse - Rotación de la elipse en grados
     * @param {number} angIni - Ángulo inicial en grados de dibujo de la elipse
     * @param {number} angFin - Ángulo final en grados de dibujo de la elipse
     * @param {boolean} ContraRelej - Si se dibuja la elipse a contrarelej (true) o siguiendo las manecillas del
    reloj (false)
     * @param {string} ColorRelleno - Color expresado en hexadecimal para el relleno
     * @param {number} GrosorBorde - Grosor del borde
     * @param {string} ColorBorde - Color expresado en hexadecimal para el borde
     */
    ElipseRellenaBorde(posX, posY, RadioX, RadioY, RotaElipse, angIni, angFin, ContraRelej, ColorRelleno,
    GrosorBorde, ColorBorde){
        this.Elipse(posX, posY, RadioX, RadioY, RotaElipse, angIni, angFin, ContraRelej, GrosorBorde, ColorBorde);
        this.ElipseRellena(posX, posY, RadioX, RadioY, RotaElipse, angIni, angFin, ContraRelej, ColorRelleno);
    }

    /**
     * Dibuja una lista de elipses
     * @param {Elipse[]} ElipseLst - Lista de objetos de la clase Elipse
     */
    ElipseLista(ElipseLst){
        for(let cont = 0; cont < ElipseLst.length; cont++){
            this.ElipseObj(ElipseLst[cont]);
        }
    }

    /**
     * Dibuja una elipse dado el objeto elipse
     * @param {Elipse} obj - Objeto Elipse
     */
    ElipseObj(obj){
        switch(obj.Tipo){
            case 1: this.Elipse(obj.posX, obj.posY, obj.RadioX, obj.RadioY, obj.RotaFigura, obj.angIni, obj.angFin,
obj.Relej, obj.GrosorBorde, obj.ColorBorde); break;
            case 2: this.ElipseRellena(obj.posX, obj.posY, obj.RadioX, obj.RadioY, obj.RotaFigura, obj.angIni,
obj.angFin, obj.Relej, obj.ColorRelleno); break;
            case 3: this.ElipseRellenaBorde(obj.posX, obj.posY, obj.RadioX, obj.RadioY, obj.RotaFigura, obj.angIni,
obj.angFin, obj.Relej, obj.ColorRelleno, obj.GrosorBorde, obj.ColorBorde); break;
        }
    }

    /**
     * Dibuja un círculo
     * @param {number} posX - Coordenada X del centro del círculo
     * @param {number} posY - Coordenada Y del centro del círculo
     * @param {number} Radio - Radio del círculo
     * @param {number} GrosorBorde - Grosor del perímetro del círculo
     * @param {string} ColorBorde - Color expresado en hexadecimal para el borde
     */
    Circulo(posX, posY, Radio, GrosorBorde, ColorBorde){
        this.Contexto.beginPath();
        this.Contexto.lineWidth = GrosorBorde;
        this.Contexto.strokeStyle = ColorBorde;
        this.Contexto.arc(posX, posY, Radio, 0, 2 * Math.PI, false);
        this.Contexto.stroke(); //Dibuja el círculo
        this.Contexto.closePath();
    }

    /**
     * Dibujar un círculo relleno
     * @param {number} posX - Coordenada X del centro del círculo
     * @param {number} posY - Coordenada Y del centro del círculo
     * @param {number} Radio - Radio del círculo
     * @param {string} ColorRelleno - Color expresado en hexadecimal para el relleno
     */
    CirculoRelleno(posX, posY, Radio, ColorRelleno){
        this.Contexto.beginPath();
        this.Contexto.fillStyle = ColorRelleno;
        this.Contexto.arc(posX, posY, Radio, 0, 2 * Math.PI, false);
        this.Contexto.fill(); //Rellena con el color
        this.Contexto.closePath();
    }
}

/**

```



```

* Dibujar un círculo relleno con borde
* @param {number} posX - Coordenada X del centro del círculo
* @param {number} posY - Coordenada Y del centro del círculo
* @param {number} Radio - Radio del círculo
* @param {string} ColorRelleno - Color expresado en hexadecimal para el relleno
* @param {number} GrosorBorde - Grosor del borde
* @param {string} ColorBorde - Color expresado en hexadecimal para el borde
*/
CirculoRellenoBorde(posX, posY, Radio, ColorRelleno, GrosorBorde, ColorBorde) {
    this.CirculoRelleno(posX, posY, Radio, ColorRelleno);
    this.Circulo(posX, posY, Radio, GrosorBorde, ColorBorde);
}

/**
* Dibuja una lista de círculos
* @param {Circulo[]} CirculoLst - Lista de círculos
*/
CirculoLista(CirculoLst){
    for(let cont = 0; cont < CirculoLst.length; cont++){
        this.CirculoObj(CirculoLst[cont]);
    }

/**
* Dibuja un círculo dado el objeto círculo
* @param {Circulo} obj - Objeto círculo
*/
CirculoObj(obj){
    switch(obj.Tipo){
        case 1: this.Circulo(obj.posX, obj.posY, obj.Radio, obj.GrosorBorde, obj.ColorBorde); break;
        case 2: this.CirculoRelleno(obj.posX, obj.posY, obj.Radio, obj.ColorRelleno); break;
        case 3: this.CirculoRellenoBorde(obj.posX, obj.posY, obj.Radio, obj.ColorRelleno, obj.GrosorBorde,
obj.ColorBorde); break;
    }
}

/**
* Dibuja una cadena de texto con un color de borde
* @param {number} posX - Posición en X del texto
* @param {number} posY - Posición en Y del texto
* @param {string} Texto - Texto a mostrar en pantalla
* @param {string} Color - Color expresado en hexadecimal para el texto
* @param {string} Fuente - Características de la fuente a usar
*/
DibujarCadenas(posX, posY, Texto, Color, Fuente){
    this.Contexto.beginPath();
    this.Contexto.font = Fuente; //Tipo, tamaño, fuente
    this.Contexto.strokeStyle = Color;
    this.Contexto.strokeText(Texto, posX, posY); //Inferior
    this.Contexto.closePath();
}

/**
* Dibuja una cadena de texto con color de relleno
* @param {number} posX - Posición en X del texto
* @param {number} posY - Posición en Y del texto
* @param {string} Texto - Texto a mostrar en pantalla
* @param {string} Color - Color expresado en hexadecimal para el relleno del texto
* @param {string} Fuente - Características de la fuente a usar
*/
DibujarCadenasRelleno(posX, posY, Texto, Color, Fuente){
    this.Contexto.beginPath();
    this.Contexto.font = Fuente; //Tipo, tamaño, fuente
    this.Contexto.fillStyle = Color;
    this.Contexto.fillText(Texto, posX, posY); //Superior
    this.Contexto.closePath();
}

// ===== GRAFICO MATEMATICO 2D =====

/**
* Hace el gráfico matemático generado por la ecuación Y=F(X)
* @param {string} Ecuacion - Expresión matemático del tipo Y = F(X), por ejemplo Y = sen(X)-cos(2*X)+tan(X^3) .
Se pueden usar las siguientes funciones sen() seno, cos() coseno, tan() tangente, abs() valor absoluto, asn()
arcoseno, acs() arcocoseno, atn() arcotangente, log() logaritmo natural, cei() valor techo, exp() exponencial, sqr()
raíz cuadrada; + suma, - resta, * multiplicación, / división, ^ potencia; paréntesis y una variable independiente X
* @param {number} Xini - Valor X inicial desde dónde se evalúa la ecuación
* @param {number} Xfin - Valor X final hasta dónde se evalúa la ecuación
* @param {number} numPuntos - Total de puntos que se van a calcular entre Xini y Xfin
* @param {number} Grosor - Grosor de la línea
* @param {string} Color - Color expresado en hexadecimal para la línea
*/
Matematico2D(Ecuacion, Xini, Xfin, numPuntos, Grosor, Color){
    //Llama al evaluador de expresiones
    let evaluador2022 = new Evaluador4();
    evaluador2022.Analizar(Ecuacion);

    //Calcula los puntos de la ecuación a graficar

```

```

let pasoX = (Xfin - Xini) / numPuntos;
let Ymin = Number.MAX_VALUE; //El mínimo valor de Y obtenido
let Ymax = -Ymin; //El máximo valor de Y obtenido
let maximoXreal = -Ymin; //El máximo valor de X (difiere de Xfin)

let puntos = [];
for (let X = Xini; X <= Xfin; X += pasoX) {
    evaluador2022.DarValorVariable('x', X);
    let valY = -1*evaluador2022.Evaluar(); //Se invierte el valor porque el eje Y aumenta hacia abajo
    if (valY > Ymax) Ymax = valY;
    if (valY < Ymin) Ymin = valY;
    if (X > maximoXreal) maximoXreal = X;
    puntos.push(new Punto(X, valY));
}

//¡OJO! X puede que no llegue a ser Xfin, por lo que la variable maximoXreal almacena el valor máximo de X

//Calcula los puntos a poner en la pantalla
let convierteX = this.LienzoAncho() / (maximoXreal - Xini);
let convierteY = this.LienzoAlto() / (Ymax - Ymin);

for (let cont = 0; cont < puntos.length; cont++) {
    puntos[cont].posX = Math.round(convierteX * (puntos[cont].posX - Xini));
    puntos[cont].posY = Math.round(convierteY * (puntos[cont].posY - Ymin));
}

//Hace el gráfico
this.LineasPuntos(puntos, Grosor, Color);
}

/**
 * Convierte los números RGB (Red, Green, Blue) en su equivalente hexadecimal de color
 * @param {number} Rojo - Valor de 0 a 254 de rojo
 * @param {number} Verde - Valor de 0 a 254 de verde
 * @param {number} Azul - Valor de 0 a 254 de azul
 * @return {string} Color en hexadecimal
 */
ColorRGB(Rojo, Verde, Azul) {
    return "#" + this.Hexadecimal(Rojo) + this.Hexadecimal(Verde) + this.Hexadecimal(Azul);
}

/**
 * Convierte un número en hexadecimal
 * @param {number} Numero - Número a convertir
 * @returns {string} - Número en hexadecimal
 */
Hexadecimal(Numero) {
    if (isNaN(Numero)) return "00";
    return "0123456789ABCDEF".charAt((Numero-Numero%16)/16)+"0123456789ABCDEF".charAt(Numero%16);
}

/**
 * Retorna un color al azar en formato hexadecimal
 * @returns {string} - Color en formato hexadecimal
 */
ColorAzar() {
    let Rojo = Math.round(Math.random() * 255);
    let Verde = Math.round(Math.random() * 255);
    let Azul = Math.round(Math.random() * 255);
    return this.ColorRGB(Rojo, Verde, Azul);
}
}

/** Clase para mantener un conjunto de puntos
 * @Author Rafael Alberto Moreno Parra
 * @Version 1.0 (05 de enero de 2024)
 * */
class Punto {
    /**
     * Constructor
     * @param {number} posX - Valor X de la coordenada
     * @param {number} posY - Valor Y de la coordenada
     */
    constructor(posX, posY) {
        this.posX = posX;
        this.posY = posY;
    }
}

/** Clase para mantener un conjunto de líneas
 * @Author Rafael Alberto Moreno Parra
 * @Version 1.0 (05 de enero de 2024)
 * */
class Linea {
    /**
     * Constructor

```

```

* @param {number} posXini - Valor X de la coordenada de inicio
* @param {number} posYini - Valor Y de la coordenada de inicio
* @param {number} posXfin - Valor X de la coordenada final
* @param {number} posYfin - Valor Y de la coordenada final
* @param {number} Grosor - Grosor de la línea
* @param {string} Color - Color expresado en hexadecimal de la línea
*/
constructor(posXini, posYini, posXfin, posYfin, Grosor, Color){
    this.posXini = posXini;
    this.posYini = posYini;
    this.posXfin = posXfin;
    this.posYfin = posYfin;
    this.Grosor = Grosor;
    this.Color = Color;
}

/**
 * Trasladar una línea
 * @param {number} mueveX - Traslado en el eje X
 * @param {number} mueveY - Traslado en el eje Y
 */
Trasladar(mueveX, mueveY){
    this.posXini += mueveX;
    this.posYini += mueveY;
    this.posXfin += mueveX;
    this.posYfin += mueveY;
}

/**
 * Girar la línea calculando su centroide
 * @param {number} angulo - Ángulo en grados de giro
 */
Girar(angulo){
    let anguloR = angulo * Math.PI / 180;

    //Giro de las dos coordenadas
    let posXinig = Math.round(this.posXini * Math.cos(anguloR) - this.posYini * Math.sin(anguloR));
    let posYinig = Math.round(this.posXini * Math.sin(anguloR) + this.posYini * Math.cos(anguloR));
    let posXfing = Math.round(this.posXfin * Math.cos(anguloR) - this.posYfin * Math.sin(anguloR));
    let posYfing = Math.round(this.posXfin * Math.sin(anguloR) + this.posYfin * Math.cos(anguloR));

    //Giro del centroide
    let centroX = (this.posXini + this.posXfin) / 2;
    let centroY = (this.posYini + this.posYfin) / 2;
    let centroXg = Math.round(centroX * Math.cos(anguloR) - centroY * Math.sin(anguloR));
    let centroYg = Math.round(centroX * Math.sin(anguloR) + centroY * Math.cos(anguloR));

    //¿Cuánto se desplazó el centroide?
    let desplazaX = centroXg - centroX;
    let desplazaY = centroYg - centroY;

    //Retira el desplazamiento
    this.posXini = posXinig - desplazaX;
    this.posYini = posYinig - desplazaY;
    this.posXfin = posXfing - desplazaX;
    this.posYfin = posYfing - desplazaY;
}
}

/** Clase para mantener un conjunto de triángulos
 * @Author Rafael Alberto Moreno Parra
 * @Version 1.0 (05 de enero de 2024)
 * */
class Triangulo {
    /**
     * Constructor
     * @param {number} posX1 - Valor X de la primera coordenada
     * @param {number} posY1 - Valor Y de la primera coordenada
     * @param {number} posX2 - Valor X de la segunda coordenada
     * @param {number} posY2 - Valor Y de la segunda coordenada
     * @param {number} posX3 - Valor X de la tercera coordenada
     * @param {number} posY3 - Valor Y de la tercera coordenada
     * @param {number} Tipo - Tipo triángulo: 1. Sólo perímetro, 2. Relleno, 3. Perímetro de un color y relleno de
otro color
     * @param {number} GrosorBorde - Grosor del borde
     * @param {string} ColorBorde - Color expresado en hexadecimal del borde
     * @param {string} ColorRelleno - Color expresado en hexadecimal del relleno
     */
    constructor(posX1, posY1, posX2, posY2, posX3, posY3, Tipo, GrosorBorde, ColorBorde, ColorRelleno){
        this.posX1 = posX1;
        this.posY1 = posY1;
        this.posX2 = posX2;
        this.posY2 = posY2;
        this.posX3 = posX3;
        this.posY3 = posY3;
        this.Tipo = Tipo; //1. Solo borde, 2. Sólo relleno, 3. Borde y relleno
        this.GrosorBorde = GrosorBorde;
    }
}

```

```

        this.ColorBorde = ColorBorde;
        this.ColorRelleno = ColorRelleno;
    }

/**
 * Trasladar un triángulo
 * @param {number} mueveX - Traslado en el eje X
 * @param {number} mueveY - Traslado en el eje Y
 */
Trasladar(mueveX, mueveY){
    this.posX1 += mueveX;
    this.posY1 += mueveY;
    this.posX2 += mueveX;
    this.posY2 += mueveY;
    this.posX3 += mueveX;
    this.posY3 += mueveY;
}

/**
 * Girar el triángulo calculando su centroide
 * @param {number} angulo - Ángulo en grados de giro
 */
Girar(angulo){
    let anguloR = angulo * Math.PI / 180;

    //Giro de las tres coordenadas
    let posX1g = Math.round(this.posX1 * Math.cos(anguloR) - this.posY1 * Math.sin(anguloR));
    let posY1g = Math.round(this.posX1 * Math.sin(anguloR) + this.posY1 * Math.cos(anguloR));
    let posX2g = Math.round(this.posX2 * Math.cos(anguloR) - this.posY2 * Math.sin(anguloR));
    let posY2g = Math.round(this.posX2 * Math.sin(anguloR) + this.posY2 * Math.cos(anguloR));
    let posX3g = Math.round(this.posX3 * Math.cos(anguloR) - this.posY3 * Math.sin(anguloR));
    let posY3g = Math.round(this.posX3 * Math.sin(anguloR) + this.posY3 * Math.cos(anguloR));

    //Giro del centroide
    let centroX = (this.posX1 + this.posX2 + this.posX3) / 3;
    let centroY = (this.posY1 + this.posY2 + this.posY3) / 3;
    let centroXg = Math.round(centroX * Math.cos(anguloR) - centroY * Math.sin(anguloR));
    let centroYg = Math.round(centroX * Math.sin(anguloR) + centroY * Math.cos(anguloR));

    //¿Cuánto se desplazó el centroide?
    let desplazaX = centroXg - centroX;
    let desplazaY = centroYg - centroY;

    //Retira el desplazamiento
    this.posX1 = posX1g - desplazaX;
    this.posY1 = posY1g - desplazaY;
    this.posX2 = posX2g - desplazaX;
    this.posY2 = posY2g - desplazaY;
    this.posX3 = posX3g - desplazaX;
    this.posY3 = posY3g - desplazaY;
}
}

/** Clase para mantener un conjunto de rectángulos
 * @Author Rafael Alberto Moreno Parra
 * @Version 1.0 (05 de enero de 2024)
 * */
class Rectangulo {
    /**
     * Constructor
     * @param {number} posX - Valor X de la coordenada superior izquierda
     * @param {number} posY - Valor Y de la coordenada superior izquierda
     * @param {number} Ancho - Ancho del rectángulo
     * @param {number} Alto - Alto del rectángulo
     * @param {number} Tipo - Tipo rectángulo: 1. Sólo perímetro, 2. Relleno, 3. Perímetro de un color y relleno de
otro color
     * @param {number} GrosorBorde - Grosor línea del borde
     * @param {string} ColorBorde - Color expresado en hexadecimal del borde
     * @param {string} ColorRelleno - Color expresado en hexadecimal del relleno
     */
    constructor(posX, posY, Ancho, Alto, Tipo, GrosorBorde, ColorBorde, ColorRelleno){
        this.posX = posX;
        this.posY = posY;
        this.Ancho = Ancho;
        this.Alto = Alto;
        this.Tipo = Tipo; //1. Solo borde, 2. Sólo relleno, 3. Borde y relleno
        this.GrosorBorde = GrosorBorde;
        this.ColorBorde = ColorBorde;
        this.ColorRelleno = ColorRelleno;
    }

    /**
     * Trasladar un rectángulo
     * @param {number} mueveX - Traslado en el eje X
     * @param {number} mueveY - Traslado en el eje Y
     */
    Trasladar(mueveX, mueveY){

```



```

        this.posX += mueveX;
        this.posY += mueveY;
    }
}

/** Clase para mantener un conjunto de polígonos de 4 lados
 * @Author Rafael Alberto Moreno Parra
 * @Version 1.0 (05 de enero de 2024)
 * */
class Poligono4Lados {
    /**
     * Constructor
     * @param {number} posX1 - Valor X de la primera coordenada
     * @param {number} posY1 - Valor Y de la primera coordenada
     * @param {number} posX2 - Valor X de la segunda coordenada
     * @param {number} posY2 - Valor Y de la segunda coordenada
     * @param {number} posX3 - Valor X de la tercera coordenada
     * @param {number} posY3 - Valor Y de la tercera coordenada
     * @param {number} posX4 - Valor X de la cuarta coordenada
     * @param {number} posY4 - Valor Y de la cuarta coordenada
     * @param {number} Tipo - Tipo polígono de 4 lados: 1. Sólo perímetro, 2. Relleno, 3. Perímetro de un color y
relleno de otro color
     * @param {number} GrosorBorde- Grosor línea del borde
     * @param {string} ColorBorde - Color expresado en hexadecimal del borde
     * @param {string} ColorRelleno - Color expresado en hexadecimal del relleno
     */
    constructor(posX1, posY1, posX2, posY2, posX3, posY3, posX4, posY4, Tipo, GrosorBorde, ColorBorde, ColorRelleno){
        this.posX1 = posX1;
        this.posY1 = posY1;
        this.posX2 = posX2;
        this.posY2 = posY2;
        this.posX3 = posX3;
        this.posY3 = posY3;
        this.posX4 = posX4;
        this.posY4 = posY4;
        this.Tipo = Tipo; //1. Solo borde, 2. Sólo relleno, 3. Borde y relleno
        this.GrosorBorde = GrosorBorde;
        this.ColorBorde = ColorBorde;
        this.ColorRelleno = ColorRelleno;
    }

    /**
     * Trasladar un polígono de 4 lados
     * @param {number} mueveX - Traslado en el eje X
     * @param {number} mueveY - Traslado en el eje Y
     */
    Trasladar(mueveX, mueveY){
        this.posX1 += mueveX;
        this.posY1 += mueveY;
        this.posX2 += mueveX;
        this.posY2 += mueveY;
        this.posX3 += mueveX;
        this.posY3 += mueveY;
        this.posX4 += mueveX;
        this.posY4 += mueveY;
    }

    /**
     * Girar el polígono de 4 lados calculando su centroide
     * @param {number} angulo - Ángulo en grados de giro
     */
    Girar(angulo){
        let anguloR = angulo * Math.PI / 180;

        //Giro de las cuatro coordenadas
        let posX1g = Math.round(this.posX1 * Math.cos(anguloR) - this.posY1 * Math.sin(anguloR));
        let posY1g = Math.round(this.posX1 * Math.sin(anguloR) + this.posY1 * Math.cos(anguloR));
        let posX2g = Math.round(this.posX2 * Math.cos(anguloR) - this.posY2 * Math.sin(anguloR));
        let posY2g = Math.round(this.posX2 * Math.sin(anguloR) + this.posY2 * Math.cos(anguloR));
        let posX3g = Math.round(this.posX3 * Math.cos(anguloR) - this.posY3 * Math.sin(anguloR));
        let posY3g = Math.round(this.posX3 * Math.sin(anguloR) + this.posY3 * Math.cos(anguloR));
        let posX4g = Math.round(this.posX4 * Math.cos(anguloR) - this.posY4 * Math.sin(anguloR));
        let posY4g = Math.round(this.posX4 * Math.sin(anguloR) + this.posY4 * Math.cos(anguloR));

        //Giro del centroide
        let centroX = (this.posX1 + this.posX2 + this.posX3 + this.posX4) / 4;
        let centroY = (this.posY1 + this.posY2 + this.posY3 + this.posY4) / 4;
        let centroXg = Math.round(centroX * Math.cos(anguloR) - centroY * Math.sin(anguloR));
        let centroYg = Math.round(centroX * Math.sin(anguloR) + centroY * Math.cos(anguloR));

        //¿Cuánto se desplazó el centroide?
        let desplazaX = centroXg - centroX;
        let desplazaY = centroYg - centroY;

        //Retira el desplazamiento
        this.posX1 = posX1g - desplazaX;
        this.posY1 = posY1g - desplazaY;
    }
}

```

```

        this.posX2 = posX2g - desplazaX;
        this.posY2 = posY2g - desplazaY;
        this.posX3 = posX3g - desplazaX;
        this.posY3 = posY3g - desplazaY;
        this.posX4 = posX4g - desplazaX;
        this.posY4 = posY4g - desplazaY;
    }
}

/** Clase para mantener un conjunto de elipses
 * @Author Rafael Alberto Moreno Parra
 * @Version 1.0 (05 de enero de 2024)
 * */
class Elipse {
    /**
     * Constructor
     * @param {number} posX - Posición X de la coordenada del centro de la elipse
     * @param {number} posY - Posición Y de la coordenada del centro de la elipse
     * @param {number} RadioX - Longitud del radio en el eje X
     * @param {number} RadioY - Longitud del radio en el eje Y
     * @param {number} RotaFigura - Giro de la elipse
     * @param {number} angIni - Ángulo inicial de dibujado de la elipse
     * @param {number} angFin - Ángulo final de dibujado de la elipse
     * @param {boolean} ContraReloj - Si el dibujado es contrareloj es true
     * @param {number} Tipo - Tipo elipse: 1. Sólo el borde, 2. Relleno, 3. Borde de un color y relleno de otro color
     * @param {number} GrosorBorde - Grosor del borde
     * @param {string} ColorBorde - Color expresado en hexadecimal del borde
     * @param {string} ColorRelleno - Color expresado en hexadecimal del relleno
     */
    constructor(posX, posY, RadioX, RadioY, RotaFigura, angIni, angFin, ContraReloj, Tipo, GrosorBorde, ColorBorde, ColorRelleno){
        this.posX = posX;
        this.posY = posY;
        this.RadioX = RadioX;
        this.RadioY = RadioY;
        this.RotaFigura = RotaFigura;
        this.angIni = angIni;
        this.angFin = angFin;
        this.Reloj = ContraReloj;
        this.Tipo = Tipo; //1. Solo borde, 2. Sólo relleno, 3. Borde y relleno
        this.GrosorBorde = GrosorBorde;
        this.ColorBorde = ColorBorde;
        this.ColorRelleno = ColorRelleno;
    }

    /**
     * Trasladar una elipse
     * @param {number} mueveX - Traslado en el eje X
     * @param {number} mueveY - Traslado en el eje Y
     */
    Trasladar(mueveX, mueveY){
        this.posX += mueveX;
        this.posY += mueveY;
    }
}

/** Clase para mantener un conjunto de círculos
 * @Author Rafael Alberto Moreno Parra
 * @Version 1.0 (05 de enero de 2024)
 * */
class Circulo {
    /**
     * Constructor
     * @param {number} posX - Posición X de la coordenada del centro del círculo
     * @param {number} posY - Posición Y de la coordenada del centro del círculo
     * @param {number} Radio - Radio del círculo
     * @param {number} Tipo - Tipo de círculo: 1. Sólo color en el perímetro, 2. Relleno, 3. Relleno de un color y
    perímetro de otro color
     * @param {number} GrosorBorde - Grosor del perímetro
     * @param {string} ColorBorde - Color expresado en hexadecimal del borde
     * @param {string} ColorRelleno - Color expresado en hexadecimal del relleno
     */
    constructor(posX, posY, Radio, Tipo, GrosorBorde, ColorBorde, ColorRelleno){
        this.posX = posX;
        this.posY = posY;
        this.Radio = Radio;
        this.Tipo = Tipo; //1. Solo borde, 2. Sólo relleno, 3. Borde y relleno
        this.GrosorBorde = GrosorBorde;
        this.ColorBorde = ColorBorde;
        this.ColorRelleno = ColorRelleno;
    }

    /**
     * Trasladar un círculo
     * @param {number} mueveX - Traslado en el eje X
     * @param {number} mueveY - Traslado en el eje Y
     */

```

```
Trasladar(mueveX, mueveY){
    this.posX += mueveX;
    this.posY += mueveY;
}
}
```

Y esta es la forma de usar esa librería:

291.html

```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="1200" height="800"></canvas>
<script src="LibreriaGrafica.js"></script>
<script>
//Objeto para hacer los gráficos
let Graficos = new Grafica(document.getElementById('milienzo'));

//Borde y fondo del lienzo
let ColorRelleno = 'blue';
let GrosorBorde = 10;
let ColorBorde = 'red';
Graficos.LienzoFondoBorde(ColorRelleno, GrosorBorde, ColorBorde);

//Dibuja la línea Xini, Yini, Xfin, Yfin, Grosor, Color
let ColorLinea = Graficos.ColorRGB(22, 184, 168); //En formato RGB
let Xini = 150, Yini = 70;
let Xfin = 340, Yfin = 350;
let Grosor = 5;
Graficos.Linea(Xini, Yini, Xfin, Yfin, Grosor, ColorLinea);
</script>
</body></html>
```

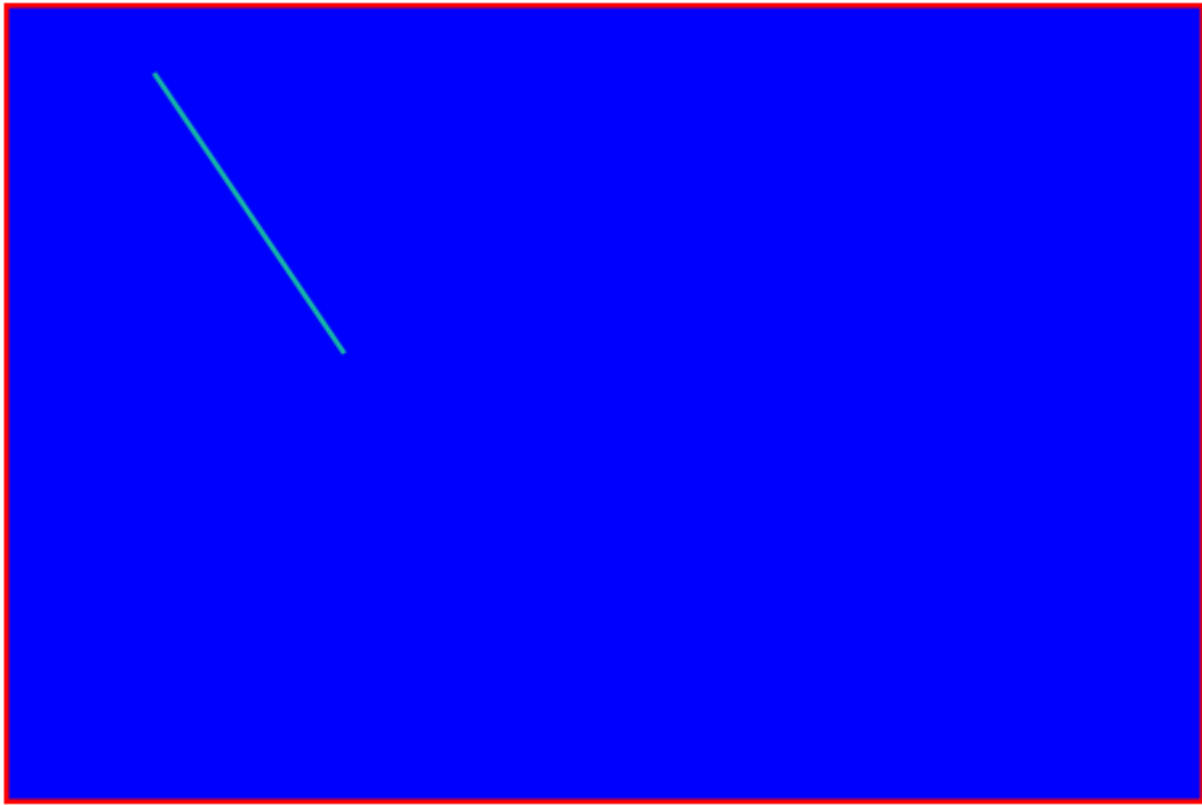


Ilustración 61: Uso de librería gráfica

```

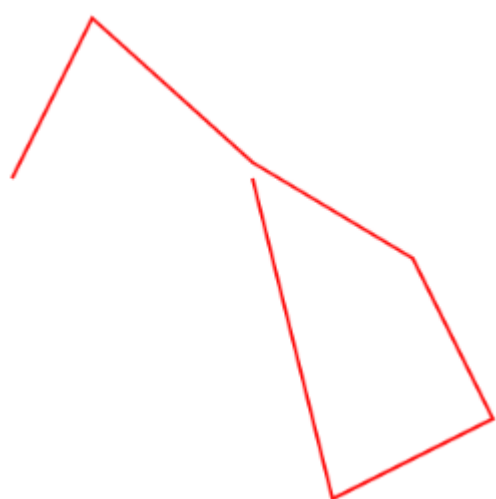
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="1200" height="800"></canvas>
<script src="LibreriaGrafica.js"></script>
<script>
//Objeto para hacer los gráficos
let Graficos = new Grafica(document.getElementById('milienzo'));

//Trabaja con una lista de puntos
let Puntos = [];
Puntos.push(new Punto(150, 150));
Puntos.push(new Punto(200, 50));
Puntos.push(new Punto(300, 140));
Puntos.push(new Punto(400, 200));
Puntos.push(new Punto(450, 300));
Puntos.push(new Punto(350, 350));
Puntos.push(new Punto(300, 150));

let Color = Graficos.ColorRGB(254, 0, 0); //Color en formato RGB
let GrosorLinea = 2;

//Dibuja las líneas
Graficos.LineasPuntos(Puntos, GrosorLinea, Color);
</script></body></html>

```



*Ilustración 62: Uso de librería gráfica*

```

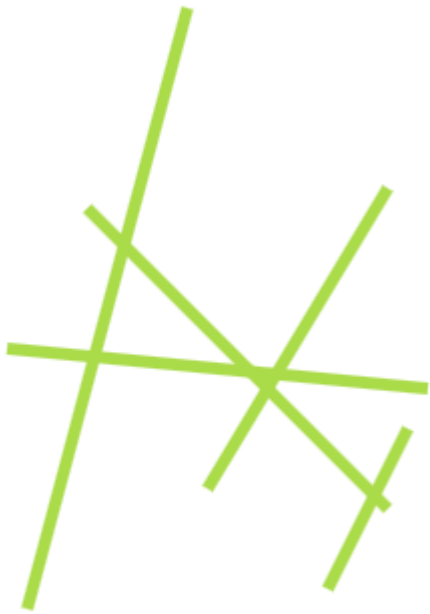
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="1200" height="800"></canvas>
<script src="LibreriaGrafica.js"></script>
<script src="AzarLibreria.js"></script>
<script>
//Objeto para hacer los gráficos
let Graficos = new Grafica(document.getElementById('milienzo'));

let Color = Graficos.ColorAzar(); //Algún color al azar
let GrosorLinea = Azar.EnteroEntre(2, 10); //Grueso entre 1 y 10

//Trabaja con una lista de líneas
let Lineas = [];
Lineas.push(new Linea(150, 150, 300, 300, GrosorLinea, Color)); //Xini, Yini, Xfin, Yfin, Grueso, Color
Lineas.push(new Linea(200, 50, 120, 350, GrosorLinea, Color));
Lineas.push(new Linea(300, 140, 210, 290, GrosorLinea, Color));
Lineas.push(new Linea(320, 240, 110, 220, GrosorLinea, Color));
Lineas.push(new Linea(270, 340, 310, 260, GrosorLinea, Color));

//Dibuja las líneas: Lista
Graficos.LineasListas(Lineas);
</script></body></html>

```



*Ilustración 63: Uso de librería gráfica*

```

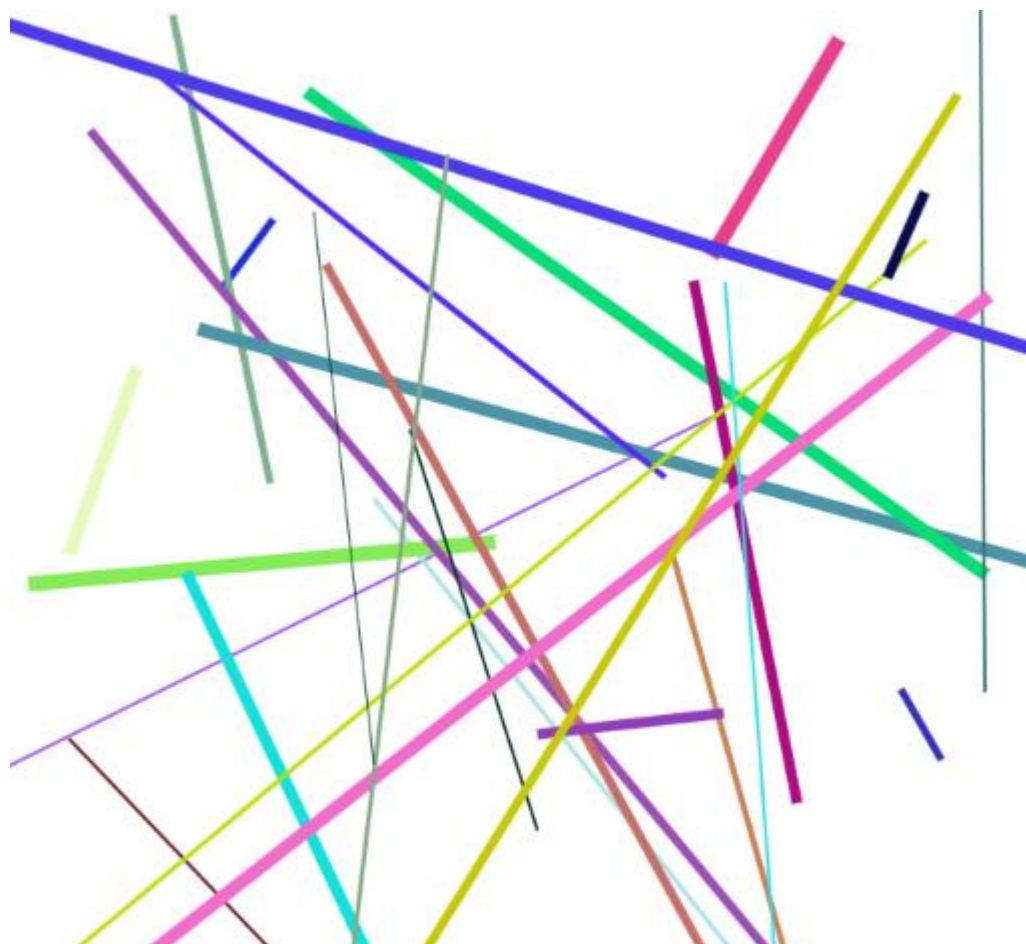
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="1200" height="800"></canvas>
<script src="LibreriaGrafica.js"></script>
<script src="AzarLibreria.js"></script>
<script>
//Objeto para hacer los gráficos
let Graficos = new Grafica(document.getElementById('milienzo'));

//Lista de líneas
let Lineas = [];
for (let cont = 1; cont <= 30; cont++){
    let Color = Graficos.ColorAzar(); //Algún color al azar
    let Grueso = Azar.EnteroEntre(1, 10); //Grueso entre 1 y 10

    //Trabaja con una lista de líneas
    let Xini = Azar.EnteroEntre(1, 800); let Yini = Azar.EnteroEntre(1, 800);
    let Xfin = Azar.EnteroEntre(1, 800); let Yfin = Azar.EnteroEntre(1, 800);
    Lineas.push(new Linea(Xini, Yini, Xfin, Yfin, Grueso, Color));
}

//Dibuja las líneas: Lista
Graficos.LineasListas(Lineas);
</script></body></html>

```



*Ilustración 64: Uso de librería gráfica*



```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="1200" height="800"></canvas>
<script src="LibreriaGrafica.js"></script>
<script>
//Objeto para hacer los gráficos
let Graficos = new Grafica(document.getElementById('milienzo'));

//Dibuja un triángulo: posX1, posY1, posX2, posY2, posX3, posY3, Grosor, Color
let ColorBorde = 'blue';
let posX1 = 10, posY1 = 10;
let posX2 = 90, posY2 = 130;
let posX3 = 220, posY3 = 30;
let Grosor = 4;
Graficos.Triangulo(posX1, posY1, posX2, posY2, posX3, posY3, Grosor, ColorBorde);

//Dibuja un triángulo relleno: posX1, posY1, posX2, posY2, posX3, posY3, ColorRelleno
let ColorRelleno = 'gray';
posX1 = 10; posY1 = 210;
posX2 = 190; posY2 = 330;
posX3 = 420; posY3 = 130;
Graficos.TrianguloRelleno(posX1, posY1, posX2, posY2, posX3, posY3, ColorRelleno);

//Dibuja un triángulo relleno con borde: posX1, posY1, posX2, posY2, posX3, posY3, ColorRelleno,
GrosorBorde, ColorBorde
posX1 = 10; posY1 = 370;
posX2 = 190; posY2 = 460;
posX3 = 420; posY3 = 380;
Graficos.TrianguloRellenoBorde(posX1, posY1, posX2, posY2, posX3, posY3, ColorRelleno, Grosor,
ColorBorde);
</script></body></html>
```

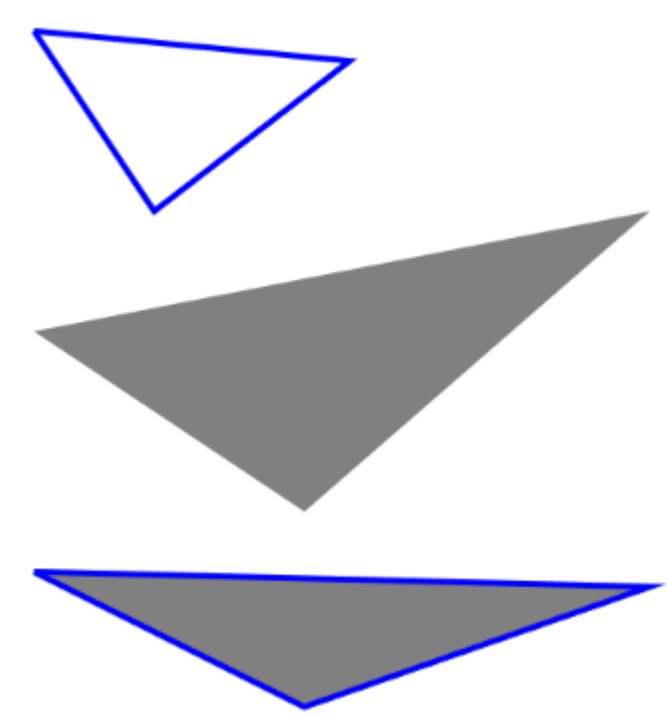


Ilustración 65: Uso de librería gráfica



```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="1200" height="800"></canvas>
<script src="LibreriaGrafica.js"></script>
<script src="AzarLibreria.js"></script>
<script>
//Objeto para hacer los gráficos
let Graficos = new Grafica(document.getElementById('milienzo'));

//Lista de triángulos
let Triangulos = [];

//Genera triángulos al azar
for (let cont = 1; cont <= 5; cont++){
    let ColorRelleno = Graficos.ColorAzar(); //Algún color al azar
    let ColorBorde = Graficos.ColorAzar(); //Algún color al azar
    let GrosorBorde = Azar.EnteroEntre(1, 10); //Grueso entre 1 y 10

    //Coordenadas de las tres esquinas del triángulo
    let posX1 = Azar.EnteroEntre(1, 700); let posY1 = Azar.EnteroEntre(1, 700);
    let posX2 = Azar.EnteroEntre(1, 700); let posY2 = Azar.EnteroEntre(1, 700);
    let posX3 = Azar.EnteroEntre(1, 700); let posY3 = Azar.EnteroEntre(1, 700);

    //Tipo de triángulo: 1. Solo borde, 2. Sólo relleno, 3. Borde y relleno
    let Tipo = Azar.EnteroEntre(1, 3);

    Triangulos.push(new Triangulo(posX1, posY1, posX2, posY2, posX3, posY3, Tipo, GrosorBorde,
ColorBorde, ColorRelleno));
}

Graficos.TrianguloLista(Triangulos);
</script></body></html>
```

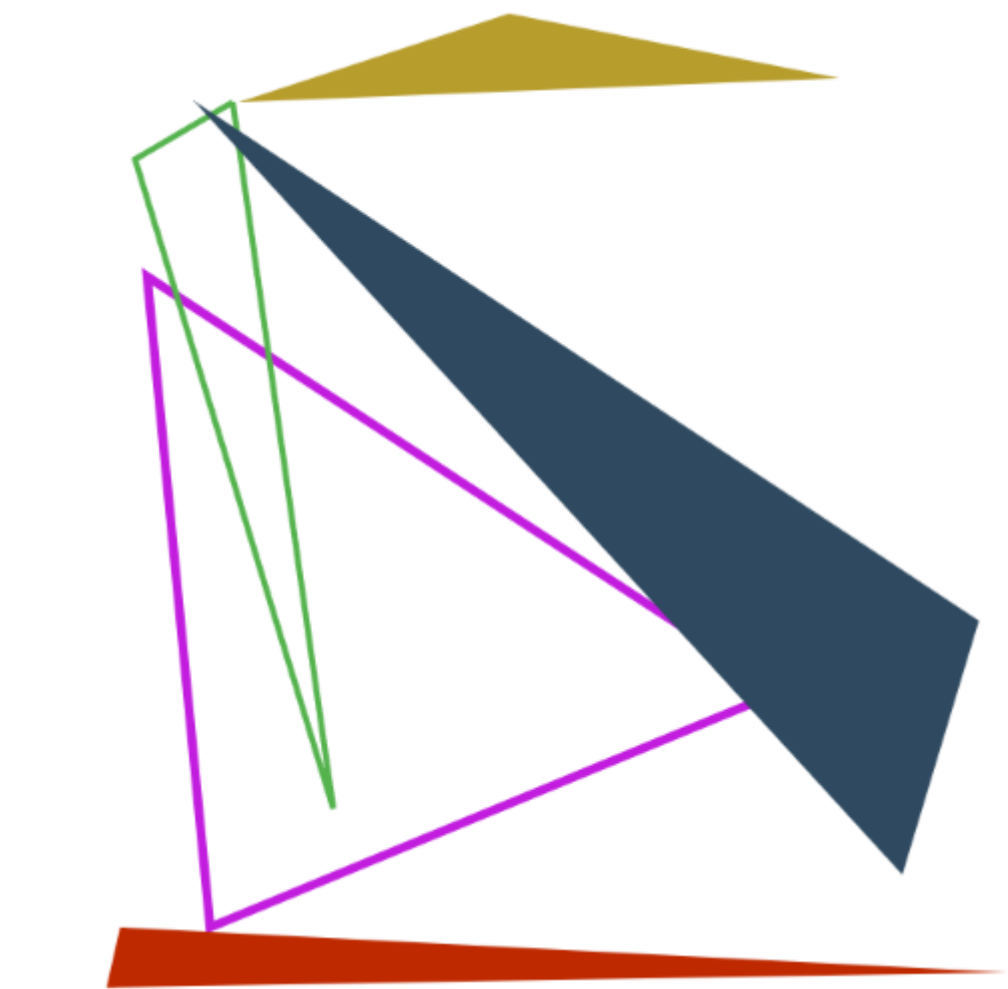


Ilustración 66: Uso de librería gráfica



```
<!DOCTYPE HTML><html lang="es"><head></head><body>
<canvas id="milienzo" width="1200" height="800"></canvas>
<script src="LibreriaGrafica.js"></script>
<script>
//Objeto para hacer los gráficos
let Graficos = new Grafica(document.getElementById('milienzo'));

//Dibuja un rectángulo: posX, posY, Ancho, Alto, Grosor, Color
let ColorBorde = 'blue';
let posX = 10, posY = 10;
let Ancho = 90, Alto = 50;
let Grosor = 4;
Graficos.Rectangulo(posX, posY, Ancho, Alto, Grosor, ColorBorde);

//Dibuja un rectángulo relleno: posX, posY, Ancho, Alto, ColorRelleno
let ColorRelleno='gray';
posX = 10; posY = 210;
Graficos.RectanguloRelleno(posX, posY, Ancho, Alto, ColorRelleno);

//Dibuja un rectángulo relleno con borde: posX, posY, Ancho, Alto, ColorRelleno, GrosorBorde, ColorBorde
posX = 10; posY = 370;
Graficos.RectanguloRellenoBorde(posX, posY, Ancho, Alto, ColorRelleno, Grosor, ColorBorde);

//Un objeto de tipo rectángulo
posX = 150;
posY = 550;
let Tipo = 3;
ColorRelleno = Graficos.ColorAzar();
ColorBorde = Graficos.ColorAzar();
MiRectangulo = new Rectangulo(posX, posY, Ancho, Alto, Tipo, Grosor, ColorBorde, ColorRelleno);
Graficos.RectanguloObj(MiRectangulo);
MiRectangulo.Trasladar(400, 100);
Graficos.RectanguloObj(MiRectangulo);
</script></body></html>
```



Ilustración 67: Uso de librería gráfica

De 007.html a 073.html hay diversos ejemplos de uso de la librería.

- [1] W3Schools, «JavaScript Tutorial,» julio 2025. [En línea]. Available: <https://www.w3schools.com/js/>. [Último acceso: 2026].
- [2] Mozilla, «JavaScript,» julio 2025. [En línea]. Available: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>. [Último acceso: 2026].
- [3] W3Techs, «Usage of client-side programming languages for websites,» 2026. [En línea]. Available: [https://w3techs.com/technologies/overview/client\\_side\\_language/all](https://w3techs.com/technologies/overview/client_side_language/all). [Último acceso: 2026].
- [4] Adobe, «Flash,» diciembre 2020. [En línea]. Available: <https://www.adobe.com/co/products/flashplayer/end-of-life-alternative.html?msockid=0d2ac1506dfe6a15329cd7496c516b12>. [Último acceso: 2026].
- [5] ECMA International, «ECMAScript® 2026 Language Specification,» 18 diciembre 2025. [En línea]. Available: <https://tc39.es/ecma262/>. [Último acceso: 15 enero 2026].
- [6] W3Schools, «ECMAScript 2026,» 2026. [En línea]. Available: [https://www.w3schools.com/js/js\\_2026.asp](https://www.w3schools.com/js/js_2026.asp). [Último acceso: 15 enero 2026].
- [7] Mozilla, «What is a web server?,» 2026. [En línea]. Available: [https://developer.mozilla.org/en-US/docs/Learn/Common\\_questions/Web\\_mechanics/What\\_is\\_a\\_web\\_server](https://developer.mozilla.org/en-US/docs/Learn/Common_questions/Web_mechanics/What_is_a_web_server). [Último acceso: 2026].
- [8] M. Rouse, «database management system (DBMS),» 2026. [En línea]. Available: <https://www.techtarget.com/searchdatamanagement/definition/database-management-system>. [Último acceso: 2026].
- [9] MariaDB, «MariaDB,» 2026. [En línea]. Available: <https://mariadb.org/>. [Último acceso: 2026].
- [10] MySQL, «MySQL,» 2026. [En línea]. Available: <https://www.mysql.com/>. [Último acceso: 2026].